

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ ІНТЕЛЕКТУАЛЬНИХ ТЕХНОЛОГІЙ І
ЗВ'ЯЗКУ
Факультет Телекомунікацій та радіотехніки

Іщенко А.Д., к.т.н. Русаловський В.Б.

Програмування мобільних платформ

Від першого застосунку до повноцінного проєкту на Kotlin та Jetpack
Compose

Навчальний посібник

*для студентів закладів вищої освіти, які навчаються за спеціальностями
галузі знань «Інженерія, виробництво та будівництво» та «Інформаційні
технології»*

*Рекомендовано вченою радою Державного університету інтелектуальних технологій і
зв'язку (протокол № ____ від « ____ » _____ 2026 р.)*

Одеса – 2026

УДК 004.42(075.8)

Рекомендовано до друку вченою радою Державного університету інтелектуальних технологій і зв'язку (протокол № ____ від « ____ » _____ 2026 р.)

Рецензенти:

[Прізвище І. Б.] – доктор технічних наук, професор, [посада, установа];

[Прізвище І. Б.] – кандидат технічних наук, доцент, [посада, установа].

Програмування мобільних платформ. Від першого застосунку до повноцінного проєкту на Kotlin та Jetpack Compose : навч. посіб. / А. Д. Іщенко, В. Б. Русаловський. – Одеса : ДУІТЗ, 2026. – 85 с.

Навчальний посібник призначений для факультативного курсу «Програмування мобільних платформ». Стисло розглянуто створення нативних Android-застосунків мовою Kotlin з інтерфейсом на Jetpack Compose: основи мови Kotlin, побудову інтерфейсу, роботу зі станом, архітектуру MVVM, локальне збереження даних (Room, DataStore), роботу з REST API, Bluetooth Low Energy, геолокацію, карти, камеру, сенсори, сповіщення, фонові задачі, авторизацію, базову безпеку та тестування. Матеріал подано на прикладі наскрізного проєкту – застосунку персонального моніторингу здоров'я «Smart Life». Кожен розділ містить теоретичні відомості, ключові фрагменти коду, схеми, скриншоти результату, типові помилки, практичні завдання та контрольні питання; повний код проєкту постачається в git-репозиторії. Для студентів закладів вищої освіти, викладачів і всіх, хто починає вивчати мобільну розробку.

ISBN [ISBN]

УДК 004.42(075.8)

© Іщенко А. Д., Русаловський В. Б., 2026

© ДУІТЗ, 2026

ЗМІСТ

Вступ	5
Розділ 1. Вступ до мобільної розробки та основи Kotlin	7
1.1 Мета розділу та результати навчання	7
1.2 Мобільні платформи та архітектура Android	7
1.3 Середовище розробки та перший застосунок	10
1.4 Основи мови Kotlin	12
1.5 Типові помилки	14
1.6 Практичні завдання	15
1.7 Контрольні питання	15
Розділ 2. Інтерфейс, стан і архітектура застосунку	16
2.1 Мета розділу та результати навчання	16
2.2 Декларативний інтерфейс і навігація	16
2.3 Стан застосунку і ViewModel	20
2.4 Архітектура MVVM і впровадження залежностей	22
2.5 Типові помилки	25
2.6 Практичні завдання	25
2.7 Контрольні питання	26
Розділ 3. Робота з даними: локальне сховище та мережа	27
3.1 Мета розділу та результати навчання	27
3.2 Асинхронність: корутини та потоки Flow	27
3.3 Локальне зберігання даних: Room і DataStore	29
3.4 Мережа: REST API та Retrofit	32
3.5 Типові помилки	35
3.6 Практичні завдання	35
3.7 Контрольні питання	35
Розділ 4. Апаратні можливості пристрою	36
4.1 Мета розділу та результати навчання	36
4.2 Дозволи Android	37
4.3 Bluetooth Low Energy	38
4.4 Геолокація і карти	41
4.5 Камера і галерея	46
4.6 Сенсори	48
4.7 Типові помилки	50
4.8 Практичні завдання	50
4.9 Контрольні питання	51
Розділ 5. Сповіщення, фонові задачі, авторизація та безпека	52
5.1 Мета розділу та результати навчання	52

5.2 Сповіщення.....	52
5.3 Фонові задачі.....	55
5.4 Авторизація та робота з користувачами.....	58
5.5 Безпека мобільних застосунків.....	61
5.6 Типові помилки.....	64
5.7 Практичні завдання	64
5.8 Контрольні питання.....	65
Розділ 6. Тестування та фінальний проєкт «Smart Life».....	66
6.1 Мета розділу та результати навчання.....	66
6.2 Тестування мобільного застосунку.....	66
6.3 Фінальний проєкт Smart Life 1.0	68
6.4 Типові помилки.....	71
6.5 Практичні завдання	71
6.6 Контрольні питання.....	72
Висновки.....	73
Список використаних джерел.....	74
Додаток А. Структура фінального проєкту.....	76
Додаток Б. Інструкція з запуску проєкту	77
Додаток В. Створення APK	78
Додаток Г. Тестування на фізичному Android-пристрої.....	79
Додаток Д. Контрольні питання до курсу	80
Додаток Е. Практичні (лабораторні) роботи.....	81
Додаток Ж. Варіанти індивідуальних проєктів	82
Додаток К. Критерії оцінювання фінального проєкту	83
Додаток Л. Глосарій основних термінів.....	84
Додаток М. Використані бібліотеки	85

ВСТУП

Мобільні застосунки стали основним способом взаємодії людей із цифровими сервісами, тому навички мобільної розробки є важливою складовою підготовки фахівців з інформаційних технологій. Посібник призначений для факультативного курсу «Програмування мобільних платформ» і розрахований на студентів, які знають основи програмування (змінні, умови, цикли, функції), але ще не створювали Android-застосунків.

Мета курсу – сформувати практичні навички створення нативних Android-застосунків мовою Kotlin: від першого простого застосунку до повноцінного проєкту, що поєднує інтерфейс, локальні дані, мережу та апаратні можливості смартфона. Завдання курсу: ознайомити з платформою Android та інструментами розробки; навчити основ Kotlin, побудови інтерфейсу на Jetpack Compose і керування станом; навчити проєктувати застосунок за архітектурою MVVM; сформувати вміння працювати з базою даних, REST API, авторизацією, Bluetooth LE, геолокацією, картами, камерою, сенсорами, сповіщеннями і фоновими задачами; ознайомити з базовими принципами безпеки і тестування.

Головна ідея курсу – **один застосунок, що росте разом зі студентом**. Починаючи з розділу 2, кожен підрозділ додає до застосунку моніторингу здоров'я «Smart Life» нову можливість, а розділ 6 підсумовує фінальну версію 1.0.

Посібник складається з шести розділів. Розділ 1 знайомить з мобільними платформами, архітектурою Android, середовищем розробки та мовою Kotlin. Розділ 2 присвячено побудові інтерфейсу на Jetpack Compose, роботі зі станом і архітектурі MVVM. У розділі 3 розглянуто корутини, локальне зберігання даних і роботу з мережею. Розділ 4 охоплює апаратні можливості пристрою: дозволи, Bluetooth LE, геолокацію й карти, камеру і сенсори. Розділ 5 присвячено сповіщенням, фоновим задачам, авторизації та безпеці, а розділ 6 – тестуванню та фінальному проєкту.

Кожен розділ починається з мети і таблиці результатів навчання та основних понять. Далі йдуть тематичні підрозділи, у яких теоретичні відомості супроводжуються схемами і таблицями, а практична частина містить опис прикладу, послідовність створення, ключові фрагменти коду з поясненням, інструкцію запуску на емуляторі та скріншоти результату з описом. Завершують розділ типові помилки, практичні завдання, контрольні питання.

Таблиця 1 – Технологічний стек курсу

Компонент	Версія	Призначення
Android Studio / JDK	2025.3.3 Patch 1 (Panda 3)+ / 17+	середовище розробки
Gradle / AGP / Kotlin	9.7.1 / 9.4.1 / 2.4.20	збірка і мова
compileSdk / targetSdk / minSdk	37 / 36 / 26	версії Android API
Compose BOM, Navigation	2026.09.00, 2.10.1	інтерфейс і навігація
Lifecycle, Room, DataStore	2.11.0, 2.8.5, 1.2.1	стан і локальні дані
Retrofit + kotlinx.serialization	3.0.0 / 1.11.0	REST API та JSON
Play services location, osmdroid	21.4.0, 6.1.20	геолокація і карта
Coil, WorkManager	3.6.3, 2.11.2	зображення, фонові задачі

Вихідний код і умовні позначення. Щоб посібник залишався компактним, у тексті наведено лише **ключові фрагменти** коду; пропущені частини позначено коментарем `// ...` з описом реалізованої логіки. Повний код постачається разом із посібником у вигляді трьох проєктів: `HelloMobile/` (підрозділ 1.3), `KotlinBasics/` (підрозділ 1.4) і `SmartLife/` – git-репозиторій, у якому стан проєкту після кожного етапу збережено міткою `stage-03 ... stage-18` (фінальна версія – `v1.0`). Список міток виводить команда `git tag`, команда `git checkout <мітка>` відновлює відповідний стан проєкту, а `git diff <мітка1> <мітка2>` показує зміни між двома етапами. Шляхи файлів наводяться від пакета `ua.edu.mobile.smartlife` (`app/src/main/java/ua/edu/mobile/smartlife/`).

Усі скріншоти застосунків – реальні знімки Android Emulator (профіль Medium Phone, Android 17, API 37). Системні діалоги показано англійською, бо така мова емулятора. Знімки середовища Android Studio позначено «Скріншот для виконання студентом»: інтерфейс IDE залежить від версії, тому студент виконує їх самостійно. Схеми побудовано в нотації UML засобами PlantUML (вихідні файли *.puml додано до посібника; їх можна відкрити й відредагувати на сайті planttext.com).

РОЗДІЛ 1. ВСТУП ДО МОБІЛЬНОЇ РОЗРОБКИ ТА ОСНОВИ KOTLIN

1.1 Мета розділу та результати навчання

Мета розділу – ознайомити з особливостями мобільних платформ і будовою операційної системи Android, навчити працювати в середовищі Android Studio, створити й запустити перший застосунок на емуляторі та опанувати синтаксис мови Kotlin в обсязі, потрібному для подальших розділів. Результати навчання за темами розділу наведено в таблиці 1.1, а основні поняття – в таблиці 1.2.

Таблиця 1.1 – Результати навчання за темами розділу

Тема	Що студент знає і вміє після опрацювання
Платформа Android і середовище розробки	Види мобільних застосунків (нативні, кросплатформні, вебзастосунки); складові платформи Android; призначення Android Studio, SDK, емулятора і Gradle; структуру Android-проекту; життєвий цикл Activity; як запустити застосунок на емуляторі.
Мова Kotlin	Різницю між <code>val</code> і <code>var</code> ; базові типи і рядкові шаблони; вирази <code>if/when</code> ; цикли <code>for/while</code> і діапазони; функції з параметрами за замовчуванням і лямбди; класи, <code>data class</code> , <code>enum class</code> , <code>object</code> ; <code>nullable</code> -типи й оператори <code>?.</code> , <code>?:</code> , <code>!!</code> ; колекції та функції <code>filter</code> , <code>map</code> , <code>sumOf</code> .

Таблиця 1.2 – Основні поняття розділу

Поняття	Визначення
Android SDK	набір бібліотек та інструментів для створення застосунків під певну версію Android (API level)
AVD (емулятор)	віртуальний Android-пристрій для запуску застосунку на комп'ютері
Gradle, AGP	система збірки та плагін, що вміє збирати Android-проекти
AndroidManifest.xml	«паспорт» застосунку: назва, іконка, Activity, дозволи
Composable-функція	функція з анотацією <code>@Composable</code> , що описує частину інтерфейсу
<code>val / var</code>	незмінне / змінне посилання на значення
<code>when</code>	вираз вибору, заміна ланцюжка <code>if-else</code>
лямбда	анонімна функція <code>{ x -> x * 2 }</code> , яку передають як параметр
<code>nullable</code> -тип	тип із <code>?</code> , що допускає значення <code>null</code>
функція-розширення	функція, «додана» до існуючого класу: <code>fun Double.format()</code>

1.2 Мобільні платформи та архітектура Android

Сучасний ринок смартфонів фактично поділено між двома платформами: **Android** від Google, під керуванням якої працює понад 70 % смартфонів у світі, та **iOS** від Apple. Android є відкритою платформою: її вихідний код (AOSP) доступний виробникам, тому вона працює на пристроях сотень брендів – від бюджетних телефонів до годинників, телевізорів і автомобільних систем. Це дає розробнику величезну аудиторію, але вимагає враховувати різноманіття екранів, версій системи і апаратних можливостей.

Мобільний застосунок працює в умовах, яких немає на настільному комп'ютері: невеликий сенсорний екран, обмежений заряд батареї, нестабільна мережа, постійні переривання (дзвінок, сповіщення, перемикання між застосунками). Операційна система може будь-коли зупинити застосунок, щоб звільнити пам'ять, тому розробник має зберігати стан і коректно реагувати на події життєвого циклу. Доступ до камери, геолокації чи контактів захищений дозволами, які користувач надає явно.

Розрізняють три підходи до створення мобільних застосунків (табл. 1.3). **Нативні** застосунки пишуть мовою і засобами самої платформи (Kotlin для Android, Swift для iOS), вони мають повний доступ до апаратури і найкращу продуктивність. **Кросплатформні** фреймворки (Flutter, React Native, Kotlin Multiplatform) дозволяють писати один код для кількох платформ. **Вебзастосунки (PWA)** працюють у браузері й мають обмежений доступ до пристрою. У курсі використано нативний підхід – Kotlin і Jetpack Compose є рекомендованим Google стеком для нових Android-застосунків, а отримані знання легко переносяться на Kotlin Multiplatform.

Таблиця 1.3 – Порівняння підходів до мобільної розробки

Критерій	Нативний	Кросплатформний	Вебзастосунок (PWA)
Мова і засоби	Kotlin, Android SDK	Dart (Flutter), JavaScript (React Native), Kotlin Multiplatform	HTML, CSS, JavaScript
Продуктивність	найвища	близька до нативної	залежить від браузера
Доступ до апаратури	повний	через плагіни	обмежений
Кількість кодових баз	окрема для кожної платформи	одна спільна	одна
Поширення	Google Play, APK	магазини застосунків	за посиланням у браузері

Операційна система Android побудована шарами (рис. 1.1). В основі лежить **ядро Linux**, що керує процесами, пам'яттю, живленням і драйверами пристроїв. **Рівень апаратної абстракції (HAL)** надає стандартні інтерфейси до камери, Bluetooth, сенсорів і GPS, завдяки чому однаковий код працює на обладнанні різних виробників. Над ним розташовані нативні бібліотеки (SQLite, OpenGL ES, медіакодеки) та середовище виконання **ART**, яке виконує код застосунків. Розробник працює з верхнім рівнем – **Java/Kotlin API Framework**: це системні сервіси та класи, через які застосунок показує інтерфейс, надсилає сповіщення, отримує координати чи дані сенсорів.

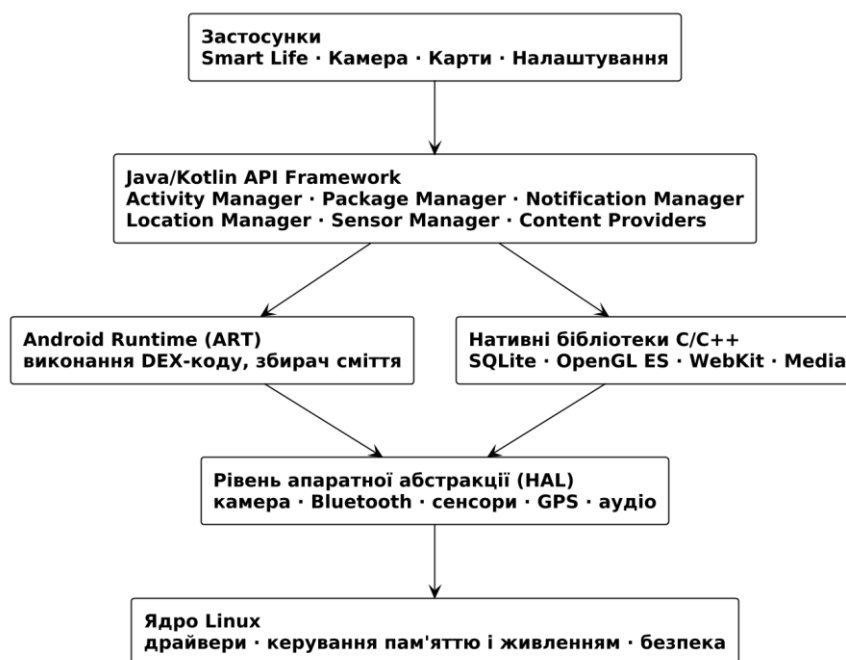


Рисунок 1.1 – Архітектура платформи Android

Схема на рисунку 1.1 читається згори вниз: кожен рівень користується послугами рівня під ним. Наш застосунок Smart Life звертається лише до класів Framework – наприклад, `SensorManager` для сенсорів – а вже Framework через ART, бібліотеки і HAL доходить до драйвера конкретного датчика в ядрі Linux.

Кожен Android-застосунок складається з **компонентів**, які система створює і викликає сама (табл. 1.4). Компоненти оголошуються в маніфесті `AndroidManifest.xml` – «паспорті» застосунку, де також перелічено дозволи, іконку і назву.

Таблиця 1.4 – Компоненти Android-застосунку

Компонент	Призначення	Приклад у Smart Life
Activity	екран, з яким взаємодіє користувач; точка входу в застосунок	MainActivity з інтерфейсом на Compose
Service	тривала робота без інтерфейсу	службу використовує WorkManager для фонових задач
BroadcastReceiver	реакція на системні або власні широкомовні	WaterActionReceiver – кнопка

Компонент	Призначення	Приклад у Smart Life
	повідомлення	в сповіщенні
ContentProvider	надання даних іншим застосункам	FileProvider передає фото камері

Android швидко розвивається: щороку виходить нова версія системи з новим **рівнем API** (API level) – цілим числом, що позначає набір доступних програмних інтерфейсів. У файлі збірки задають три параметри: `minSdk` – найстаріша версія, на якій застосунок встановиться; `targetSdk` – версія, під поведінку якої застосунок адаптовано; `compileSdk` – версія SDK, з якою компілюється код. Відповідність версій і рівнів API, важливих для курсу, наведено в таблиці 1.5.

Таблиця 1.5 – Версії Android і рівні API

Версія Android	Рівень API	Зміни, важливі для курсу
8.0	26	канали сповіщень; мінімальна версія Smart Life (<code>minSdk</code>)
12	31	окремі дозволи Bluetooth, вибір приблизної геолокації, обов'язковий <code>FLAG_IMMUTABLE</code>
13	33	дозвіл на сповіщення <code>POST_NOTIFICATIONS</code> , системний Photo Picker
16	36	цільова версія Smart Life (<code>targetSdk</code>)
17	37	версія SDK для компіляції (<code>compileSdk</code>) і образ емулятора в посібнику

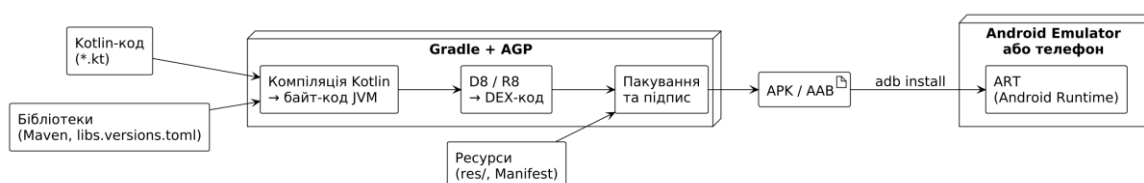


Рисунок 1.2 – Шлях від вихідного коду до застосунку на пристрої

Рисунок 1.2 читається зліва направо. Вихідні файли Kotlin разом із бібліотеками потрапляють у Gradle, де плагін AGP спочатку компілює їх у байт-код JVM, потім інструмент D8 (або R8 для релізу) перетворює байт-код у формат DEX, зрозумілий Android. На етапі пакування до DEX-коду додаються ресурси і маніфест, а готовий файл підписується. Отриманий APK встановлюється на емулятор або телефон командою `adb install` і виконується середовищем ART.

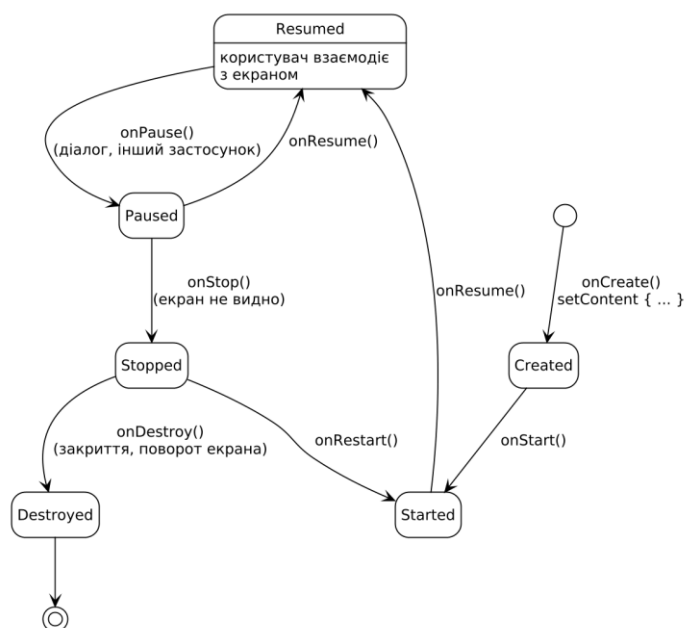


Рисунок 1.3 – Життєвий цикл Activity

На рисунку 1.3 стани Activity показано прямокутниками, а переходи між ними підписано методами, які викликає система. Після запуску Activity послідовно проходить стани Created, Started і Resumed – у стані Resumed користувач взаємодіє з екраном. Якщо поверх з'являється діалог або інший застосунок, Activity переходить у Paused, а коли її зовсім не видно – у Stopped; з цих станів вона може повернутися назад. Стан Destroyed настає при закритті екрана, а також під час повороту, після якого Activity створюється заново.

Життєвий цикл пояснює одну з найпоширеніших проблем початківців: під час повороту екрана Activity знищується і створюється заново, тож усі дані, збережені лише в її полях, втрачаються. Способи зберегти стан розглянуто в підрозділі 2.3.

1.3 Середовище розробки та перший застосунок

Android Studio – офіційне середовище розробки Android-застосунків на основі IntelliJ IDEA. Разом з ним встановлюються **Android SDK** (бібліотеки платформи й інструменти командного рядка, зокрема adb – міст між комп'ютером і пристроєм), вбудований JDK та **Android Emulator**. Основні інструменти середовища, якими користуються в курсі, наведено в таблиці 1.6.

Таблиця 1.6 – Основні інструменти Android Studio

Інструмент	Призначення
Редактор коду і Design/Preview	написання коду, перегляд composable-функцій з анотацією @Preview без запуску
Device Manager	створення і запуск емуляторів, підключені телефони
SDK Manager	встановлення платформ Android, інструментів збірки, образів емулятора
Logcat	журнал повідомлень застосунку і системи з фільтрацією за тегом і рівнем
App Inspection	перегляд бази даних (Database Inspector), фонових задач і мережевих запитів
Gradle (панель і Build Output)	запуск задач збірки, перегляд помилок компіляції

Проект збирає **Gradle** з плагіном **Android Gradle Plugin (AGP)**. Скрипти збірки пишуться мовою Kotlin (файли *.gradle.kts), а версії всіх бібліотек зібрано в одному **каталозі версій** gradle/libs.versions.toml, звідки на них посилаються як libs.androidx.core.ktx. Така організація дозволяє оновлювати бібліотеку в одному місці для всього проекту. Під час першої синхронізації Gradle завантажує потрібну версію самого себе (через Gradle Wrapper) і всі залежності з репозиторіїв Google Maven і Maven Central.

Практичний приклад. Застосунок HelloMobile показує привітання, кнопку і лічильник натискань. На ньому вивчаємо структуру проекту, Compose-функції та поняття стану.

Спочатку встановіть Android Studio: разом із середовищем встановлюються Android SDK, вбудований JDK та емулятор. У вікні *SDK Manager* додатково встановіть платформу Android SDK Platform 37, для якої компілюватиметься проєкт.

Далі створіть проєкт командою *File ▶ New ▶ New Project* і оберіть шаблон *Empty Activity*. У полі *Name* вкажіть *HelloMobile*, у полі *Package name* – *ua.edu.mobile.hellomobile*, мінімальну версію SDK оберіть *API 26*, а мовою скриптів збірки – *Kotlin DSL*. Після натискання *Finish* Android Studio запускає синхронізацію Gradle; перший раз вона триває кілька хвилин, бо завантажуються всі залежності.

Поки триває синхронізація, створіть віртуальний пристрій: у вікні *Device Manager* додайте емулятор з профілем *Medium Phone* і образом Android 17 (API 37). Коли проєкт синхронізовано, замініть вміст файлу *MainActivity.kt* кодом, наведеним нижче у цьому підрозділі і запустіть застосунок кнопкою *Run* (Shift+F10).

У створеному проєкті варто одразу розібратися з основними файлами. Файл *settings.gradle.kts* перелічує модулі проєкту і репозиторії бібліотек, *gradle/libs.versions.toml* є каталогом версій усіх залежностей, *app/build.gradle.kts* задає версії SDK і підключені бібліотеки модуля. У папці *app/src/main* лежать маніфест

AndroidManifest.xml, Kotlin-код у папці java/ і ресурси в папці res/ (рядки, іконки, теми).

Ключові фрагменти реалізації наведено нижче.

Файл

HelloMobile/app/src/main/java/ua/edu/mobile/hellomobile/MainActivity.kt:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent { // інтерфейс описується Compose-функціями
            HelloMobileTheme {
                Scaffold(modifier.fillMaxSize()) { padding ->
                    Greeting("студенте", Modifier.padding(padding))
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    var clicks by remember { mutableIntStateOf(0) } // стан екрана
    Column(modifier.fillMaxSize(), horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center) {
        Text("Привіт, $name!", style = MaterialTheme.typography.headlineMedium)
        // ... текст-підзаголовок і відступи Spacer
        Button(onClick = { clicks++ }) { Text("Натисни мене") }
        Text("Кількість натискань: $clicks")
    }
}
```

Клас MainActivity успадковує ComponentActivity – базовий клас Activity з підтримкою Jetpack Compose. Система викликає його метод onCreate() під час створення екрана. Першим рядком викликається реалізація батьківського класу, далі enableEdgeToEdge() дозволяє інтерфейсу займати весь екран, включно з областю під рядком стану.

Виклик setContent { ... } замінює собою класичну XML-розмітку: усередині фігурних дужок інтерфейс описується викликами composable-функцій. HelloMobileTheme застосовує кольори і шрифти теми Material 3, а Scaffold створює каркас екрана і передає у лямбду параметр padding – відступи від системних панелей, які потрібно застосувати до вмісту, щоб текст не опинився під рядком стану.

Функція Greeting позначена анотацією @Composable, тобто описує частину інтерфейсу. Рядок var clicks by remember { mutableIntStateOf(0) } створює стан – ціле число, яке Compose запам'ятовує між перемальовуваннями. Column розміщує дочірні елементи вертикально і вирівнює їх по центру екрана. Кнопка Button в обробнику onClick збільшує clicks на одиницю; оскільки змінилося значення стану, Compose автоматично повторно викликає (рекомпонує) ті функції, які його читають, і текст «Кількість натискань» оновлюється без жодного додаткового коду. Функцію з анотацією @Preview Android Studio показує у вікні Design без запуску емулятора.

Запуск на емуляторі. Оберіть емулятор у списку пристроїв на панелі інструментів і натисніть Run. Gradle збере APK, Android Studio встановить його через adb і запустить MainActivity. Перший запуск емулятора триває 1–2 хвилини; не закривайте його між запусками.

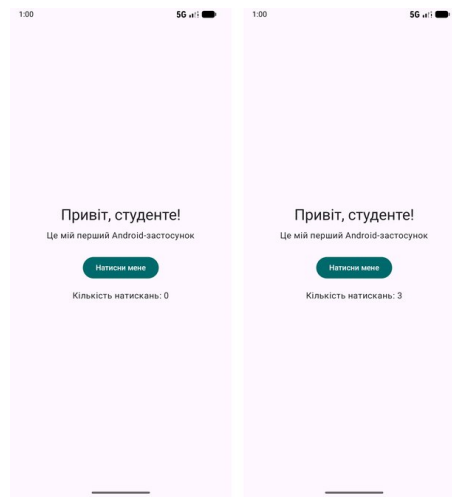


Рисунок 1.4 – Застосунок HelloMobile: після запуску (ліворуч) і після трьох натискань кнопки (праворуч)

На рисунку 1.4 ліворуч застосунок одразу після запуску: по центру екрана розміщено заголовок «Привіт, студенте!», підзаголовок, кнопку «Натисни мене» і текст «Кількість натискань: 0». Праворуч – той самий екран після трьох натискань: змінився лише рядок лічильника, що демонструє автоматичне оновлення інтерфейсу при зміні стану.

1.4 Основи мови Kotlin

Kotlin – статично типізована мова від JetBrains, з 2019 року основна мова Android-розробки. Компілюється в байт-код JVM, повністю сумісна з Java. Головні переваги: лаконічність, виведення типів, **null-безпека** (компілятор не дозволяє звернутися до значення, яке може бути null, без перевірки) та функціональний стиль роботи з колекціями.

Незмінні змінні оголошуються через `val`, змінні – через `var`. Рекомендовано за замовчуванням використовувати `val`. Тип виводиться автоматично: `val steps = 5000` має тип `Int`. У рядки підставляють значення шаблонами: "Кроків: `$steps`", "ІМТ: `${bmi(70.0, 1.75)}`". Вирази `if` і `when` повертають значення, тому ними можна ініціалізувати змінні.

data class автоматично отримує `equals`, `hashCode`, `toString` і `copy` – ідеально для моделей даних. **enum class** описує обмежений набір значень (тип запису: вода, пульс, сон). Змінна типу `String?` може містити null; безпечний виклик `name?.length` повертає null замість падіння, а оператор Елвіса `?:` задає значення за замовчуванням.

Kotlin є статично типізованою мовою: тип кожної змінної відомий під час компіляції, хоча його часто не пишуть явно, бо компілятор виводить тип зі значення. Основні вбудовані типи наведено в таблиці 1.7. Усі вони є повноцінними об'єктами з методами, наприклад `"72".toInt()` або `3.7.roundToInt()`, а перетворення між числовими типами виконується лише явно.

Таблиця 1.7 – Основні типи даних Kotlin

Тип	Приклад значення	Використання в Smart Life
Int, Long	72, 1_695_000_000_000L	пульс, ідентифікатор запису, час у мілісекундах
Double, Float	0.25, 9.81f	об'єм води, значення сенсорів
Boolean	true	чи ввімкнено нагадування
String, Char	"Вода", 'S'	назви, нотатки
List<T>, Map<K, V>, Set<T>	listOf(1, 2)	список записів, групування за типом
T?	null	координати, яких може не бути

Функції в Kotlin оголошуються словом `fun`; якщо тіло складається з одного виразу, його записують після знака `=`. Параметри можуть мати значення за замовчуванням і передаватися за іменем, що замінює численні перевантаження. Функції є «громадянами першого класу»: їх можна зберігати в змінних і передавати як параметри у вигляді **лямбд** `{ x -> x * 2 }`. Саме на лямбдах побудовано Jetpack Compose (`Button(onClick = { ... })`) та обробку колекцій.

Об'єктно-орієнтовані можливості Kotlin знайомі з інших мов, але мають важливі відмінності: класи за замовчуванням закриті для наслідування (потрібне слово `open`), властивості оголошуються прямо в конструкторі, а `data class` автоматично отримує методи для роботи з даними. **Інтерфейси** можуть містити реалізацію методів за замовчуванням. **Sealed-ієрархії** (`sealed interface`) описують закритий набір варіантів, наприклад стани завантаження, і компілятор перевіряє, що в `when` оброблено кожен з них. **Функції-розширення** додають методи до існуючих класів без наслідування: `fun Double.format(): String` можна викликати як `0.25.format()`.

Особливе місце займають **функції області видимості** (`scope functions`) стандартної бібліотеки. Вони виконують блок коду в контексті об'єкта і роблять код компактнішим; їх відмінності підсумовано в таблиці 1.8.

Таблиця 1.8 – Функції області видимості Kotlin

Функція	Звернення до об'єкта	Повертає	Типове використання
<code>let</code>	<code>it</code>	результат блоку	дія лише для не-null значення: <code>x?.let { ... }</code>
<code>apply</code>	<code>this</code>	сам об'єкт	налаштування об'єкта після створення
<code>also</code>	<code>it</code>	сам об'єкт	додаткова дія (логування) без зміни ланцюжка
<code>run</code>	<code>this</code>	результат блоку	обчислення з використанням властивостей об'єкта

Нарешті, для асинхронної роботи Kotlin має **корутини** – легкі задачі, які можна призупинити й продовжити, не блокуючи потік. Функції, що можуть призупинятися, позначають словом `suspend`. Корутини детально розглянуто в підрозділі 3.2, а в цьому розділі достатньо знати, що мережеві запити та робота з базою в Android виконуються саме в них.

Практичний приклад. Консольний проєкт `KotlinBasics` містить дев'ять файлів (`Variables.kt`, `Conditions.kt`, `Loops.kt`, `Functions.kt`, `Classes.kt`, `NullSafety.kt`, `Collections.kt`, `Objects.kt`, `Main.kt`), кожен ілюструє одну тему на даних Smart Life: кроки, воду, пульс, сон.

Створіть консольний проєкт командою `File ▸ New ▸ New Project ▸ Kotlin` (система збірки `Gradle`) з назвою `KotlinBasics` або відкрийте готовий проєкт з матеріалів посібника. У папці `src/main/kotlin` створіть пакет `basics` і для кожної теми окремий файл з функцією-демонстрацією (`variablesDemo()`, `conditionsDemo()` тощо). У файлі `Main.kt` функція `main()` по черзі викликає всі демонстрації і друкує заголовок перед кожною. Запускається програма зеленою стрілкою біля `fun main()`, а результат з'являється у вікні *Run*.

Ключові фрагменти реалізації наведено нижче.

```
// Classes.kt - клас із поведінкою і data class для даних
class UserProfile(val name: String, var age: Int) {
    var dailyStepGoal: Int = 8000
    fun introduce() = "Я $name, мені $age р., моя ціль - $dailyStepGoal кроків"
}

data class HealthRecord(val id: Int, val title: String, val value: Double, val unit: String)

// Conditions.kt - when як вираз з діапазонами
fun temperatureLevel(celsius: Double): String = when {
    celsius < 0 -> "мороз"
    celsius in 0.0..15.0 -> "прохолодно"
    celsius in 15.0..25.0 -> "комфортно"
    else -> "спекотно"
}

// Functions.kt - параметр за замовчуванням
```

```
fun greet(name: String, greeting: String = "Привіт") = "$greeting, $name!"

// NullSafety.kt - безпечна робота з null
var nickname: String? = null
println("Ім'я: ${nickname ?: "Гість"}") // ?: - оператор Елвіса
email?.let { println("Надсилаємо лист на $it") } // let - лише для не-null

// Collections.kt - функціональна обробка списку лямбдами
val totalWater = records.filter { it.title == "Вода" }.sumOf { it.value }
val byTitle = records.groupBy { it.title } // Map<String, List<HealthRecord>>
// ... Loops.kt, Objects.kt: цикли з діапазонами, object, companion object,
// sealed interface LoadState і функція-розширення String.capitalizeFirst()
```

Клас `UserProfile` показує звичайний клас Kotlin: властивості `name` і `age` оголошено прямо в заголовку (первинному конструкторі), причому `name` незмінна (`val`), а вік можна змінювати (`var`). Властивість `dailyStepGoal` має значення за замовчуванням, а метод `introduce()` повертає рядок, у який шаблонами `$name` і `$age` підставлено значення властивостей.

`HealthRecord` оголошено як `data class`: компілятор автоматично генерує для нього `toString()` (саме тому запис можна одразу надрукувати), порівняння `equals()` за значеннями полів і метод `copy()`, який створює змінену копію, наприклад `record.copy(value = 2.0)`.

Функція `temperatureLevel` використовує `when` без аргументу: гілки перевіряються зверху вниз, і перша істинна умова визначає результат. Вираз `celsius in 0.0..15.0` перевіряє належність діапазону. Оскільки `when` тут є виразом, його результат одразу повертається з функції. Функцію `greet` можна викликати з одним аргументом – тоді параметр `greeting` отримає значення за замовчуванням «Привіт».

У фрагменті `null`-безпеки змінна `nickname` має тип `String?`, тобто може містити `null`. Оператор Елвіса `?:` повертає ліве значення, якщо воно не `null`, інакше – праве («Гість»). Безпечний виклик `email?.let { ... }` виконає блок лише для непорожнього значення. Останній фрагмент показує обробку колекції ланцюжком функцій: `filter` залишає лише записи води, а `sumOf` підсумовує їх значення; `groupBy` розкладає записи в словник за назвою показника.

Запуск на емуляторі. Приклади підрозділу – консольні і виконуються на JVM без емулятора: результат з'являється у вікні *Run* Android Studio або IntelliJ IDEA.

Фрагмент реального виводу програми (вікно *Run*):

```
===== 2.3 Умови =====
Чудова активність!
Час прогулятися
===== 2.5 Функції =====
ІМТ: 22,9
Привіт, Олено!
===== 2.6-2.7 Класи та data class =====
Я Олена, мені 20 р., моя ціль - 10000 кроків
```

У фрагменті видно результати кількох демонстрацій: функція оцінки кроків вивела «Чудова активність!» для великої кількості кроків і «Час прогулятися» для малої, функція ІМТ повернула 22,9, функція `greet` з одним аргументом використала привітання за замовчуванням, а метод `introduce()` сформував речення з властивостей об'єкта.

1.5 Типові помилки

Середовище розробки і перший запуск. Найчастіша помилка першого запуску – повідомлення *SDK location not found*: Gradle не знає, де встановлено Android SDK. Досить відкрити проєкт в Android Studio, і вона сама створить файл `local.properties` зі шляхом до SDK. Якщо синхронізація завершується помилкою *Gradle sync failed*, найчастіше причина в мережі: перевірте підключення до інтернету і повторіть *Sync Project with Gradle Files*.

Якщо емулятор не запускається або працює дуже повільно, у BIOS не ввімкнено апаратну віртуалізацію (VT-x або AMD-V) чи не встановлено *Android Emulator Hypervisor Driver*. Нарешті, студенти часто помічають, що лічильник скидається після повороту

екрана: `remember` зберігає значення лише доти, доки існує `Activity`, а поворот її перестворює. Виправлення – `rememberSaveable` або `ViewModel` (підрозділ 2.3).

Мова Kotlin. Помилка компіляції *Val cannot be reassigned* означає спробу змінити значення, оголошене через `val`: або оголосить змінну як `var`, або створить нове значення. Повідомлення *Only safe (?) or non-null asserted (!!) calls are allowed* з'являється, коли до nullable-значення звертаються без перевірки – використайте `?.`, `?:` або перевірку `if (x != null)`, після якої компілятор сам «розумно» приведе тип. Оператор `!!` дозволяє обійти перевірку, але при значенні `null` програма впаде з `NullPointerException`, тому ним варто користуватися лише тоді, коли `null` неможливий. Нарешті, якщо `when` використовується як вираз, компілятор вимагає гілку `else` або обробку всіх можливих варіантів.

1.6 Практичні завдання

Практичні завдання розділу наведено в таблиці 1.9. Основне завдання виконується на основі версії `Smart Life`, отриманої в підрозділі, а завдання для самостійної роботи поглиблює тему і може оцінюватися додатково.

Таблиця 1.9 – Практичні завдання розділу

Тема	Практичне завдання	Завдання для самостійної роботи
Перший застосунок	Додайте до <code>HelloMobile</code> кнопку «Скинути», яка обнуляє лічильник, і текст, що змінюється при досягненні 10 натискань.	Змініть кольорову тему застосунку у файлі <code>ui/theme/Color.kt</code> , а назву – у <code>res/values/strings.xml</code> . Запустіть застосунок у темній темі емулятора.
Основний Kotlin	Напишіть функцію <code>sleepQuality(hours: Double): String</code> з <code>when</code> (менше 6 – «недостатньо», 6–9 – «норма», більше 9 – «забагато») і виведіть оцінку для списку записів сну.	Створіть <code>data class DaySummary</code> і функцію-розширення <code>List<HealthRecord>.summary()</code> , що рахує воду за день, середній пульс і останній сон.

1.7 Контрольні питання

1. Чим нативний застосунок відрізняється від кросплатформного?
2. Яке призначення `Gradle` і файлу `libs.versions.toml`?
3. Що описує `AndroidManifest.xml`?
4. Назвіть методи життєвого циклу `Activity`.
5. Що таке стан у `Compose` і як він пов'язаний з рекомпозицією?
6. Чим `val` відрізняється від `var`?
7. Що генерує компілятор для `data class`?
8. Як працюють оператори `?.` і `?:`?
9. Чим `filter` відрізняється від `map`?
10. Коли доречно використовувати `enum class`?

РОЗДІЛ 2. ІНТЕРФЕЙС, СТАН І АРХІТЕКТУРА ЗАСТОСУНКУ

2.1 Мета розділу та результати навчання

Мета розділу – навчити будувати інтерфейс мобільного застосунку на Jetpack Compose, організовувати навігацію між екранами, правильно зберігати стан і проектувати застосунок за архітектурою MVVM. У результаті створюється каркас застосунку Smart Life, на який у наступних розділах «нализуються» дані, мережа й апаратні можливості. Результати навчання наведено в таблиці 2.1, основні поняття – в таблиці 2.2.

Таблиця 2.1 – Результати навчання за темами розділу

Тема	Що студент знає і вміє після опрацювання
Інтерфейс і навігація	Принцип декларативного UI; макети Column, Row, Box і модифікатори; компоненти Text, Button, TextField, Card, Scaffold, TopAppBar, NavigationBar; списки LazyColumn; діалоги AlertDialog; навігацію Navigation Compose з типобезпечними маршрутами.
Стан застосунку	Функції remember, rememberSaveable, mutableStateOf; принцип підйому стану (state hoisting); односпрямований потік даних; призначення і життєвий цикл ViewModel; StateFlow та collectAsStateWithLifecycle().
Архітектура	Шари UI і Data та їх відповідальність; шаблон MVVM; роль репозиторію; принцип «єдиного джерела правди»; ручне впровадження залежностей (AppContainer) і фабрики ViewModel; структуру пакетів проекту.

Таблиця 2.2 – Основні поняття розділу

Поняття	Визначення
Рекомпозиція	повторний виклик composable-функцій після зміни даних
Modifier	ланцюжок налаштувань розміру, відступів, вигляду й поведінки елемента
Material 3	система дизайну Google: компоненти, кольори, типографіка
NavHost / маршрут	контейнер екранів / опис адреси екрана з аргументами
back stack	стек відкритих екранів, по якому працює кнопка «Назад»
Стан (state)	дані, від яких залежить вигляд інтерфейсу
state hoisting	винесення стану з composable-функції до її викликача
ViewModel	клас, що зберігає стан екрана і переживає поворот
StateFlow	«гарячий» потік з поточним значенням для спостереження за станом
viewModelScope	область корутин, що скасовується разом з ViewModel
Репозиторій	клас, що приховує джерела даних за простим інтерфейсом
єдине джерело правди	один власник кожного типу даних
DI	передача залежностей ззовні, а не створення всередині класу
AppContainer	контейнер, що створює і зберігає залежності застосунку
ViewModelProvider.Factory	фабрика, що вміє створювати ViewModel з параметрами

2.2 Декларативний інтерфейс і навігація

Історично інтерфейс Android-застосунків описували XML-файлами розмітки, а з коду знаходили елементи (`findViewById`) і змінювали їх вручну: встановлювали текст, ховали кнопки, оновлювали списки через адаптери. Такий **імперативний** підхід вимагає багато коду і легко призводить до розбіжностей між даними та екраном. **Jetpack Compose** – сучасний інструментарій Google, у якому інтерфейс описується функціями Kotlin, а синхронізацію з даними бере на себе бібліотека.

У **декларативному** підході розробник описує, як має виглядати екран для поточних даних, а не змінює елементи вручну. Коли дані змінюються, Compose повторно викликає потрібні функції (**рекомпозиція**). Інтерфейс складається з вкладених composable-функцій: макети розміщують дочірні елементи (Column – вертикально, Row – горизонтально, Box – шарами), а **модифікатори** (`Modifier.padding(16.dp).fillMaxWidth()`) задають розмір, відступи, фон і реакцію на натискання. Розміри вказують у dp (незалежні від щільності пікселів), шрифти – у sp.

Scaffold – каркас екрана з місцями для верхньої панелі, нижньої навігації, кнопки дії (FAB) і `snackbar`. Довгі списки показує `LazyColumn`: він створює лише видимі елементи. Для переходів між екранами використовують `Navigation Compose`: `NavHost` містить граф екранів, `NavController` виконує переходи, а маршрути описуються `@Serializable`-класами, тому аргументи (наприклад, `recordId`) перевіряються компілятором. Граф навігації `Smart Life` показано на рисунку 2.1.

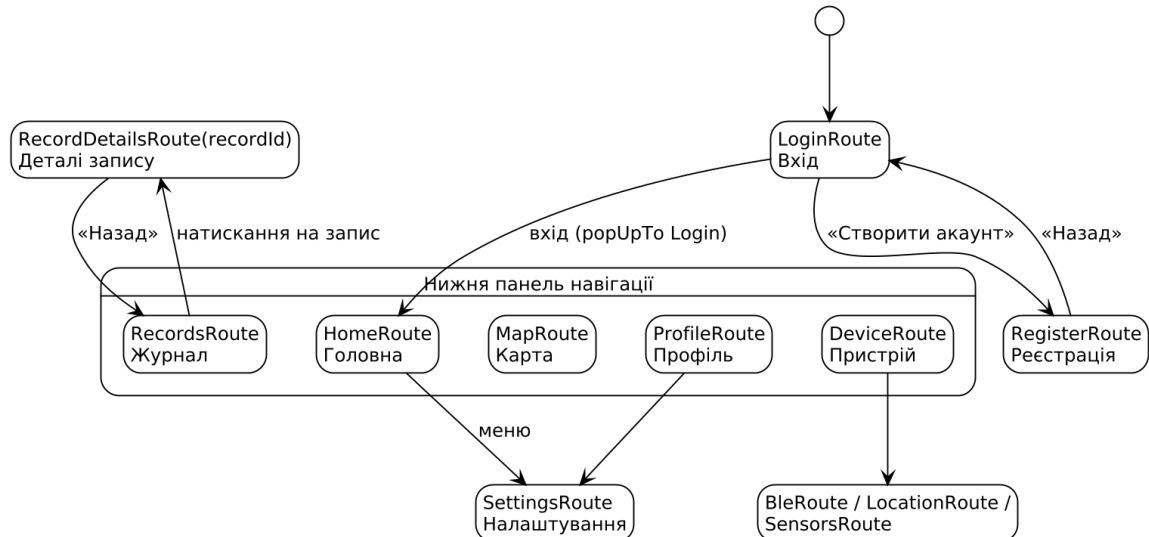


Рисунок 2.1 – Граф навігації фінальної версії `Smart Life`

На рисунку 2.1 кожен прямокутник – це маршрут з назвою екрана, а стрілки – можливі переходи з підписами дій. Застосунок стартує з екрана входу, звідки можна перейти до реєстрації або, після входу, на головний екран. П'ять вкладок усередині рамки «Нижня панель навігації» перемикаються між собою в будь-якому порядку; з журналу відкривається екран деталей запису, з головного екрана і профілю – налаштування, а з вкладки «Пристрій» – екрани Bluetooth, геолокації та сенсорів. У версії цього підрозділу панель має три вкладки, решту буде додано пізніше.

Відображення кадру в `Compose` проходить три фази. Під час **композиції** виконуються `composable`-функції і будується дерево елементів інтерфейсу. На фазі **компонування** (`layout`) кожен елемент вимірює своїх нащадків і визначає розміри та позиції. На фазі **малювання** елементи виводяться на екран. Якщо змінився лише колір, `Compose` може пропустити композицію і компонування, а якщо змінилися дані – повторно виконує лише ті функції, які ці дані читають. На рисунку 2.2 показано дерево функцій головного екрана `Smart Life`.

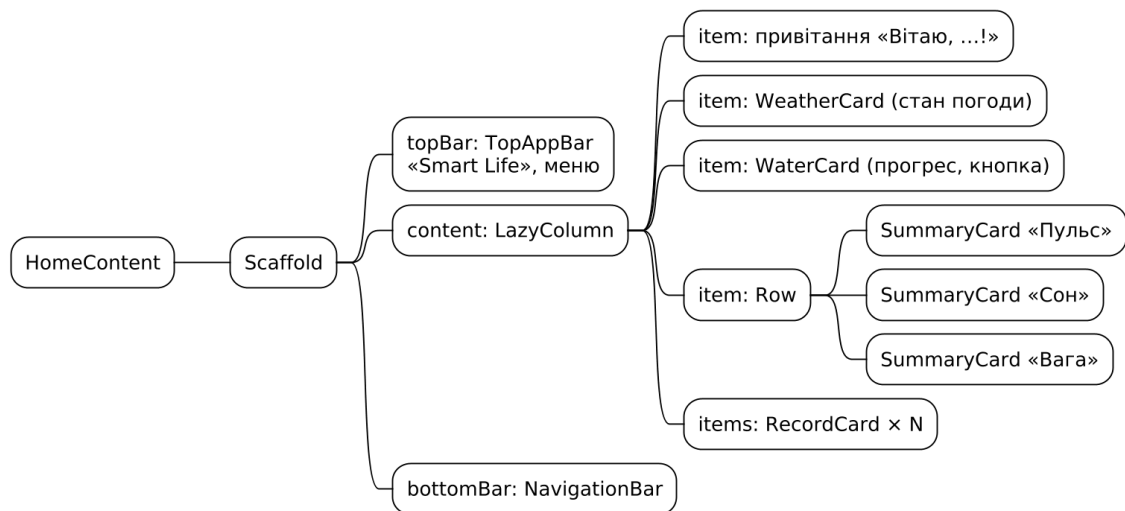


Рисунок 2.2 – Дерево composable-функцій головного екрана

Коренем дерева на рисунку 2.2 є функція `HomeContent`. Вона викликає `Scaffold`, який має три «слоти»: верхню панель `TopAppBar`, вміст і нижню панель навігації. Вміст – це `LazyColumn`, кожен елемент якого (`item`) є окремою гілкою: привітання, картка погоди, картка води, рядок `Row` з трьома картками підсумків і список `RecordCard`. Коли користувач додає склянку води, змінюється лише гілка `WaterCard` – решта дерева не перевиконується.

Для побудови екранів використовують невеликий набір базових функцій (табл. 2.3). Макети відповідають за розташування, компоненти `Material 3` – за вигляд і поведінку стандартних елементів. Тема `MaterialTheme` задає кольорову схему (`colorScheme`), типографіку і форми; усі компоненти беруть кольори з теми, тому перемикання світлої і темної теми не потребує змін у коді екранів.

Таблиця 2.3 – Основні функції Jetpack Compose

Група	Функції	Призначення
Макети	<code>Column</code> , <code>Row</code> , <code>Box</code> , <code>Spacer</code>	розміщення вертикально, горизонтально, шарами; відступ
Списки	<code>LazyColumn</code> , <code>LazyRow</code> , <code>items()</code>	прокручувані списки, що створюють лише видимі елементи
Каркас	<code>Scaffold</code> , <code>TopAppBar</code> , <code>NavigationBar</code> , <code>FloatingActionButton</code>	структура екрана <code>Material 3</code>
Вміст	<code>Text</code> , <code>Icon</code> , <code>Image</code> , <code>Card</code>	текст, іконки, зображення, картки
Введення	<code>Button</code> , <code>TextField</code> , <code>Switch</code> , <code>Slider</code> , <code>FilterChip</code>	кнопки, поля, перемикачі
Діалоги	<code>AlertDialog</code> , <code>ModalBottomSheet</code> , <code>Snackbar</code>	підтвердження, нижні панелі, короткі повідомлення

Модифікатори застосовуються в тому порядку, в якому записані: `Modifier.padding(16.dp).background(Color.Gray)` дасть сірий фон усередині відступу, а `Modifier.background(Color.Gray).padding(16.dp)` – сірий фон разом з відступом. Розміри елементів задають у незалежних від щільності екрана одиницях `dp`, а розміри шрифту – у `sp`, які враховують налаштування розміру тексту користувачем.

Практичний приклад. Прототип `Smart Life`: екран входу з перевіркою полів; головний екран з підсумками дня; журнал показників зі списком карток і діалогом додавання; екран деталей запису; профіль. Нижня панель навігації перемикає розділи «Головна», «Журнал», «Профіль». Дані поки тестові (`FakeData`).

Спершу створіть проект `SmartLife` з пакетом `ua.edu.mobile.smartlife` так само, як `HelloMobile`, і додайте до каталогу версій `libs.versions.toml` бібліотеку `Navigation Compose`, плагін `kotlin-serialization` (він потрібен для типобезпечних маршрутів) і набір іконок `material-icons-extended`.

Потім підготуйте дані. У файлі `data/model/HealthRecord.kt` опишіть перелік типів показників `RecordType` і клас запису `HealthRecord`, а в `data/FakeData.kt` – кілька тестових записів, щоб екрани не були порожніми.

Далі опишіть навігацію: у `ui/navigation/Routes.kt` оголошіть маршрути всіх екранів і список пунктів нижньої панелі. Після цього створіть самі екрани в пакетах `ui/auth`, `ui/home`, `ui/records` і `ui/profile`, а картку запису, яка повторюється на кількох екранах, винесіть у спільний компонент `ui/components/RecordCard.kt`. Наостанок зберіть усе разом у функції `SmartLifeApp()` (файл `ui/SmartLifeApp.kt`) і викличте її з `setContent` у `MainActivity`.

Ключові фрагменти реалізації наведено нижче.

```
// ui/navigation/Routes.kt - типобезпечні маршрути
@Serializable object LoginRoute
@Serializable object HomeRoute
@Serializable object RecordsRoute
@Serializable data class RecordDetailsRoute(val recordId: Long)

// ui/SmartLifeApp.kt - граф навігації
NavHost(navController, startDestination = LoginRoute) {
    composable<LoginRoute> {
        LoginScreen(onLoggedIn = {
            navController.navigate(HomeRoute) {
                popUpTo(LoginRoute) { inclusive = true } // «Назад» не веде на вхід
            }
        })
    }
    composable<RecordsRoute> {
        RecordsScreen(onRecordClick = { id -> navController.navigate(RecordDetailsRoute(id)) })
    }
    // ... Home, Profile, RecordDetails; NavigationBar y bottomBar Scaffold
}

// ui/records/RecordsScreen.kt - список і діалог
Scaffold(
    topBar = { TopAppBar(title = { Text("Журнал показників") }) },
    floatingActionButton = {
        FloatingActionButton(onClick = { showAddDialog = true }) {
            Icon(Icons.Filled.Add, contentDescription = "Додати запис")
        }
    }
) { padding ->
    LazyColumn(Modifier.padding(padding), contentPadding = PaddingValues(16.dp)) {
        items(records, key = { it.id }) { record ->
            RecordCard(record, onClick = { onRecordClick(record.id) })
        }
    }
}

// ... AddRecordDialog: вибір типу, поле значення з перевіркою, нотатка
```

Маршрути оголошено як `@Serializable`-об'єкти і класи. Екрани без параметрів описуються об'єктами (`HomeRoute`), а екран деталей – класом `RecordDetailsRoute` з полем `recordId`; бібліотека навігації сама перетворює такий об'єкт на адресу і назад, тому помилитися з типом аргументу неможливо – це перевіряє компілятор.

У `NavHost` для кожного маршруту викликається `composable<Маршрут> { ... }` з екраном, який треба показати. Зверніть увагу, що екрани не отримують `navController`: вони приймають лямбди на кшталт `onLoggedIn` або `onRecordClick`, а що саме відбудеться (перехід, закриття), вирішує `SmartLifeApp`. Після входу виконується `navigate(HomeRoute)` з параметром `popUpTo(LoginRoute) { inclusive = true }`: екран входу видаляється зі стеку, тому кнопка «Назад» на головному екрані закриває застосунок, а не повертає до форми входу.

Список журналу побудовано на `LazyColumn`, який створює лише ті картки, що видно на екрані, – це важливо для довгих списків. Параметр `key = { it.id }` повідомляє `Compose`, який елемент є яким, тож при додаванні чи видаленні запису інші картки не перемальовуються даремно. Натискання на картку передає `id` запису нагору, і `SmartLifeApp` відкриває маршрут `RecordDetailsRoute(id)`, а екран деталей читає ідентифікатор через `toRoute()`. На цьому етапі список зберігається в `remember`

{ mutableStateListOf() } усередині екрана, тому додані записи зникають після повороту – це ми виправимо в підрозділі 2.3.

Запуск на емуляторі. Запустіть застосунок, спробуйте увійти з порожніми полями (з'явиться помилка «Введіть логін і пароль»), потім увійдіть з будь-якими непорожніми даними, перейдіть між вкладками, додайте запис кнопкою «+», відкрийте деталі запису і поверніться кнопкою «Назад».

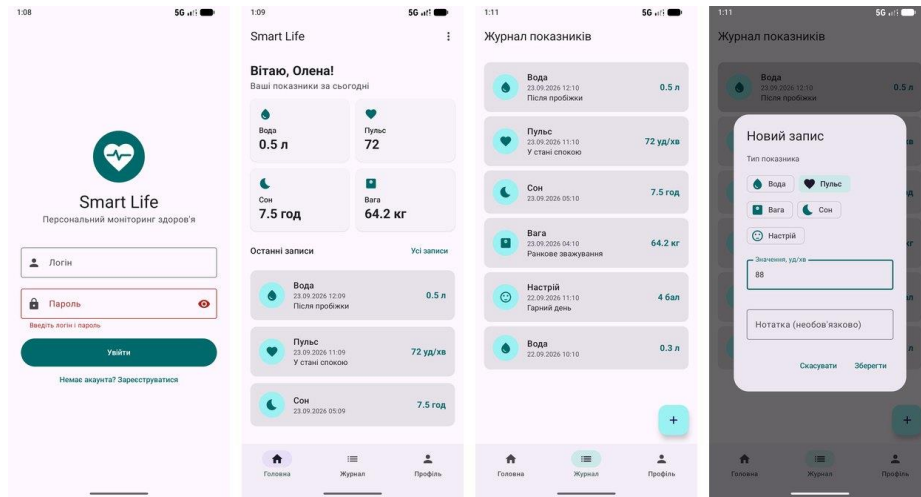


Рисунок 2.3 – Прототип Smart Life: перевірка полів входу, головний екран, журнал, діалог додавання запису

На рисунку 2.3 зліва направо показано чотири екрани прототипу. На екрані входу поля «Логін» і «Пароль» підсвічено червоним, а під ними виведено повідомлення «Введіть логін і пароль» – так спрацьовує перевірка порожніх полів. На головному екрані під привітанням «Вітаю, Олена!» розташовано чотири картки підсумків дня (вода, пульс, сон, вага) і список останніх записів, а внизу – нижня панель з вкладками «Головна», «Журнал», «Профіль». Третій екран – журнал показників: кожна картка містить іконку типу, назву, дату, нотатку і значення з одиницями виміру, а в правому нижньому куті знаходиться кнопка «+». Четвертий екран показує діалог «Новий запис», де обрано тип «Пульс», введено значення 88 і є поле для нотатки.

2.3 Стан застосунку і ViewModel

Стан – будь-які дані, що змінюються з часом і впливають на інтерфейс: текст у полі, список записів, прапорець «йде завантаження». `remember` зберігає значення між рекомпозиціями, але не переживає перестворення `Activity` (поворот, зміна мови); `rememberSaveable` зберігає прості значення в `Bundle` і переживає поворот. Для даних екрана використовують **ViewModel** – об'єкт, який живе довше за `Activity` і знищується лише при остаточному закритті екрана (`onCleared()`).

Стан і події організують за принципом **односпрямованого потоку даних** (UDF, рис. 2.4): стан тече вниз від `ViewModel` до `UI`, події – вгору від `UI` до `ViewModel`. `UI` ніколи не змінює дані напряму. **Підйом стану** означає, що `composable`-функція не зберігає стан сама, а отримує значення і лямбду зміни як параметри – таку функцію легше тестувати й перевикористовувати.

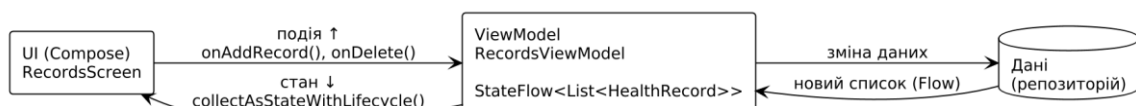


Рисунок 2.4 – Односпрямований потік даних (UDF)

На рисунку 2.4 показано, як рухаються дані між трьома учасниками. `ViewModel` передає екрану стан – список записів – через `collectAsStateWithLifecycle()` (стрілка «стан ↓»). Коли користувач натискає кнопку, екран не змінює дані сам, а надсилає

ViewModel подію, наприклад `onAddRecord()` (стрілка «подія ↑»). ViewModel змінює дані в репозиторії, отримує від нього оновлений список і знову передає його екрану. Дані завжди обертаються по колу в одному напрямку.

ViewModel зберігає стан у `StateFlow` – потоці, який завжди має поточне значення і повідомляє підписників про зміни. Змінний `MutableStateFlow` роблять приватним, назовні віддають лише незмінний `StateFlow`.

Різні способи зберігання стану відрізняються тим, як довго живуть дані (табл. 2.4). Обирати слід найкоротший достатній варіант: прапорець відкритого меню не потребує бази даних, а список записів не можна тримати в `remember`.

Таблиця 2.4 – Способи зберігання стану та час їх життя

Спосіб	Що переживає	Що не переживає	Приклад у Smart Life
<code>remember</code>	рекомпозицію	поворот екрана, закриття	відкрите меню на головному екрані
<code>rememberSaveable</code>	поворот, тимчасове знищення процесу	закриття екрана	текст у полях діалогу запису
ViewModel	поворот екрана	закриття екрана, перезапуск застосунку	стан екрана журналу
Room, DataStore	перезапуск застосунку і пристрою	видалення застосунку	записи, налаштування, сесія

Під час роботи зі станом у Compose важливо дотримуватися двох правил. По-перше, стан має бути **незмінним**: замість зміни списку створюється новий список, інакше Compose не помітить змін. По-друге, **composable-функції** мають бути **без побічних ефектів**: вони можуть викликатися багато разів і в довільному порядку, тому запити до мережі чи бази виконують не в них, а у ViewModel або в спеціальних ефектах `LaunchedEffect` і `DisposableEffect`.

Практичний приклад. У Smart Life список записів переноситься в `RecordsViewModel`, з'являються лічильник води на головному екрані («+ склянка»), видалення запису з підтвердженням і збереження стану після повороту.

Щоб працювати з ViewModel у Compose, додайте до проєкту бібліотеки `lifecycle-viewmodel-compose` і `lifecycle-runtime-compose`. Потім створіть клас `ui/records/RecordsViewModel.kt`, у якому зберігатиметься список записів і функції для його зміни: додавання запису, додавання склянки води і видалення.

Після цього змініть `SmartLifeApp`: отримайте в ній одну спільну ViewModel і передавайте в екрани не саму ViewModel, а її стан і лямбди-обробники. Локальні стани, які мають пережити поворот (текст у полях діалогу, прапорець відкритого діалогу), переведіть з `remember` на `rememberSaveable`. Наостанок перевірте результат: увімкніть автоповорот на емуляторі, поверніть його і переконайтеся, що записи і введений текст не зникли.

Ключові фрагменти реалізації наведено нижче.

```
// ui/records/RecordsViewModel.kt
class RecordsViewModel : ViewModel() {
    private val _records = MutableStateFlow(FakeData.records) // змінює лише ViewModel
    val records: StateFlow<List<HealthRecord>> = _records.asStateFlow()

    fun addRecord(type: RecordType, value: Double, note: String) {
        _records.update { current ->
            val newId = (current.maxOfOrNull { it.id } ?: 0) + 1
            listOf(HealthRecord(id = newId, type = type, value = value, note = note)) + current
        }
    }
    fun addWaterGlass() = addRecord(RecordType.WATER, 0.25, "Склянка води")
    fun deleteRecord(id: Long) = _records.update { list -> list.filterNot { it.id == id } }
}

// Екран лише показує стан і передає події вгору
val records by viewModel.records.collectAsStateWithLifecycle()
RecordsContent(records = records, onAdd = viewModel::addRecord, onRecordClick = ...)
// ... підтвердження видалення: AlertDialog, стан діалогу в rememberSaveable
```

У класі `RecordsViewModel` стан зберігається у двох властивостях. Приватна `_records` має тип `MutableStateFlow`, тобто її значення можна змінювати – але лише всередині `ViewModel`. Назовні через `asStateFlow()` віддається властивість `records` лише для читання. Так гарантується односпрямований потік даних: екран не може змінити список напрямку, він може тільки попросити `ViewModel`.

Функція `addRecord` викликає `_records.update { current -> ... }`. Усередині лямбди обчислюється новий ідентифікатор (найбільший існуючий плюс один) і створюється **новий** список: новий запис, до якого додано всі попередні. Стан у `Compose` має бути незмінним – якби ми додали елемент до старого списку методом `add()`, посилення на список не змінилося б і `Compose` не помітив би змін. З тієї ж причини `deleteRecord` повертає новий список, відфільтрований через `filterNot`.

На боці екрана `collectAsStateWithLifecycle()` підписується на `StateFlow` лише тоді, коли екран видимий, і перетворює кожне нове значення на стан `Compose`. Екран розділено на дві функції: `RecordsScreen` отримує `ViewModel` і передає дані далі, а `RecordsContent` приймає лише список і лямбди. Друга функція нічого не знає про `ViewModel`, тому її легко показати в `@Preview` і протестувати (підрозділ 6.2).

Запуск на емуляторі. Додайте кілька записів і склянок води, поверніть емулятор кнопкою обертання на панелі (попередньо увімкніть автоповорот) – дані залишаються. Видаліть запис з екрана деталей і перевірте, що він зник з журналу та підсумків.

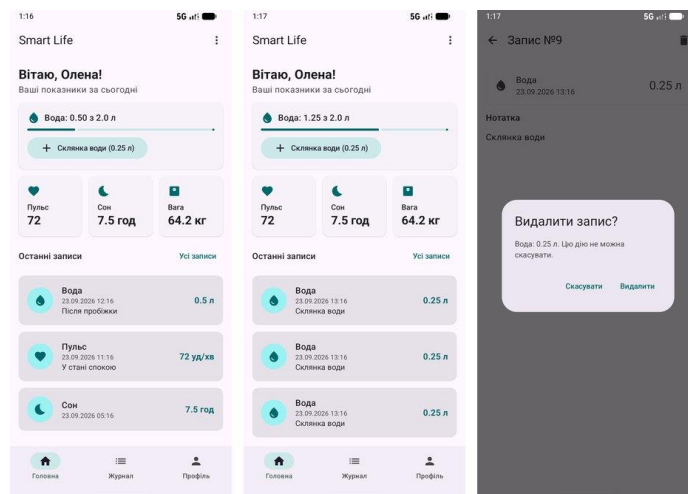


Рисунок 2.5 – Стан у `ViewModel`: лічильник води до і після натискань, підтвердження видалення запису

На рисунку 2.5 ліворуч – головний екран з карткою води «Вода: 0.50 з 2.0 л», смугою прогресу і кнопкою «Склянка води (0.25 л)». Посередині – той самий екран після трьох натискань кнопки: випито 1.25 л, смуга прогресу збільшилася, а у списку останніх записів з'явилися три записи «Склянка води» по 0.25 л. Праворуч – екран деталей запису №9 з діалогом «Видалити запис?», що показує тип і значення запису та кнопки «Скасувати» і «Видалити».

2.4 Архітектура MVVM і впровадження залежностей

Коли весь код знаходиться в екранах, застосунок важко змінювати і тестувати. **Архітектура** розділяє відповідальність: **UI-шар** (екрани і `ViewModel`) показує стан і обробляє дії користувача; **шар даних** (репозиторії і джерела даних) знає, звідки беруться дані – пам'ять, база, мережа. `ViewModel` звертається до інтерфейсу `RecordRepository`, не знаючи конкретної реалізації, тому в підрозділі 3.3 ми замінимо зберігання в пам'яті на базу даних, не змінюючи жодного екрана (рис. 2.6).

MVVM (Model – View – `ViewModel`): Model – дані і бізнес-логіка; View – екрани `Compose`; `ViewModel` – посередник, який перетворює дані на стан екрана. **Єдине джерело правди**: кожен тип даних має одного власника (репозиторій), усі екрани спостерігають за ним через `Flow` і тому завжди узгоджені.

Впровадження залежностей (DI): клас не створює свої залежності сам, а отримує їх через конструктор. У курсі використано ручний DI: клас `AppContainer` створює репозиторії один раз на весь застосунок, а `AppViewModelProvider.Factory` передає їх у `ViewModel`. У великих проєктах цю роль виконують Hilt або Koin.

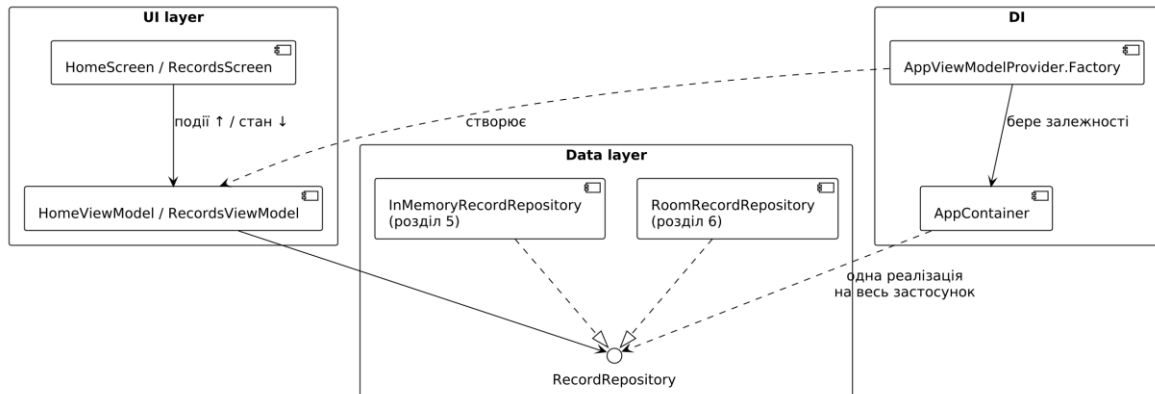


Рисунок 2.6 – Архітектура MVVM у Smart Life

Рисунок 2.6 поділено на три області. В UI-шарі екрани обмінюються з `ViewModel` подіями і станом. `ViewModel` залежать лише від інтерфейсу `RecordRepository` у шарі даних; дві його реалізації – `InMemoryRecordRepository` для цього підрозділу і `RoomRecordRepository` для підрозділу 3.3 – позначено пунктирними стрілками «реалізує». В області DI фабрика `AppViewModelProvider.Factory` бере залежності з `AppContainer` і створює `ViewModel`, а контейнер вирішує, яку одну реалізацію репозиторію використовуватиме весь застосунок.

MVVM – не єдиний архітектурний шаблон для мобільних застосунків. Порівняння найпоширеніших шаблонів наведено в таблиці 2.5. Для Compose найприродніші MVVM і MVI, оскільки обидва побудовано на односпрямованому потоці стану; у курсі використано MVVM як простіший для початківців.

Таблиця 2.5 – Архітектурні шаблони мобільних застосунків

Шаблон	Складові	Особливості
MVC	Model, View, Controller	контролер часто зливається з Activity і розростається
MVP	Model, View, Presenter	презентер керує View через інтерфейс; багато шаблонного коду
MVVM	Model, View, ViewModel	View спостерігає за станом ViewModel; підтримується бібліотеками Jetpack
MVI	Model, View, Intent	увесь стан екрана – один незмінний об'єкт, дії описуються як події

Архітектура спирається на кілька загальних принципів проєктування. **Розділення відповідальності:** кожен клас має одну причину для змін – екран відповідає за вигляд, `ViewModel` за стан, репозиторій за дані. **Залежність від абстракцій:** `ViewModel` знає лише інтерфейс репозиторію, а не його реалізацію. **Єдине джерело правди** для кожного типу даних. Дотримання цих принципів робить можливим те, що буде зроблено в підрозділі 3.3: заміну зберігання в пам'яті на базу даних без зміни жодного екрана.

Практичний приклад. Smart Life: інтерфейс `RecordRepository` з реалізацією в пам'яті, окремі `HomeViewModel`, `RecordsViewModel`, `RecordDetailsViewModel`, клас `SmartLifeApplication` з контейнером залежностей. Зовні застосунок не змінився – змінилася внутрішня будова.

Почніть з організації пакетів: створіть пакети `data/repository` для репозиторіїв, `di` для контейнера залежностей і окремі пакети `ui/<екран>` для кожного екрана з його `ViewModel`. Потім опишіть інтерфейс `RecordRepository` з операціями над записами і його першу реалізацію `InMemoryRecordRepository`, яка поки що зберігає записи в пам'яті.

Далі створіть клас `AppContainer`, який створює репозиторій, і клас `SmartLifeApplication`, що успадковує `Application` і зберігає єдиний екземпляр контейнера. Клас застосунку обов'язково зареєструйте в маніфесті атрибутом `android:name=".SmartLifeApplication"`. Після цього створіть `AppViewModelProvider.Factory` і по одній `ViewModel` для кожного екрана, а в самих екранах отримуйте `ViewModel` викликом `viewModel(factory = AppViewModelProvider.Factory)`.

Ключові фрагменти реалізації наведено нижче.

```
// data/repository/RecordRepository.kt - контракт шару даних
interface RecordRepository {
    fun observeRecords(): Flow<List<HealthRecord>>
    fun observeRecord(id: Long): Flow<HealthRecord?>
    suspend fun addRecord(record: HealthRecord): Long
    suspend fun updateRecord(record: HealthRecord)
    suspend fun deleteRecord(id: Long)
}

// di/AppContainer.kt - одна реалізація на весь застосунок
class AppContainer(private val context: Context) {
    val recordRepository: RecordRepository by lazy {
        InMemoryRecordRepository(FakeData.records)
    }
    // ... у наступних розділах: settingsRepository, weatherRepository, authRepository тощо
}

// ui/AppViewModelProvider.kt - фабрика ViewModel
object AppViewModelProvider {
    val Factory = viewModelFactory {
        initializer { HomeViewModel(smartLifeApplication().container.recordRepository) }
        initializer { RecordsViewModel(smartLifeApplication().container.recordRepository) }
        initializer {
            RecordDetailsViewModel(
                savedStateHandle = createSavedStateHandle(),
                recordRepository = smartLifeApplication().container.recordRepository
            )
        }
    }
}

// ui/records/RecordsViewModel.kt - стан з репозиторію
val records: StateFlow<List<HealthRecord>> = recordRepository.observeRecords()
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())
```

Інтерфейс `RecordRepository` – це контракт шару даних: він говорить, *що* можна робити із записами, але не *як*. Методи спостереження `observeRecords()` і `observeRecord(id)` повертають `Flow`, тобто потік, який видаватиме новий список щоразу, коли дані змінюються. Методи зміни позначено `suspend`: вони можуть виконуватися довго (запис у базу, запит до сервера), тому викликаються з корутин і не блокують інтерфейс.

У `AppContainer` властивість `recordRepository` має тип інтерфейсу, а не конкретного класу, і створюється через `by lazy`: об'єкт з'являється під час першого звернення, а далі весь застосунок отримує той самий екземпляр. Саме це робить репозиторій єдиним джерелом правди – усі екрани бачать однакові дані.

`AppViewModelProvider.Factory` описує, як створити кожну `ViewModel`. Блок `initializer { ... }` отримує доступ до об'єкта застосунку (`smartLifeApplication()`) і передає в конструктор `ViewModel` потрібні залежності з контейнера. Для `RecordDetailsViewModel` додатково викликається `createSavedStateHandle()`: через нього `ViewModel` отримує аргумент маршруту `recordId` без участі екрана.

Останній фрагмент показує, як `ViewModel` перетворює потік з репозиторію на стан екрана. Оператор `stateIn` робить з холодного `Flow` гарячий `StateFlow` з початковим значенням `emptyList()`. Параметр `SharingStarted.WhileSubscribed(5_000)` означає: поки хтось підписаний – отримувати дані, а через 5 секунд після того, як екран зник, зупинити підписку. Затримки в 5 секунд досить, щоб під час повороту екрана підписка не переривалася.

Запуск на емуляторі. Запустіть застосунок: поведінка має бути такою самою, як у підрозділі 2.3. Змініть запис на одному екрані – він оновиться на всіх екранах, бо всі спостерігають за одним репозиторієм.

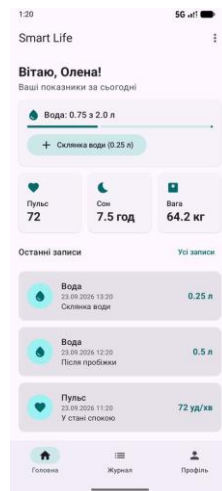


Рисунок 2.7 – Головний екран після переходу на MVVM: дані надходять з репозиторію

Зовні головний екран на рисунку 2.7 не відрізняється від попереднього розділу: картка води (0.75 з 2.0 л), картки пульсу, сну і ваги та список останніх записів. Різниця у внутрішній будові – ці дані тепер надходять з репозиторію через `HomeViewModel`, тож запис, доданий у журналі, одразу з’являється і тут.

2.5 Типові помилки

Інтерфейс і навігація. Якщо компілятор повідомляє *Unresolved reference* для `composable<...>`, перевірте, що підключено плагін `kotlin-serialization` і маршрут має анотацію `@Serializable`. Повторне натискання на вкладку нижньої панелі без додаткових параметрів навігації створює в стеку кілька копій одного екрана – тому для вкладок використовують `launchSingleTop = true` і `popUpTo(...)` { `saveState = true` }.

Коли список «стрибає» або картки анімуються неправильно після видалення, найімовірніше, в `items()` не вказано `key`. А якщо текст налазить на рядок стану чи нижню панель, до вмісту не застосовано відступи `innerPadding`, які `Scaffold` передає в лямбду.

Стан. Якщо дані зникають після повороту екрана, стан зберігається в `remember` замість `ViewModel` або `rememberSaveable`. Якщо ж інтерфейс не оновлюється після зміни даних, найчастіше список було змінено «на місці» (методом `add` чи `remove`), а не замінено новим. Повідомлення *Cannot create an instance of class ViewModel* означає, що `ViewModel` має параметри конструктора, а фабрику для її створення не вказано, – цю задачу розв’язує підрозділ 2.4. Нарешті, не передавайте `ViewModel` у дрібні `composable`-функції: вони мають отримувати лише дані й лямбди.

Архітектура. Помилка *ClassCastException: Application cannot be cast to SmartLifeApplication* означає, що клас застосунку не вказано в атрибуті `android:name` маніфесту. Якщо два екрани показують різні дані, репозиторій, скоріш за все, створено двічі в обхід контейнера. Не передавайте у `ViewModel` `Context` `Activity` – `ViewModel` живе довше за `Activity` і утримуватиме її в пам’яті; використовуйте лише залежності з контейнера. І загалом бізнес-логіку слід переносити з `composable`-функцій у `ViewModel` або репозиторій – екран має лише відображати стан.

2.6 Практичні завдання

Практичні завдання розділу наведено в таблиці 2.6. Основне завдання виконується на основі версії `Smart Life`, отриманої в підрозділі, а завдання для самостійної роботи поглиблює тему і може оцінюватися додатково.

Таблиця 2.6 – Практичні завдання розділу

Тема	Практичне завдання	Завдання для самостійної роботи
------	--------------------	---------------------------------

Тема	Практичне завдання	Завдання для самостійної роботи
Інтерфейс і навігація	Додайте на екран деталей кнопку «Поділитися», яка показує Snackbar з текстом запису, і пункт меню «Про застосунок» з діалогом.	Створіть окремий екран «Статистика» з підсумками за типами записів і додайте його четвертою вкладкою нижньої панелі.
Стан	Додайте до <code>RecordsViewModel</code> фільтр за типом запису (<code>StateFlow<RecordType?></code>) і чипи-фільтри над списком.	Реалізуйте скасування видалення: після видалення показуйте Snackbar з дією «Скасувати», що повертає запис.
Архітектура	Додайте в репозиторій метод <code>observeRecordsByType(type)</code> і використайте його у фільтрі журналу з підрозділу 2.3.	Перепишіть створення залежностей з використанням бібліотеки Hilt і порівняйте обсяг коду з ручним DI.

2.7 Контрольні питання

1. Чим декларативний UI відрізняється від імперативного?
2. Для чого потрібні модифікатори і чи важливий їх порядок?
3. Чому для довгих списків використовують `LazyColumn`?
4. Як передати аргумент між екранами в Navigation Compose?
5. Що робить `popUpTo`?
6. Чим `remember` відрізняється від `rememberSaveable`?
7. Чому `ViewModel` переживає поворот екрана?
8. Що таке підйом стану?
9. Навіщо розділяти `MutableStateFlow` і `StateFlow`?
10. Чому стан має бути незмінним?
11. Які шари виділяють в архітектурі Android-застосунку?
12. Навіщо `ViewModel` залежить від інтерфейсу репозиторію, а не від класу?
13. Що таке єдине джерело правди?
14. Для чого потрібна фабрика `ViewModel`?
15. Що робить `stateIn` з `WhileSubscribed`?

РОЗДІЛ 3. РОБОТА З ДАНИМИ: ЛОКАЛЬНЕ СХОВИЩЕ ТА МЕРЕЖА

3.1 Мета розділу та результати навчання

Мета розділу – навчити виконувати тривалі операції асинхронно за допомогою корутин і потоків Flow, зберігати дані на пристрої в базі даних Room і сховищі DataStore, а також отримувати дані із сервера через REST API. Після розділу записи Smart Life зберігаються між запусками, а на головному екрані з'являється погода з мережі. Результати навчання наведено в таблиці 3.1, основні поняття – в таблиці 3.2.

Таблиця 3.1 – Результати навчання за темами розділу

Тема	Що студент знає і вміє після опрацювання
Корутини і Flow	Призначення головного потоку і причини ANR; suspend-функції, області видимості і диспетчери корутин; холодні Flow і гарячі StateFlow; оператори потоків і callbackFlow
Локальні дані	Способи зберігання даних в Android (внутрішні файли, DataStore, SQLite/Room) і коли який обирати; компоненти Room (@Entity, @Dao, @Database); операції CRUD; спостереження за даними через Flow; роботу з DataStore Preferences; поняття KSP.
Мережа	Будову HTTP-запиту і відповіді; методи REST API і коди статусу; формат JSON і DTO-класи з @Serializable; анотації Retrofit (@GET, @POST, @Query, @Body, @Header); обробку IOException і HttpException; опис станів запиту через sealed interface; дозвіл INTERNET.

Таблиця 3.2 – Основні поняття розділу

Поняття	Визначення
головний потік	потік, у якому обробляються дотики і малюється інтерфейс
корутина	обчислення, яке можна призупинити і продовжити, не блокуючи потік
диспетчер	визначає потік або пул потоків для виконання корутини
Flow / StateFlow	холодний асинхронний потік значень / гарячий потік з поточним значенням
CRUD	Create, Read, Update, Delete – базові операції з даними
Entity	клас-рядок таблиці бази даних
DAO	Data Access Object – інтерфейс із запитам до таблиці
KSP	інструмент, що генерує код за анотаціями під час збірки
DataStore	асинхронне сховище пар «ключ – значення»
REST API	інтерфейс сервера з ресурсами за адресами та HTTP-методами
JSON	текстовий формат обміну даними з об'єктами і масивами
DTO	клас, що точно повторює структуру JSON-відповіді
Retrofit	бібліотека для опису HTTP API інтерфейсом Kotlin
sealed interface	закритий набір варіантів стану, всі обробляються в when

3.2 Асинхронність: корутини та потоки Flow

У кожного Android-застосунку є **головний потік** (main thread), у якому система обробляє дотики і малює інтерфейс. Щоб анімації були плавними, кадр має формуватися приблизно кожні 16 мс. Якщо в головному потоці виконати запит до сервера або запис у базу, які тривають сотні мілісекунд, інтерфейс «замерзне», а через 5 секунд система покаже діалог *Application Not Responding* (ANR). Тому всі тривалі операції виконуються асинхронно.

У Kotlin для цього використовують **корутини** – обчислення, які можна призупинити і продовжити пізніше, не блокуючи потік. Функція з модифікатором suspend може викликати інші призупинювані функції; у точці призупинення корутина «відпускає» потік, і той виконує іншу роботу, а коли результат готовий, корутина продовжується з того самого місця. Завдяки цьому асинхронний код виглядає як звичайний послідовний: без колбеків і вкладених обробників.

Кожна корутина запускається в **області видимості** (CoroutineScope), яка обмежує її життя. У ViewModel використовують viewModelScope: при знищенні ViewModel усі її корутини автоматично скасовуються, тому запит, запущений з екрана, не продовжить роботу після закриття екрана. Потік виконання обирає **диспетчер** (табл. 3.3);

перемикається між диспетчерами можна функцією `withContext`. Бібліотеки Room і Retrofit самі переносять свою роботу у фонові потоки, тому з ViewModel їхні `suspend`-функції викликаються безпосередньо.

Таблиця 3.3 – Диспетчери корутин

Диспетчер	Потоки	Для чого використовується
<code>Dispatchers.Main</code>	головний потік	оновлення інтерфейсу; за замовчуванням у <code>viewModelScope</code>
<code>Dispatchers.IO</code>	розширюваний пул потоків	файли, база даних, мережа (блокувальні операції)
<code>Dispatchers.Default</code>	пул за кількістю ядер процесора	обчислення: сортування, обробка зображень, розбір великих даних

На рисунку 3.1 показано, як виконується збереження запису в Smart Life. Натискання кнопки обробляється в головному потоці, ViewModel запускає корутину і викликає `suspend`-функцію репозиторію. У цей момент корутина призупиняється, а головний потік залишається вільним і продовжує малювати інтерфейс. Room виконує вставку в пулі потоків `Dispatchers.IO`, після чого корутина відновлюється в головному потоці. Далі потік даних з бази сам повідомляє про новий список, і екран оновлюється.

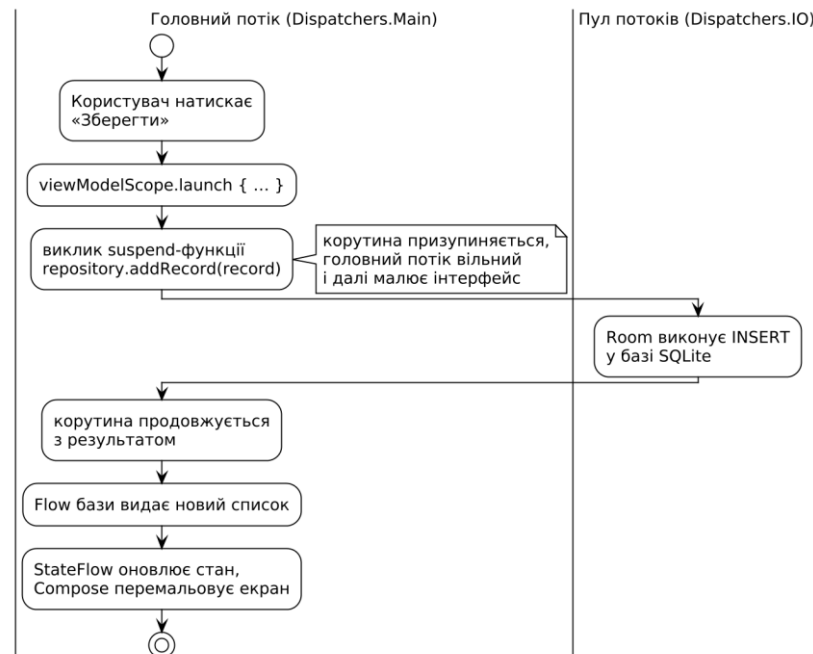


Рисунок 3.1 – Виконання корутини в головному та фоновому потоках

Для даних, що змінюються з часом, використовують **Flow** – асинхронний потік значень. Звичайний Flow є **холодним**: він починає обчислення лише тоді, коли хтось викликає `collect`, і для кожного збирача виконується заново. Саме такі потоки повертають запити Room і DataStore. **StateFlow** є **гарячим** потоком: він завжди має поточне значення, існує незалежно від збирачів і розсилає кожне нове значення всім підписникам. ViewModel перетворює холодний потік з репозиторію на StateFlow оператором `stateIn`, а екран збирає його через `collectAsStateWithLifecycle()`. Між джерелом і збирачем потік можна обробляти операторами, подібними до функцій колекцій: `map`, `filter`, `combine` (об'єднання кількох потоків), `runningFold` (накопичення). Для перетворення API на колбеках на потік служить будівник `callbackFlow`, який широко застосовується в розділі 4.

```
// Запуск корутини у ViewModel і збереження файлу у фоновому потоці
fun onPhotoSelected(uri: Uri) {
    viewModelScope.launch { // головний потік
        try {
            val path = photoStorage.savePhoto(uri) // призупинення
            settingsRepository.setProfilePhotoPath(path)
        }
    }
}
```

```

        message.value = "Фото профілю оновлено"
    } catch (e: CancellationException) { throw e }
    catch (e: Exception) { message.value = "Не вдалося зберегти фото: ${e.message}" }
}
suspend fun savePhoto(source: Uri): String = withContext(Dispatchers.IO) {
    // ... копіювання файлу: блокувальна операція, тому в Dispatchers.IO
}

```

У наведеному фрагменті (спрощено з ProfileViewModel і ProfilePhotoStorage) ViewModel запускає корутину в viewModelScope, тобто в головному потоці. Виклик savePhoto призупиняє корутину, а сама функція за допомогою withContext(Dispatchers.IO) виконує копіювання у фоновому пулі і повертає шлях до файлу. Після цього корутину продовжується в головному потоці, зберігає шлях у налаштуваннях і показує повідомлення; помилка копіювання перехоплюється і перетворюється на текст для користувача, а CancellationException передається далі. Якщо користувач закрити екран під час копіювання, viewModelScope скасує корутину.

3.3 Локальне зберігання даних: Room і DataStore

Дані в пам'яті зникають після закриття застосунку. Android пропонує кілька способів постійного зберігання: **файли** у внутрішній пам'яті (context.filesDir) – для фото і документів; **DataStore** – для невеликих наборів «ключ – значення» (налаштування, прапорці); **база даних SQLite** – для структурованих даних, пошуку і сортування. Внутрішні файли, DataStore і база належать лише застосунку: інші застосунки не мають до них доступу, а при видаленні застосунку вони видаляються.

Room – бібліотека Jetpack над SQLite. Клас з анотацією @Entity описує таблицю, інтерфейс @Dao – запити, абстрактний клас @Database – саму базу (рис. 3.2). Реалізацію DAO генерує KSP (Kotlin Symbol Processing) під час збірки, а SQL-запити перевіряються компілятором. Запит, що повертає Flow, автоматично видає новий результат після кожної зміни таблиці – так екран оновлюється без додаткового коду. Операції запису є suspend-функціями і не блокують головний потік.

DataStore Preferences зберігає налаштування асинхронно (на відміну від застарілого SharedPreferences) і також віддає їх як Flow.

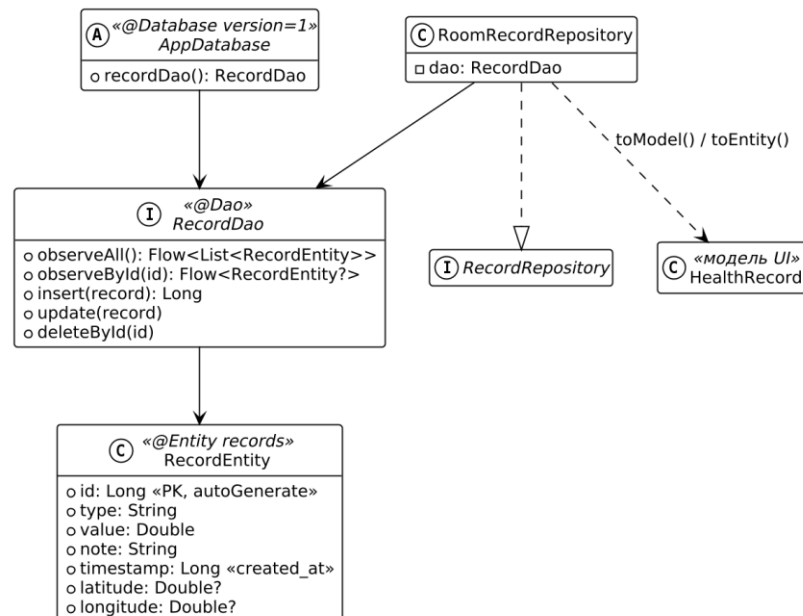


Рисунок 3.2 – Компоненти Room і їх зв'язок з репозиторієм

На діаграмі класів (рис. 3.2) видно три компоненти Room і їхнє використання. AppDatabase надає доступ до RecordDao, а DAO працює з рядками таблиці RecordEntity, поля якої перелічено в прямокутнику сутності. RoomRecordRepository реалізує інтерфейс

RecordRepository, використовує DAO і перетворює сутності на модель HealthRecord для інтерфейсу.

Вибір способу зберігання залежить від характеру даних. Порівняння способів наведено в таблиці 3.4. Для зберігання даних в інших застосунках (спільні фото, документи) призначено окреме спільне сховище з системою MediaStore, яке в курсі використовується лише через Photo Picker (підрозділ 4.5).

Таблиця 3.4 – Способи локального зберігання даних в Android

Спосіб	Формат даних	Переваги	Приклад у Smart Life
Файли у filesDir/cacheDir	довільні байти	зображення, документи будь-якого розміру	фото профілю
DataStore Preferences	пари «ключ – значення»	асинхронний, типізовані ключі, Flow	тема, норма води, сесія
SQLite через Room	таблиці, SQL-запити	пошук, сортування, зв'язки, перевірка запитів компілятором	журнал показників
SharedPreferences (застарілий)	пари «ключ – значення»	простий синхронний API	не використовується

Структура бази даних змінюється разом із застосунком: додаються стовпці і таблиці. Щоб користувачі не втратили дані після оновлення, кожна зміна @Entity супроводжується збільшенням номера версії бази і **міграцією** – SQL-командами, які переводять стару структуру в нову (ALTER TABLE records ADD COLUMN ...). Room перевіряє, що структура бази відповідає опису сутностей, і відмовляється працювати з невідповідною базою.

Практичний приклад. Smart Life: записи зберігаються в базі smart_life.db, їх можна додати, змінити і видалити; екран «Налаштування» дозволяє задати ім'я, тему (світла, темна, системна) і денну норму води.

Спочатку підключіть до проєкту плагін KSP і бібліотеки: room-runtime і room-ktx для роботи з базою, room-compiler (підключається конфігурацією ksp(...), бо генерує код) і datastore-preferences для налаштувань.

Потім опишіть базу даних. У пакеті data/local створіть сутність RecordEntity, інтерфейс доступу RecordDao і клас бази AppDatabase, а також функції-перетворювачі toModel() і toEntity(), що переводять запис бази в модель застосунку і назад. На основі DAO напишіть реалізацію RoomRecordRepository і замініть нею в AppContainer реалізацію в пам'яті.

Для налаштувань створіть data/settings/SettingsRepository.kt з класом UserSettings (ім'я, тема, норма води), SettingsViewModel і екран SettingsScreen. Наостанок у MainActivity прочитайте тему з налаштувань і передайте її в SmartLifeTheme, щоб зміна теми застосовувалася одразу.

Ключові фрагменти реалізації наведено нижче.

```
// data/local/RecordEntity.kt та RecordDao.kt
@Entity(tableName = "records")
data class RecordEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val type: String, val value: Double, val note: String,
    @ColumnInfo(name = "created_at") val timestamp: Long,
    val latitude: Double?, val longitude: Double?
)

@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY created_at DESC")
    fun observeAll(): Flow<List<RecordEntity>>
    @Insert suspend fun insert(record: RecordEntity): Long
    @Update suspend fun update(record: RecordEntity)
    @Query("DELETE FROM records WHERE id = :id") suspend fun deleteById(id: Long)
    // ... observeById(id)
}

// di/AppContainer.kt - база створюється один раз
private val database by lazy {
    Room.databaseBuilder(context, AppDatabase::class.java, "smart_life.db").build()
}
```

```

val recordRepository: RecordRepository by lazy { RoomRecordRepository(database.recordDao()) }

// data/settings/SettingsRepository.kt - налаштування в DataStore
val settings: Flow<UserSettings> = context.dataStore.data.map { prefs ->
    UserSettings(
        userName = prefs[Keys.USER_NAME] ?: "Користувач",
        themeMode = prefs[Keys.THEME_MODE]?.let { ThemeMode.valueOf(it) } ?: ThemeMode.SYSTEM,
        waterGoalLiters = prefs[Keys.WATER_GOAL] ?: 2.0
    )
}
suspend fun setThemeMode(mode: ThemeMode) {
    context.dataStore.edit { it[Keys.THEME_MODE] = mode.name }
}

```

Клас `RecordEntity` з анотацією `@Entity(tableName = "records")` описує таблицю бази: кожна властивість стає стовпцем, а `@PrimaryKey(autoGenerate = true)` робить `id` первинним ключем, значення якого база призначає сама. Анотація `@ColumnInfo(name = "created_at")` показує, що назва стовпця може відрізнятися від назви властивості. Тип запису зберігається рядком ("WATER"), бо SQLite не знає про enum; перетворення виконують функції `toModel()` і `toEntity()`, тому решта застосунку працює зі зручною моделлю `HealthRecord`. Стовпці координат додано заздалегідь – вони знадобляться в підрозділі 4.4.

У `RecordDao` кожен метод описує один запит. Запит `SELECT` з сортуванням за датою повертає `Flow<List<RecordEntity>>`: Room відстежує таблицю і після кожної вставки, зміни чи видалення автоматично видає новий список – завдяки цьому журнал і головний екран оновлюються самі. Анотації `@Insert` і `@Update` не потребують SQL: реалізацію генерує KSP. Методи зміни мають модифікатор `suspend`, і Room виконує їх у фоновому потоці.

Базу створює `Room.databaseBuilder` у контейнері залежностей, причому лише один раз (`by lazy`). Оскільки `ViewModel` залежать від інтерфейсу `RecordRepository`, для переходу з пам'яті на базу змінено тільки рядок створення репозиторію в контейнері – жоден екран не змінився. У `SettingsRepository` налаштування читаються з `dataStore.data`, де кожен ключ перетворюється на поле `UserSettings` зі значенням за замовчуванням, а записуються функцією `edit { }`, як показано на прикладі `setThemeMode`.

Запуск на емуляторі. Додайте записи, закрийте застосунок (приберіть його зі списку недавніх) і відкрийте знову – дані збереглися. Змініть тему на «Темна» – інтерфейс перемкнеться одразу. Вміст бази можна переглянути в Android Studio: `View` ▶ `Tool Windows` ▶ `App Inspection` ▶ `Database Inspector`.

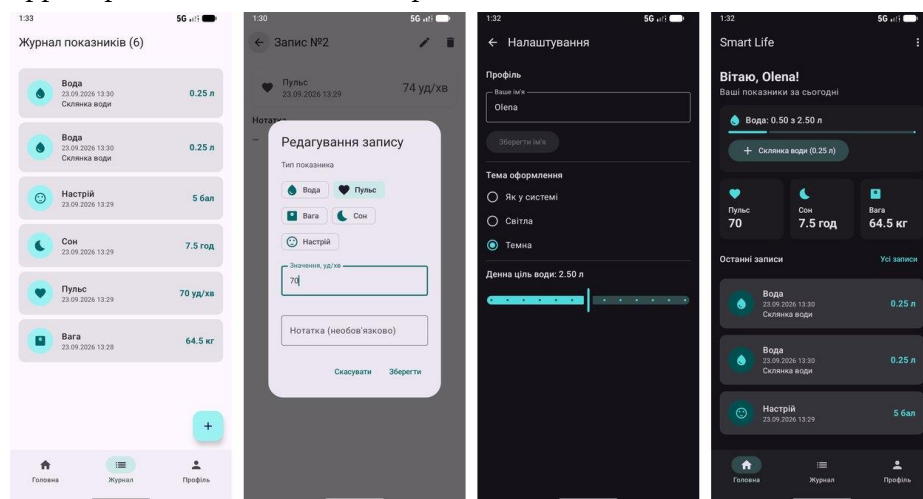


Рисунок 3.3 – Дані в Room і DataStore: журнал, редагування запису, налаштування і головний екран у темній темі

На рисунку 3.3 першим показано журнал, заголовок якого тепер містить кількість записів «(6)». Другий екран – деталі запису №2 з відкритим діалогом «Редагування запису», у якому для пульсу змінено значення на 70. Третій екран – налаштування в

темній темі: поле «Ваше ім'я», перемикач теми «Як у системі / Світла / Темна» з обраною темною темою і повзунок денної цілі води 2.50 л. Четвертий – головний екран у темній темі, де вже враховано нове ім'я та норму води «0.50 з 2.50 л». Усі ці зміни зберігаються після перезапуску застосунку.

3.4 Мережа: REST API та Retrofit

HTTP – протокол «запит – відповідь». Запит містить метод, URL (з query-параметрами після ?), заголовки і, можливо, тіло; відповідь – код статусу, заголовки і тіло. Коди **2xx** означають успіх, **4xx** – помилку клієнта (400, 401, 404), **5xx** – помилку сервера. **HTTPS** – HTTP через зашифроване з'єднання TLS; Android за замовчуванням забороняє незашифрований HTTP.

REST API – стиль, у якому кожен ресурс має адресу, а дія визначається методом: GET – отримати, POST – створити або виконати дію, PUT/PATCH – оновити, DELETE – видалити. Дані передаються у форматі **JSON**. Для кожної JSON-відповіді описують **DTO** (Data Transfer Object) – Kotlin-клас з анотацією @Serializable; назви полів, які не відповідають стилю Kotlin, зв'язують через @SerializedName.

Retrofit перетворює інтерфейс з анотаціями на HTTP-клієнт; мережа виконується в suspend-функціях поза головним потоком. **OkHttp** відповідає за з'єднання, тайм-аути і логування. Помилки мережі (немає зв'язку, тайм-аут, код 4xx/5xx) – нормальна ситуація для мобільного застосунку, тому стан запиту описують явно: Loading, Success, Error (рис. 3.4).

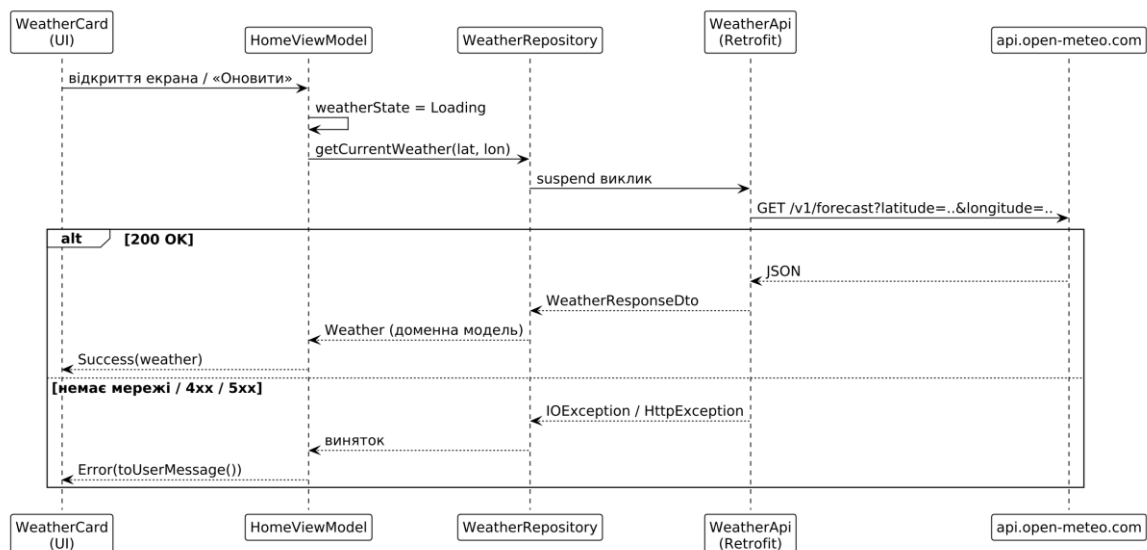


Рисунок 3.4 – Послідовність завантаження погоди

Діаграма послідовності на рисунку 3.4 показує учасників запиту зліва направо і повідомлення між ними зверху вниз. Картка погоди просить ViewModel завантажити дані, ViewModel встановлює стан Loading і звертається до репозиторію, репозиторій викликає Retrofit, а Retrofit надсилає HTTP-запит GET /v1/forecast на сервер. Далі діаграма розгалужується: у гілці «200 OK» JSON перетворюється на DTO, потім на модель Weather, і картка отримує Success; у гілці помилки виняток доходить до ViewModel і перетворюється на стан Error з текстом для користувача.

Основні методи HTTP і найпоширеніші коди відповідей наведено в таблиці 3.5. Для мобільного застосунку важливо не лише обробити успішну відповідь, а й зрозуміло пояснити користувачу кожну помилку: 401 означає, що потрібно увійти знову, 404 – що даних немає, 5xx – що проблема на боці сервера і варто спробувати пізніше.

Таблиця 3.5 – Методи HTTP та коди відповідей

Метод	Дія	Код відповіді	Значення
GET	отримати ресурс	200 OK	запит виконано успішно

Метод	Дія	Код відповіді	Значення
POST	створити ресурс або виконати дію	201 Created	ресурс створено
PUT / PATCH	замінити / частково змінити ресурс	400 Bad Request, 401 Unauthorized	некоректний запит / потрібна автентифікація
DELETE	видалити ресурс	404 Not Found, 500 Internal Server Error	ресурс не знайдено / помилка сервера

Нижче наведено фрагмент реальної відповіді сервера Open-Meteo на запит поточної погоди. Об'єкт JSON записується у фігурних дужках, пари «ключ – значення» розділено комами, вкладений об'єкт `current` містить потрібні показники. Саме цю структуру повторюють DTO-класи `WeatherResponseDto` і `CurrentWeatherDto`.

```
{
  "latitude": 50.4375, "longitude": 30.5, "timezone": "Europe/Kyiv",
  "current": {
    "time": "2026-09-23T16:30", "temperature_2m": 13.7,
    "relative_humidity_2m": 63, "apparent_temperature": 11.5,
    "wind_speed_10m": 9.0, "weather_code": 3
  }
}
```

Практичний приклад. Smart Life: на головному екрані з'являється картка погоди (температура, відчувається як, вологість, вітер) з API <https://api.open-meteo.com> (без ключа). Під час завантаження картка показує індикатор, при помилці – текст помилки і кнопку «Повторити».

Першим кроком дозвольте застосунку доступ до мережі: додайте в маніфест `<uses-permission android:name="android.permission.INTERNET"/>`. Цей дозвіл «звичайний», тому користувач його не підтверджує. Далі підключіть Retrofit, конвертер `kotlinx.serialization` і `logging-interceptor`, увімкніть генерацію класу `BuildConfig` (`buildConfig = true`) і додайте в нього поле `WEATHER_BASE_URL` з адресою сервера.

Перш ніж писати код, відкрийте запит у браузері, наприклад https://api.open-meteo.com/v1/forecast?latitude=50.45&longitude=30.52¤t=temperature_2m, і подивіться на реальну відповідь – за нею створюються DTO-класи. Потім у пакеті `data/remote` створіть інтерфейс `WeatherApi`, модуль `NetworkModule` з налаштуваннями клієнта, файл `NetworkErrors.kt` з перетворенням помилок на текст і `WeatherRepository`. Наостанок додайте в `HomeViewModel` стан `WeatherUiState` і функцію `loadWeather()`, а на головний екран – картку `WeatherCard`.

Ключові фрагменти реалізації наведено нижче.

```
// data/remote/WeatherApi.kt
interface WeatherApi {
    @GET("v1/forecast")
    suspend fun getCurrentWeather(
        @Query("latitude") latitude: Double,
        @Query("longitude") longitude: Double,
        @Query("current") fields: String = "temperature_2m,relative_humidity_2m," +
            "apparent_temperature,wind_speed_10m,weather_code"
    ): WeatherResponseDto
}

// data/remote/WeatherDto.kt - фрагмент DTO
@Serializable data class CurrentWeatherDto(
    @SerializedName("temperature_2m") val temperature: Double,
    @SerializedName("weather_code") val weatherCode: Int,    // ... вологість, вітер
)

// ui/home/WeatherUiState.kt
sealed interface WeatherUiState {
    data object Loading : WeatherUiState
    data class Success(val weather: Weather, val placeLabel: String) : WeatherUiState
    data class Error(val message: String) : WeatherUiState
}

// ui/home/HomeViewModel.kt
fun loadWeather() = viewModelScope.launch {
    _weatherState.value = WeatherUiState.Loading
    _weatherState.value = try {
        val weather = weatherRepository.getCurrentWeather(KYIV_LATITUDE, KYIV_LONGITUDE)
        WeatherUiState.Success(weather, placeLabel = "Київ (типово)")
    } catch (e) {
        WeatherUiState.Error(e.message ?: "Немає інтернету")
    }
}
```

```

    } catch (e: CancellationException) { throw e }
    catch (e: Exception) { WeatherUiState.Error(e.toUserMessage()) }
}

// data/remote/NetworkErrors.kt - зрозумілі повідомлення
fun Throwable.toUserMessage() = when (this) {
    is SocketTimeoutException -> "Сервер не відповідає. Спробуйте пізніше"
    is IOException -> "Немає з'єднання з інтернетом"
    is HttpException -> "Помилка HTTP ${code()}" // ... окремо 401, 404, 5xx
    else -> "Невідома помилка"
}

```

Інтерфейс `WeatherApi` описує один запит. Анотація `@GET("v1/forecast")` задає метод і шлях, а кожен параметр з анотацією `@Query` стає параметром адреси: `Retrofit` сам складе `baseUrl + "v1/forecast?latitude=50.45&longitude=30.52¤t=..."`. Параметр `fields` має значення за замовчуванням з переліком потрібних показників. Функцію позначено `suspend`, тож `Retrofit` виконає запит у фоновому потоці, а результат одразу перетворить з JSON на об'єкт `WeatherResponseDto`.

Поля DTO збігаються з полями JSON, а анотація `@SerializedName("temperature_2m")` зв'язує незручну для Kotlin назву з властивістю `temperature`. `NetworkModule` створює спільний `OkHttpClient` з тайм-аутами 15 секунд і логуванням тіла запитів лише в `debug`-збірці, а налаштування `Json { ignoreUnknownKeys = true }` дозволяє не описувати в DTO поля, які застосунку не потрібні. Репозиторій перетворює DTO на доменну модель `Weather` з текстовим описом погоди, тому інтерфейс не залежить від формату сервера.

`WeatherUiState` – це `sealed interface` з трьома варіантами: `Loading`, `Success` з даними і `Error` з повідомленням. Функція `loadWeather()` спершу встановлює стан `Loading`, потім у блоці `try` виконує запит і встановлює `Success`. Будь-яка помилка перетворюється функцією `toUserMessage()` на зрозумілий текст: `SocketTimeoutException` – «Сервер не відповідає», `IOException` – «Немає з'єднання з інтернетом», `HttpException` – повідомлення з кодом відповіді. Окремо перехоплюється `CancellationException` і передається далі: це не помилка, а сигнал скасування корутини, наприклад коли користувач закрив екран. Картка `WeatherCard` обробляє три стани в `when (state)`, і компілятор перевіряє, що жоден варіант не пропущено.

Запуск на емуляторі. Запустіть застосунок – картка покаже реальну погоду. Увімкніть на емуляторі режим «У літаку» і натисніть «Повторити» – з'явиться «Немає з'єднання з інтернетом». Запити видно у `Logcat` з фільтром `tag: okhttp`.

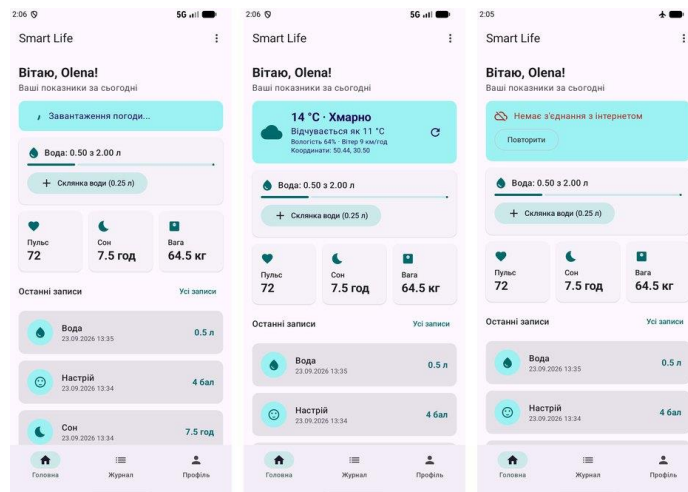


Рисунок 3.5 – Картка погоди: завантаження, успішна відповідь Open-Meteo, помилка без мережі

На рисунку 3.5 показано три стани картки погоди на головному екрані. Ліворуч – стан завантаження: у картці крутиться індикатор і виводиться напис «Завантаження погоди...». Посередині – успішна відповідь Open-Meteo: «14 °C · Хмарно», «Відчувається як 11 °C», вологість 64 %, вітер 9 км/год, координати і кнопка оновлення. Праворуч – стан помилки після ввімкнення режиму «У літаку» (значок літака в рядку стану): текст «Немає з'єднання з інтернетом».

з'єднання з інтернетом» і кнопка «Повторити». Решта головного екрана в усіх трьох станах працює як зазвичай.

3.5 Типові помилки

Локальні дані. Помилка *Cannot find implementation for AppDatabase* під час запуску означає, що генератор коду Room не підключено: додайте `ksp(libs.room.compiler)`. Виняток *Cannot access database on the main thread* виникає, коли DAO викликано поза корутиною, – позначайте методи `suspend` і викликайте їх з `viewModelScope`.

Якщо після зміни класу `@Entity` застосунок падає з повідомленням *Room cannot verify the data integrity*, структура таблиці змінилася, а версія бази – ні: збільште `version` і опишіть міграцію, а під час навчання можна просто перевстановити застосунок. Для `DataStore` важливо оголошувати `preferencesDataStore` лише один раз на рівні файлу – два екземпляри для одного файлу призводять до помилок.

Мережа. Якщо запит завершується винятком *SecurityException: Permission denied (missing INTERNET permission?)*, у маніфесті немає дозволу `INTERNET`. Виняток *NetworkOnMainThreadException* виникає при спробі виконати мережевий запит у головному потоці – використовуйте `suspend`-функції Retrofit і корутини. Retrofit також вимагає, щоб адреса в `baseUrl()` закінчувалася косою рисою, інакше з'являється помилка *base URL must end in /*.

Помилки *MissingFieldException* або *Unexpected JSON token* означають, що DTO не відповідає реальному JSON: звірте назви і типи полів, а для полів, яких у відповіді може не бути, задайте значення за замовчуванням. Якщо ж на емуляторі взагалі немає інтернету, перезапустіть його з параметром `-dns-server 8.8.8.8`.

Окремо варто пам'ятати про корутини: не використовуйте `GlobalScope` – корутини в ньому не скасовуються разом з екраном; не перехоплюйте `CancellationException` загальним `catch (e: Exception)` без повторного кидання; не викликайте блокувальні операції (`Thread.sleep`, читання файлу) у `Dispatchers.Main`.

3.6 Практичні завдання

Практичні завдання розділу наведено в таблиці 3.6. Основне завдання виконується на основі версії Smart Life, отриманої в підрозділі, а завдання для самостійної роботи поглиблює тему і може оцінюватися додатково.

Таблиця 3.6 – Практичні завдання розділу

Тема	Практичне завдання	Завдання для самостійної роботи
Локальні дані	Додайте в DAO запит <code>observeByType (type: String)</code> і пошук за нотаткою (<code>LIKE '%' // :query // '%'</code>), виведіть результат у журналі.	Реалізуйте експорт записів у CSV-файл у внутрішній пам'яті і відправлення його через <code>Intent.ACTION_SEND (FileProvider)</code> .
Мережа	Додайте прогноз на 3 дні (параметр <code>daily=temperature_2m_max, temperature_2m_min</code>) і покажіть його рядком під поточною погодою.	Кешуйте останню успішну відповідь у <code>DataStore</code> і показуйте її з позначкою «дані застаріли», якщо мережі немає.

3.7 Контрольні питання

1. Коли використовувати `DataStore`, а коли базу даних?
2. Яке призначення `@Entity`, `@Dao`, `@Database`?
3. Чому DAO-запит повертає `Flow`?
4. Що таке міграція бази даних?
5. Навіщо окремі класи `RecordEntity` і `HealthRecord`?
6. З яких частин складається HTTP-запит?
7. Чим відрізняються коди 4xx і 5xx?
8. Навіщо окремі DTO і доменна модель?
9. Чому стан запиту описують `sealed interface`?
10. Чому не можна виконувати мережеві запити в головному потоці?

РОЗДІЛ 4. АПАРАТНІ МОЖЛИВОСТІ ПРИСТРОЮ

4.1 Мета розділу та результати навчання

Мета розділу – навчити використовувати апаратні можливості смартфона: Bluetooth Low Energy, геолокацію, карти, камеру і сенсори, коректно запитуючи в користувача дозволи. У Smart Life з'являються вкладки «Карта» і «Пристрій», фото профілю та прив'язка записів до місця. Результати навчання наведено в таблиці 4.1, основні поняття – в таблиці 4.2.

Таблиця 4.1 – Результати навчання за темами розділу

Тема	Що студент знає і вміє після опрацювання
Дозволи	Звичайні і небезпечні дозволи; оголошення в маніфесті і запит під час роботи; алгоритм обробки відмови; спільна функція <code>rememberPermissionsState</code>
Bluetooth LE	Відмінності Bluetooth Classic і BLE; етапи взаємодії (advertising, scanning, connecting, discovery, data); структуру GATT (сервіс, характеристика, дескриптор, UUID); дозволи <code>BLUETOOTH_SCAN</code> і <code>BLUETOOTH_CONNECT</code> ; перетворення <code>callback-API</code> на <code>Flow</code> через <code>callbackFlow</code> ; обмеження емулятора.
Геолокація і карти	Джерела координат (GPS, Wi-Fi, мобільна мережа); різницю між <code>ACCESS_FINE_LOCATION</code> і <code>ACCESS_COARSE_LOCATION</code> ; алгоритм запиту небезпечного дозволу; методи <code>lastLocation</code> , <code>getCurrentLocation</code> , <code>requestLocationUpdates</code> ; перетворення <code>Task</code> на <code>suspend</code> через <code>await()</code> ; задавання координат на емуляторі.
Камера	Способи роботи з камерою (системний застосунок через контракт, <code>CameraX</code>); контракти <code>TakePicture</code> , <code>PickVisualMedia</code> , <code>RequestPermission</code> ; схему <code>content://Uri</code> і <code>FileProvider</code> ; системний <code>Photo Picker</code> ; копіювання файлу у внутрішню пам'ять; завантаження зображень <code>AsyncImage</code> .
Сенсори	Типи сенсорів (руху, положення, середовища); систему координат пристрою; класи <code>SensorManager</code> , <code>Sensor</code> , <code>SensorEventListener</code> ; частоту опитування; обов'язкову відписку від сенсорів; обчислення нахилу за акселерометром; віртуальні сенсори емулятора.

Таблиця 4.2 – Основні поняття розділу

Поняття	Визначення
BLE	Bluetooth Low Energy – енергоощадний режим Bluetooth
GATT	профіль, що описує дані пристрою як сервіси і характеристики
RSSI	рівень сигналу в dBm: -40 – поруч, -90 – далеко
notify	режим, у якому пристрій сам надсилає нові значення
callbackFlow	побудова <code>Flow</code> з <code>API</code> на функціях зворотного виклику
Fused Location Provider	сервіс Google Play, що поєднує GPS, Wi-Fi і мобільну мережу
небезпечний дозвіл	дозвіл, який користувач підтверджує під час роботи застосунку
точність (accuracy)	радіус у метрах, у межах якого знаходиться пристрій
await()	перетворення <code>Task</code> Google Play services на <code>suspend</code> -виклик
Плитка (tile)	фрагмент карти для певного масштабу і координат
маркер	позначка точки на карті з заголовком і описом
overlay	шар поверх карти (маркери, лінії, обробник дотиків)
API-ключ	ідентифікатор проєкту для доступу до платних API Google
AndroidView	<code>composable</code> -функція для показу класичного <code>View</code>
ActivityResult API	механізм запуску іншої Activity/запиту дозволу з отриманням результату
контракт	опис вхідних даних і результату (<code>TakePicture</code> , <code>PickVisualMedia</code>)
FileProvider	компонент, що видає безпечні <code>content://Uri</code> на файли застосунку
Photo Picker	системний вибір фото без дозволу на доступ до галереї
Coil	бібліотека завантаження зображень для <code>Compose</code>
Акселерометр	вимірює прискорення по трьох осях, м/с ²
гіроскоп	вимірює швидкість обертання навколо осей, рад/с
датчик наближення	визначає, чи є об'єкт біля екрана
SensorEventListener	інтерфейс для отримання нових значень сенсора
runningFold	оператор <code>Flow</code> , що накопичує стан з урахуванням попереднього значення

4.2 Дозволи Android

Android захищає приватні дані користувача і чутливі можливості пристрою системою **дозволів**. Застосунок оголошує потрібні дозволи в маніфесті елементами `<uses-permission>`, але цього достатньо лише для **звичайних** дозволів (normal), які не загрожують приватності, наприклад доступ до інтернету: система надає їх під час встановлення. **Небезпечні** дозволи (dangerous) – камера, геолокація, пошук Bluetooth-пристроїв, сповіщення – користувач підтверджує в системному діалозі під час роботи застосунку і може будь-коли відкликати в налаштуваннях. Дозволи, які використовує Smart Life, наведено в таблиці 4.3.

Таблиця 4.3 – Дозволи застосунку Smart Life

Дозвіл	Тип	З версії Android	Для чого потрібен
INTERNET	звичайний	–	погода, авторизація, плитки карти
ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION	небезпечний	6.0	координати, погода «тут», карта
BLUETOOTH_SCAN, BLUETOOTH_CONNECT	небезпечний	12	пошук і підключення пульсометра
CAMERA	небезпечний	6.0	фото профілю
POST_NOTIFICATIONS	небезпечний	13	нагадування і попередження

Правила роботи з небезпечними дозволами такі: запитувати дозвіл лише тоді, коли користувач користується відповідною функцією, а не під час запуску; заздалегідь пояснювати, навіщо він потрібен; коректно працювати і без дозволу (наприклад, показувати погоду для типового міста); після повторної відмови не набридати запитами, а запропонувати відкрити налаштування. Алгоритм показано на рисунку 4.1.

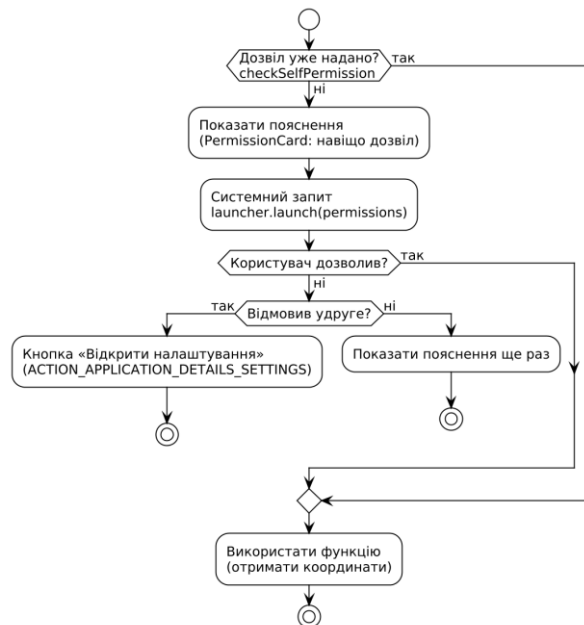


Рисунок 4.1 – Алгоритм запиту небезпечного дозволу

Блок-схема на рисунку 4.1 описує алгоритм, за яким застосунок запитує будь-який небезпечний дозвіл. Якщо дозвіл уже надано, одразу виконується потрібна дія. Якщо ні, спочатку користувачу показується пояснення, а потім системний запит. У разі згоди виконується дія; у разі першої відмови пояснення показується ще раз, а після повторної відмови система більше не показує запит, тому застосунок пропонує кнопку переходу в налаштування застосунку, де дозвіл можна увімкнути вручну.

У Compose дозвіл запитують через Activity Result API контрактом `RequestMultiplePermissions`. Щоб не повторювати цю логіку на кожному екрані, у Smart

Life її винесено в спільну функцію `rememberPermissionsState` (файл `ui/components/Permissions.kt`):

```
@Composable
fun rememberPermissionsState(permissions: List<String>, requireAll: Boolean = true):
PermissionsState {
    val context = LocalContext.current
    var granted by remember { mutableStateOf(context.hasPermissions(permissions, requireAll)) }
    var deniedOnce by rememberSaveable { mutableStateOf(false) }
    val launcher = rememberLauncherForActivityResult(RequestMultiplePermissions()) { result ->
        granted = if (requireAll) result.values.all { it } else result.values.any { it }
        if (!granted) deniedOnce = true
    }
    LifecycleResumeEffect(permissions) { // користувач міг змінити дозвіл у
налаштуваннях
        granted = context.hasPermissions(permissions, requireAll)
        onPauseOrDispose { }
    }
    return PermissionsState(granted, deniedOnce) { launcher.launch(permissions.toTypedArray()) }
}
```

Функція зберігає два значення: `granted` – чи надано дозволи зараз, і `deniedOnce` – чи відмовляв користувач хоча б раз (зберігається в `rememberSaveable`, щоб пережити поворот). Лаунчер показує системний діалог і в колбеку оновлює обидва значення; параметр `requireAll` визначає, чи потрібні всі дозволи групи, чи достатньо одного (для геолокації підходить і приблизна). `LifecycleResumeEffect` повторно перевіряє дозволи щоразу, коли екран стає активним, – користувач міг увімкнути дозвіл у налаштуваннях і повернутися. Функція повертає об'єкт `PermissionsState` зі станом і функцією запиту, а картка `PermissionCard` на його основі показує пояснення, кнопку «Надати дозвіл» або, після відмови, кнопку «Відкрити налаштування».

4.3 Bluetooth Low Energy

Bluetooth Classic призначений для безперервного потоку даних (навушники, колонки), **BLE** – для рідкісного обміну невеликими порціями даних з мінімальним енергоспоживанням (фітнес-браслети, пульсометри, датчики). Взаємодія BLE проходить етапи: пристрій-периферія періодично оголошує себе (advertising); телефон сканує ефір і отримує `ScanResult` (ім'я, MAC-адреса, рівень сигналу RSSI); телефон підключається (`connectGatt`), виявляє сервіси (`discoverServices`) і читає, записує або підписується на сповіщення характеристик (рис. 4.2).

Дані BLE-пристрою організовано за профілем **GATT**: **сервіс** – логічна група (Heart Rate, UUID `0x180D`), **характеристика** – конкретне значення (Heart Rate Measurement, `0x2A37`, властивість *notify*), **дескриптор** – метадані; щоб отримувати сповіщення, клієнт записує значення «увімкнути» в дескриптор **CCCD** (`0x2902`). Стандартні UUID визначає Bluetooth SIG.

З Android 12 сканування і підключення вимагають **небезпечних дозволів** `BLUETOOTH_SCAN` і `BLUETOOTH_CONNECT`, які користувач підтверджує в системному діалозі; на старіших версіях для сканування потрібен дозвіл на геолокацію. Загальний алгоритм запиту дозволів розглянуто в підрозділі 4.2 (рис. 4.1).

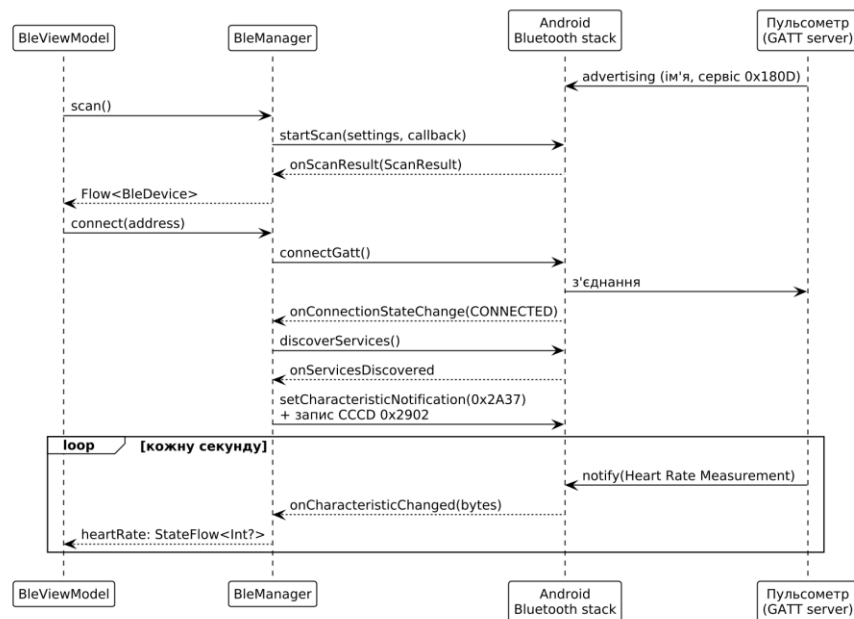


Рисунок 4.2 – Взаємодія застосунку з BLE-пульсометром

На рисунку 4.2 показано обмін повідомленнями між чотирма учасниками. Пульсометр постійно оголошує себе в ефірі. ViewModel запускає сканування через BleManager, система повертає результати, і вони потрапляють у ViewModel як потік пристроїв. Коли користувач обирає пристрій, BleManager викликає connectGatt, система встановлює з'єднання і повідомляє про нього, після чого запускається пошук сервісів і вмикаються сповіщення характеристики пульсу. Блок loop унизу означає, що далі пульсометр щосекунди надсилає нове значення, яке доходить до ViewModel як heartRate.

Стандартні UUID сервісів і характеристик, з якими працює Smart Life, визначено специфікаціями Bluetooth SIG (табл. 4.4). Повний 128-бітний UUID утворюється підстановкою короткого 16-бітного коду в базовий UUID 0000xxxx-0000-1000-8000-00805f9b34fb.

Таблиця 4.4 – Стандартні елементи GATT, що використовуються в Smart Life

Елемент	Короткий UUID	Властивості	Зміст
сервіс Heart Rate	0x180D	—	група даних пульсометра
характеристика Heart Rate Measurement	0x2A37	notify	прапорці та значення пульсу
дескриптор CCCD	0x2902	read, write	вмикання сповіщень характеристики
сервіс Battery	0x180F	—	рівень заряду пристрою
характеристика Battery Level	0x2A19	read, notify	заряд у відсотках

Практичний приклад. Smart Life: екран BLE з поясненням і запитом дозволів, пропозицією увімкнути Bluetooth, скануванням, списком знайдених пристроїв з RSSI, підключенням, переліком сервісів і показом пульсу; кнопка «Зберегти в журнал» додає запис пульсу.

Почніть з маніфесту. Оголосіть дозвіл `BLUETOOTH_SCAN` з прапорцем `neverForLocation` (застосунок не визначає за Bluetooth місце користувача), дозвіл `BLUETOOTH_CONNECT`, старі дозволи `BLUETOOTH` і `BLUETOOTH_ADMIN` з обмеженням `maxSdkVersion="30"` для Android 11 і старіших, а також `<uses-feature android:name="android.hardware.bluetooth_le" android:required="false"/>` — застосунок можна буде встановити і на пристрій без BLE.

Далі створіть клас `ble/BleManager.kt`, який приховує роботу з системним Bluetooth API: сканування, підключення і розбір пакета пульсу. Оскільки дозволи

знадобляться і в наступних розділах, винесіть роботу з ними в спільний файл `ui/components/Permissions.kt` з функцією `rememberPermissionsState` і карткою-поясненням `PermissionCard`. Наостанок створіть `BleViewModel`, екран `BleScreen` і маршрут `BleRoute`, а `bleManager` додайте в контейнер залежностей.

Ключові фрагменти реалізації наведено нижче.

```
// ble/BleManager.kt - сканування як Flow
fun scan(): Flow<BleDevice> = callbackFlow {
    val scanner = adapter?.bluetoothLeScanner
    if (scanner == null) {
        close(IllegalStateException("Bluetooth вимкнено")); return@callbackFlow
    }
    val callback = object : ScanCallback() {
        override fun onScanResult(type: Int, result: ScanResult) {
            trySend(BleDevice(result.device.name, result.device.address, result.rssi))
        }
    }
    scanner.startScan(null, ScanSettings.Builder()
        .setScanMode(ScanSettings.SCAN_MODE_LOW_LATENCY).build(), callback)
    awaitClose { scanner.stopScan(callback) } // зупинка при скасуванні
}

// Підключення і підписка на пульс
override fun onServicesDiscovered(gatt: BluetoothGatt, status: Int) {
    val ch = gatt.getService(HEART_RATE_SERVICE)
        ?.getCharacteristic(HEART_RATE_MEASUREMENT) ?: return
    gatt.setCharacteristicNotification(ch, true)
    val cccd = ch.getDescriptor(CLIENT_CONFIG_DESCRIPTOR) ?: return
    val enable = BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        gatt.writeDescriptor(cccd, enable) // Android 13+
    } else {
        cccd.value = enable; gatt.writeDescriptor(cccd) // Android 8-12 (deprecated)
    }
}

// Android 13+: значення приходить параметром
override fun onCharacteristicChanged(gatt: BluetoothGatt,
    ch: BluetoothGattCharacteristic, value: ByteArray) {
    _heartRate.value = parseHeartRate(value) // біт 0 прапорців: UINT8 або UINT16
}

// Android 8-12: старий overload, значення читається з характеристики
override fun onCharacteristicChanged(gatt: BluetoothGatt,
    ch: BluetoothGattCharacteristic) {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU)
        _heartRate.value = parseHeartRate(ch.value)
}

// ... connect(): device.connectGatt(context, false, callback, TRANSPORT_LE)

// ui/ble/BleScreen.kt - дозволи залежно від версії Android
private val blePermissions = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S)
    listOf(BLUETOOTH_SCAN, BLUETOOTH_CONNECT) else listOf(ACCESS_FINE_LOCATION)
```

Функція `scan()` перетворює системний API сканування, побудований на колбеках, на потік `Flow`. У середині `callbackFlow` спочатку перевіряється, чи є сканер: якщо `Bluetooth` вимкнено, потік закривається з помилкою. Потім створюється об'єкт `ScanCallback`, у якому метод `onScanResult` викликається системою для кожного знайденого пристрою; з результату беруться ім'я, MAC-адреса і рівень сигналу RSSI, і через `trySend` об'єкт `BleDevice` надсилається в потік. Виклик `startScan` запускає сканування, а блок `awaitClose { scanner.stopScan(callback) }` виконується, коли збір потоку буде скасовано, і гарантовано зупинить сканування. `ViewModel` збирає цей потік не довше 10 секунд (`withTimeoutOrNull`), бо сканування активно витрачає батарею.

Після підключення (`connectGatt`) система викликає `onServicesDiscovered`, коли дізнається структуру пристрою. У цьому методі за стандартним UUID знаходиться сервіс `Heart Rate` і його характеристика вимірювання пульсу. Виклик `setCharacteristicNotification` вмикає обробку сповіщень на телефоні, а запис значення `ENABLE_NOTIFICATION_VALUE` у дескриптор `CCCD` просить сам пульсометр надсилати нові значення. Оскільки проєкт підтримує `minSdk 26`, обидві операції мають дві версії. В `Android 13` (API 33) з'явилися нові перевантаження `writeDescriptor(descriptor, value)` і `onCharacteristicChanged(gatt,`

`characteristic, value`), які передають значення параметром, а старі варіанти, що працюють через поле `value`, стали застарілими (`deprecated`). На Android 12 і старіших система викликає лише старий `onCharacteristicChanged(gatt, characteristic)`, тому без нього застосунок підключився б до пульсометра, але не отримував би значень. Тому запис дескриптора виконується з перевіркою `Build.VERSION.SDK_INT`, а в колбеку реалізовано обидва перевантаження; перевірка версії у старому варіанті виключає подвійну обробку на нових пристроях. Відтоді кожне вимірювання приходить у `onCharacteristicChanged`, де функція `parseHeartRate` читає байт прапорців: якщо його нульовий біт дорівнює 0, пульс займає один байт, інакше – два. Результат записується в `StateFlow`, на який підписана `ViewModel`.

Запуск на емуляторі. Емулятор імітує Bluetooth-адаптер, але поруч немає реальних BLE-пристроїв, тому на емуляторі перевіряються дозволи, увімкнення Bluetooth і запуск сканування (список порожній). Для перевірки підключення використайте телефон (додаток Г) і реальний пульсометр або застосунок **nRF Connect** на другому телефоні в режимі *Advertiser* з сервісом Heart Rate.

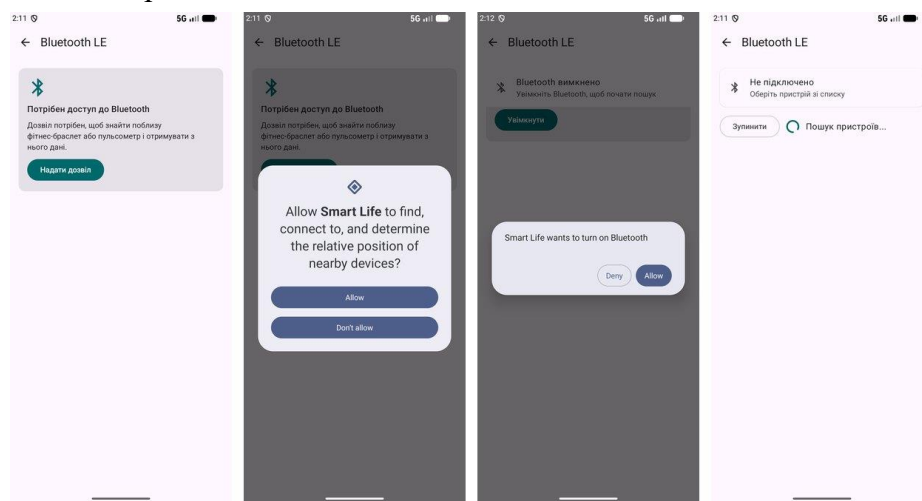


Рисунок 4.3 – Екран BLE на емуляторі: пояснення дозволу, системний запит, увімкнення Bluetooth, сканування

На рисунку 4.3 показано послідовність дій на емуляторі. Спочатку застосунок пояснює користувачу, навіщо потрібен доступ до Bluetooth, і пропонує кнопку «Надати дозвіл». Після натискання з'являється системний діалог Android «Allow Smart Life to find, connect to, and determine the relative position of nearby devices?». Коли дозвіл надано, але Bluetooth вимкнено, застосунок показує картку «Bluetooth вимкнено» і системний запит «Smart Life wants to turn on Bluetooth». Останній екран – пошук: картка стану «Не підключено», кнопка «Зупинити» та індикатор «Пошук пристроїв...»; на емуляторі поруч немає BLE-пристроїв, тому список залишається порожнім.

4.4 Геолокація і карти

Смартфон визначає місце за супутниками GPS (точність до кількох метрів, але повільно і енерговитратно), за точками Wi-Fi і базовими станціями (швидко, точність десятки – сотні метрів). **Fused Location Provider** з Google Play services автоматично поєднує ці джерела з урахуванням пріоритету: `PRIORITY_HIGH_ACCURACY` (GPS), `PRIORITY_BALANCED_POWER_ACCURACY` (близько 100 м).

Геолокація – **небезпечний дозвіл**. `ACCESS_FINE_LOCATION` дає точні координати, `ACCESS_COARSE_LOCATION` – приблизне місцезнаходження з меншою точністю (фактична точність залежить від пристрою та доступних джерел позиціонування); з Android 12 користувач може обрати лише приблизну точність, тому запитують обидва дозволи. Для роботи у фоні потрібен окремий `ACCESS_BACKGROUND_LOCATION`, який у курсі не використовується. Коректний запит дозволу (рис. 4.1): перевірити, чи дозвіл уже є;

пояснити користувачу, навіщо він потрібен; показати системний діалог; при повторній відмові запропонувати відкрити налаштування застосунку.

Методи провайдера: `lastLocation` – останні відомі координати (миттєво, але можуть бути `null`); `getCurrentLocation()` – свіжі координати; `requestLocationUpdates()` – періодичні оновлення, від яких обов'язково треба відписатися.

Карта складається з **плиток** (tiles) – квадратних зображень 256×256 пікселів, що завантажуються з сервера для кожного рівня масштабу. Поверх карти додаються **шари** (overlays): маркери, лінії маршрутів, полігони, обробники дотиків.

Google Maps SDK for Android (бібліотека `maps-compose`) – найпоширеніший варіант, але вимагає **API-ключ**: у Google Cloud Console створюють проєкт, вмикають *Maps SDK for Android*, створюють ключ (*Credentials* ▶ *Create credentials* ▶ *API key*) і обмежують його назвою пакета та SHA-1 відбитком сертифіката підпису (`./gradlew signingReport`). Ключ не можна зберігати в git: його записують у `local.properties` (`MAPS_API_KEY=...`) і підставляють у маніфест через плагін *Secrets Gradle Plugin* як `<meta-data android:name="com.google.android.geo.API_KEY" android:value="${MAPS_API_KEY}"/>`. Типові помилки: сірий екран замість карти і в Logcat *Authorization failure* – ключ відсутній, не ввімкнено Maps SDK, не збігається пакет або SHA-1 (debug і release мають різні відбитки), не підключено платіжний акаунт; на емуляторі потрібен образ *Google APIs* або *Google Play*.

Щоб приклад запускався без ключа і платіжного акаунта, у Smart Life використано **osmdroid** – бібліотеку для карт **OpenStreetMap**. Карта `osmdroid` є класичним Android View, тому в Compose її вбудовують через `AndroidView`, а методи `onResume()/onPause()` викликають відповідно до життєвого циклу екрана. `osmdroid` використано в посібнику з навчальною метою: на момент підготовки посібника репозиторій бібліотеки перебуває в архівному стані (з листопада 2024 року) і не розвивається, тому для нових комерційних проєктів варто обирати активно підтримувані рішення – Google Maps SDK (`maps-compose`) або MapLibre.

Основні відмінності двох бібліотек наведено в таблиці 4.5.

Таблиця 4.5 – Порівняння картографічних бібліотек

Критерій	Google Maps SDK (<code>maps-compose</code>)	osmdroid (OpenStreetMap)
API-ключ	обов'язковий, з обмеженням за пакетом і SHA-1	не потрібен
Інтеграція з Compose	нативні composable-функції	через <code>AndroidView</code>
Вимоги до пристрою	Google Play services, образ з Google APIs	будь-який пристрій з інтернетом
Дані карти	Google	OpenStreetMap (ліцензія ODbL, потрібен підпис)
Додатково	Street View, пошук місць, маршрути (платні API)	офлайн-кеш плиток, власні джерела плиток

Практичний приклад. Smart Life: екран «Моє місцезнаходження» з координатами, точністю, висотою і режимом відстеження; погода на головному екрані для поточних координат; кожен новий запис зберігає координати, які показуються на екрані деталей.

Спершу підключіть бібліотеки `play-services-location` (Fused Location Provider) і `kotlinx-coroutines-play-services` (функція `await()`), а в маніфесті оголошіть обидва дозволи – `ACCESS_FINE_LOCATION` і `ACCESS_COARSE_LOCATION`.

Потім створіть клас `location/LocationClient.kt`. Разом із ним оголошіть невеликий інтерфейс `LocationSource` з методом отримання останніх координат: `ViewModel` залежатимуть від інтерфейсу, і в тестах його можна буде замінити фейком. Далі створіть `LocationViewModel` і екран `LocationScreen` та додайте пункт «Геолокація» на екран «Пристрій». Наостанок використайте координати у двох місцях:

HomeViewModel.loadWeather() завантажуватиме погоду для місця користувача, а RecordsViewModel.addRecord() зберігатиме координати разом із записом.

Ключові фрагменти реалізації наведено нижче.

```
// location/LocationClient.kt
suspend fun getCurrentLocation(): LocationData? {
    if (!hasPermission()) return null
    return fusedClient
        .getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, CancellationTokenSource().token)
        .await()?.toLocationData() // Task -> suspend
}

fun locationUpdates(intervalMs: Long = 5_000): Flow<LocationData> = callbackFlow {
    val request = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, intervalMs).build()
    val callback = object : LocationCallback() {
        override fun onLocationResult(r: LocationResult) {
            r.lastLocation?.let { trySend(it.toLocationData()) }
        }
    }
    fusedClient.requestLocationUpdates(request, callback, Looper.getMainLooper())
    awaitClose { fusedClient.removeLocationUpdates(callback) } // інакше GPS не вимкнеться
}

// ui/location/LocationScreen.kt - достатньо одного з двох дозволів
val permissions = rememberPermissionsState(
    listOf(ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION), requireAll = false)
if (!permissions.granted) PermissionCard(...) // пояснення + кнопка запиту / налаштувань

// ui/records/RecordsViewModel.kt - запис прив'язується до місця
val location = runCatching { locationSource.getLastLocation() }.getOrNull()
recordRepository.addRecord(HealthRecord(type = type, value = value, note = note,
    latitude = location?.latitude, longitude = location?.longitude))
```

Функція `getCurrentLocation()` спершу перевіряє дозвіл і, якщо його немає, повертає `null` замість того, щоб падати. Потім вона просить Fused Location Provider визначити свіже місцезнаходження з високою точністю. Метод провайдера повертає `Task` з Google Play services, а функція `await()` перетворює його на звичайний `suspend`-виклик, тож код виглядає послідовним, хоча чекання відбувається асинхронно. Результат перетворюється на власний клас `LocationData` з широтою, довготою, точністю, висотою і швидкістю.

`locationUpdates()` повертає потік періодичних оновлень. Об'єкт `LocationRequest` задає пріоритет і інтервал (5 секунд), а `LocationCallback` отримує кожен нову позицію і надсилає її в потік через `trySend`. Блок `awaitClose` відписується від оновлень, коли потік перестають збирати, – без цього GPS продовжував би працювати й розряджати батарею навіть після закриття екрана.

В останньому фрагменті `RecordsViewModel` перед збереженням запису пробує отримати останні відомі координати. Виклик загорнуто в `runCatching { }.getOrNull()`: якщо дозволу немає або координати невідомі, запис просто збережеться без місця, а не завершиться помилкою. Координати потрапляють у поля `latitude` і `longitude` моделі, а звідти – у стовпці бази, підготовлені в підрозділі 3.3.

Запуск на емуляторі. Відкрийте *Extended Controls* емулятора (кнопка «...») ► *Location*, задайте точку на карті і натисніть *Set location*. На екрані геолокації дозвольте доступ *While using the app* – з'являться координати; змініть точку – у режимі відстеження значення оновляться. Координати можна задати і командою `adb emu geo fix 30.5234 50.4501`.

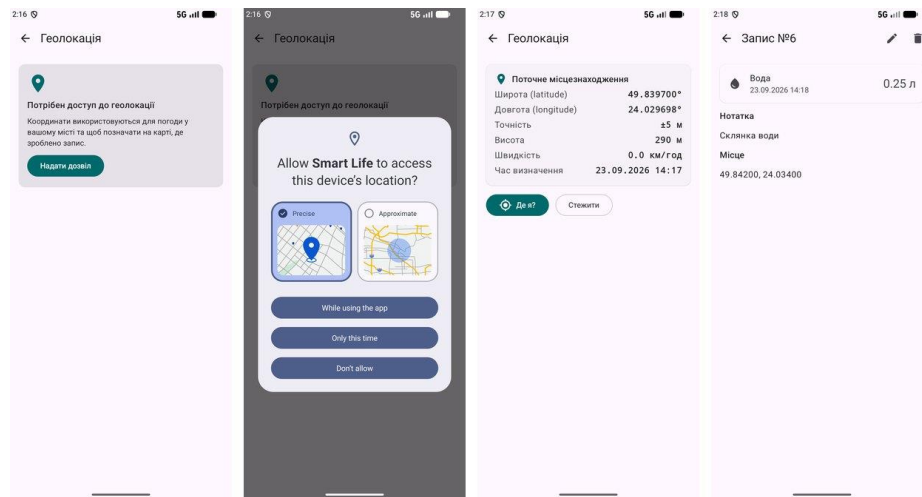


Рисунок 4.4 – Геолокація: пояснення дозволу, системний запит, координати пристрою, запис з місцем

На рисунку 4.4 перший екран – картка «Потрібен доступ до геолокації» з поясненням, що координати використовуються для погоди і позначок на карті. Другий – системний діалог Android, у якому користувач обирає точне (Precise) або приблизне (Approximate) місцезнаходження і варіант дозволу: «While using the app», «Only this time» або «Don't allow». Третій екран – отримані координати: широта 49.8397°, довгота 24.0297°, точність ± 5 м, висота 290 м, швидкість і час визначення, а також кнопки «Де я?» і «Стежити». Четвертий – деталі запису води, де з'явився розділ «Місце» з координатами, збереженими під час створення запису.

Прив'язка записів до місця відкриває шлях до карти. Нижче описано реалізацію вкладки «Карта».

Практичний приклад. Smart Life: вкладка «Карта» показує маркери всіх записів з координатами (з типом і значенням у підказці), позначку «Ви тут», кнопки масштабу і «показати всі записи»; дотик до карти ставить точку і показує її координати та відстань від користувача.

Спершу підключіть бібліотеку `org.osmdroid:osmdroid-android`; дозвіл `INTERNET`, потрібний для завантаження плиток, уже є з підрозділу 3.4. Потім створіть `MapViewModel`, який надає три потоки даних: записи, що мають координати, поточну позицію користувача і точку, обрану дотиком до карти.

Далі створіть екран `MapScreen`. Карта `osmdroid` є класичним View, тому для неї знадобиться допоміжна функція `rememberMapViewWithLifecycle()`, яка створює карту і пов'язує її з життєвим циклом екрана. Наостанок додайте у `res/drawable` векторні іконки маркерів (запис, «Ви тут», обрана точка) і вкладку «Карта» в нижню панель навігації.

Ключові фрагменти реалізації наведено нижче.

```
// ui/map/MapScreen.kt - карта, прив'язана до життєвого циклу
@Composable
private fun rememberMapViewWithLifecycle(): MapView {
    val context = LocalContext.current
    val mapView = remember {
        configureOsmdroid(context) // User-Agent і кеш плиток - вимога OpenStreetMap
        MapView(context).apply {
            setTileSource(TileSourceFactory.MAPNIK)
            setMultiTouchControls(true)
            controller.setZoom(12.0); controller.setCenter(KYIV)
        }
    }
    val lifecycle = LocalLifecycleOwner.current.lifecycle
    DisposableEffect(lifecycle) {
        val observer = LifecycleEventObserver { _, e ->
            if (e == Lifecycle.Event.ON_RESUME) mapView.onResume()
            if (e == Lifecycle.Event.ON_PAUSE) mapView.onPause()
        }
        lifecycle.addObserver(observer)
        onDispose { lifecycle.removeObserver(observer); mapView.onDetach() }
    }
}
```

```

return mapView
}

// Шари: маркери записів, «Ви тут», обрана точка
records.forEach { record ->
    overlays.add(Marker(this).apply {
        position = GeoPoint(record.latitude!!, record.longitude!!)
        title = "${record.type.title}: ${record.formattedValue()}"
        snippet = formatDate(record.timestamp)
    })
}
// ... MapEventsOverlay: singleTapConfirmedHelper -> viewModel.selectPoint(...)

// Відстань від користувача до обраної точки (по поверхні Землі, у метрах)
val meters = GeoPoint(me.latitude, me.longitude)
    .distanceToAsDouble(GeoPoint(point.latitude, point.longitude))

```

Об'єкт `MapView` створюється всередині `remember`, тому під час рекомпозицій карта не перестворюється і не втрачає масштаб та положення. Перед створенням викликається `configureOsmdroid`: сервери `OpenStreetMap` вимагають, щоб кожен клієнт мав власний `User-Agent`, а плитку кешуються в папці застосунку. Далі задаються джерело плиток `MAPNIK` (стандартний вигляд `OpenStreetMap`), масштабування двома пальцями, початковий масштаб і центр карти.

`DisposableEffect` підписує спостерігача на події життєвого циклу екрана: при `ON_RESUME` карта відновлює завантаження плиток, при `ON_PAUSE` – призупиняє його. Блок `onDispose` виконується, коли екран зникає: він прибирає спостерігача і звільняє ресурси карти методом `onDetach()`.

Сама карта вставляється в `Compose` функцією `AndroidView`. Параметр `factory` повертає вже створений `mapView`, а блок `update` викликається щоразу, коли змінюються записи, позиція користувача або обрана точка, і перемальовує шари карти. Функція `showOverlays` очищає старі шари і додає нові: маркер для кожного запису із заголовком «тип: значення» і датою, синю позначку «Ви тут», червону позначку обраної точки та прозорий шар, що перехоплює дотики до карти. Відстань від користувача до обраної точки обчислює метод `distanceToAsDouble`, який враховує кривизну Землі й повертає результат у метрах.

Запуск на емуляторі. Задайте координати в *Extended Controls* ► *Location*, додайте кілька записів у різних точках (змінюючи координати між записами) і відкрийте вкладку «Карта». Натисніть маркер – з'явиться підказка; натисніть на порожнє місце – з'явиться картка обраної точки з відстанню.

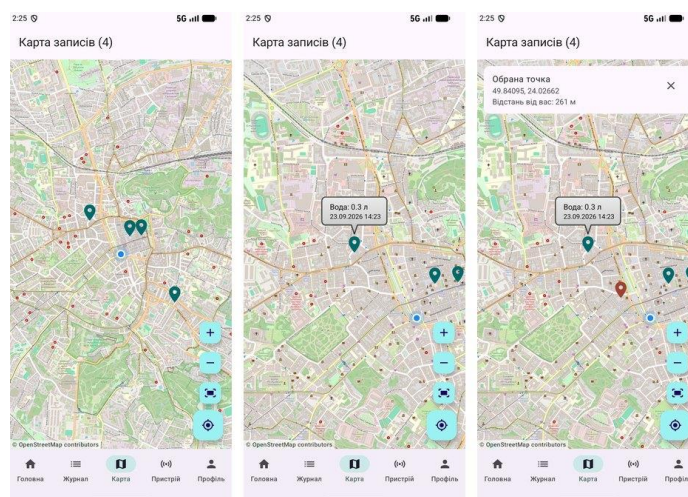


Рисунок 4.5 – Карта записів: маркери і позначка «Ви тут», підказка маркера, обрана точка з відстанню

На рисунку 4.5 ліворуч показано карту Львова з чотирма темно-зеленими маркерами записів і синьою точкою поточної позиції; праворуч розташовано кнопки наближення, віддалення, «показати всі записи» і «моє місцезнаходження», а внизу –

підпис «© OpenStreetMap contributors», якого вимагає ліцензія. Посередині – підказка, що з'являється після натискання на маркер: «Вода: 0.3 л» і час запису. Праворуч – після дотику до вільного місця карти з'явилися червона позначка і картка «Обрана точка» з координатами та відстанню від користувача (261 м).

4.5 Камера і галерея

Найпростіший спосіб отримати фото – попросити системний застосунок камери зробити знімок. Для цього використовують **Activity Result API**: контракт `ActivityResultContracts.TakePicture()` приймає `Uri` файлу, куди камера запише фото, і повертає `true` при успіху. Власний інтерфейс камери (попередній перегляд, фільтри, сканування QR) будують бібліотекою **CameraX**.

Застосунок не може передати іншому застосунку шлях до свого файлу напряму – лише **content:// Uri**, виданий **FileProvider**, з тимчасовим дозволом на запис. `FileProvider` оголошують у маніфесті і вказують дозволені папки в `res/xml/file_paths.xml`.

Для вибору фото з галереї використовують системний **Photo Picker** (`PickVisualMedia`): користувач сам обирає файл, тому застосунку не потрібен дозвіл на читання всієї галереї. Отримане зображення копіюють у внутрішню пам'ять застосунку, бо тимчасовий доступ до `Uri` з галереї може зникнути. Бібліотека **Coil** завантажує зображення у фоні, масштабує і кешує його.

Контракти **Activity Result API**, якими користується **Smart Life**, наведено в таблиці 4.6: кожен з них описує, які дані передаються під час запуску і який результат повертається.

Таблиця 4.6 – Контракти **Activity Result API**, що використовуються в **Smart Life**

Контракт	Вхідні дані	Результат	Використання
<code>RequestPermission</code>	назва дозволу	<code>Boolean</code> – надано чи ні	дозвіл <code>CAMERA</code>
<code>RequestMultiplePermissions</code>	масив дозволів	<code>Map<String, Boolean></code>	геолокація, <code>Bluetooth</code>
<code>TakePicture</code>	<code>Uri</code> файлу	<code>Boolean</code> – чи збережено фото	зйомка камерою
<code>PickVisualMedia</code>	тип медіа	<code>Uri?</code> обраного файлу	вибір фото з галереї
<code>StartActivityForResult</code>	<code>Intent</code>	<code>ActivityResult</code>	запит увімкнення <code>Bluetooth</code>

Практичний приклад. **Smart Life**: натискання на аватар або кнопку «Змінити фото» у профілі відкриває нижню панель з діями «Зробити фото», «Обрати з галереї», «Видалити фото»; фото зберігається у внутрішній пам'яті, шлях до нього – у `DataStore`.

Спершу підключіть бібліотеку `coil-compose`. У маніфесті оголосіть дозвіл `CAMERA` разом з `uses-feature ... required="false"`, щоб застосунок можна було встановити і на пристрій без камери, а також компонент `<provider>` класу `androidx.core.content.FileProvider` з `authority ${applicationId}.fileprovider`.

Потім у файлі `res/xml/file_paths.xml` вкажіть, до якої папки `FileProvider` може видавати доступ: `<cache-path name="camera_images" path="images/">`. Після цього створіть клас `data/profile/ProfilePhotoStorage.kt` для створення файлу під знімок і копіювання фото, а в `SettingsRepository` додайте збереження шляху до фото. Наостанок оновіть `ProfileViewModel` і `ProfileScreen`: додайте лаунчери **Activity Result API**, нижню панель вибору джерела `ModalBottomSheet` і показ фото через `AsyncImage`.

Ключові фрагменти реалізації наведено нижче.

```
// ui/profile/ProfileScreen.kt – три лаунчери Activity Result API
var cameraUri by rememberSaveable { mutableStateOf<Uri?>(null) }
val takePicture = rememberLauncherForActivityResult(TakePicture()) { success ->
    if (success) cameraUri?.let { viewModel::onPhotoSelected }
}
val cameraPermission = rememberLauncherForActivityResult(RequestPermission()) { granted ->
    if (granted) {
        val uri = viewModel.createCameraUri().also { cameraUri = it }
        takePicture.launch(uri)
    }
}
```

```

    } else viewModel.showMessage("Без дозволу на камеру зробити фото неможливо")
}
val pickPhoto = rememberLauncherForActivityResult(PickVisualMedia()) { uri ->
    if (uri != null) viewModel.onPhotoSelected(uri) // дозвіл не потрібен
}
// onTakePhoto = { cameraPermission.launch(Manifest.permission.CAMERA) }
// onPickPhoto = { pickPhoto.launch(PickVisualMediaRequest(PickVisualMedia.ImageOnly)) }

// data/profile/ProfilePhotoStorage.kt
fun createCameraUri(): Uri {
    val file = File(File(context.cacheDir, "images").apply { mkdirs() },
        "camera_${System.currentTimeMillis()}.jpg")
    return FileProvider.getUriForFile(context, "${context.packageName}.fileprovider", file)
}
suspend fun savePhoto(source: Uri): String = withContext(Dispatchers.IO) {
    val target = File(profileDir, "photo_${System.currentTimeMillis()}.jpg")
    context.contentResolver.openInputStream(source)?.use { it.copyTo(target.outputStream()) }
    // ... видалення попередніх фото
    target.absolutePath
}
// Показ: AsyncImage(model = File(photoPath), contentScale = ContentScale.Crop, ...)

```

На екрані профілю зареєстровано три лаунчери. Лаунчер `takePicture` використовує контракт `TakePicture`: він запускає камеру і повертає `true`, якщо знімок успішно записано за переданим `Uri`. Лаунчер `cameraPermission` запитує дозвіл `CAMERA`; у разі згоди `ViewModel` створює файл і `content-Uri`, `Uri` запам'ятовується у змінній `cameraUri` і передається камері, а у разі відмови показується повідомлення. Лаунчер `pickPhoto` відкриває системний `Photo Picker` з фільтром «лише зображення» і повертає `Uri` обраного фото або `null`, якщо користувач нічого не вибрав.

Змінна `cameraUri` зберігається в `rememberSaveable`: поки працює камера, система може знищити `Activity` застосунку, і без збереження ми не знали б, куди записано фото. Функція `createCameraUri()` створює порожній файл у папці `cache/images` і через `FileProvider.getUriForFile` отримує для нього `content-Uri`, яким камера може скористатися, не маючи доступу до файлів застосунку.

Незалежно від джерела, отримане зображення передається в `onPhotoSelected`, а функція `savePhoto` копіює його у внутрішню пам'ять застосунку (`filesDir/profile`) у потоці `Dispatchers.IO`, видаляє попереднє фото і повертає шлях до нового файлу. Кожен файл отримує нове ім'я з часом створення – інакше бібліотека `Coil` показала б стару версію фото зі свого кешу. Шлях зберігається в `DataStore`, а `AsyncImage` завантажує фото у фоні й обрізає його до кола.

Запуск на емуляторі. Камера емулятора показує віртуальну 3D-сцену (налаштовується в *AVD Manager* ▶ *Advanced Settings* ▶ *Camera*). Для перевірки галереї завантажте зображення на емулятор перетягуванням файлу у вікно або командою `adb push avatar.jpg /sdcard/Pictures/`.

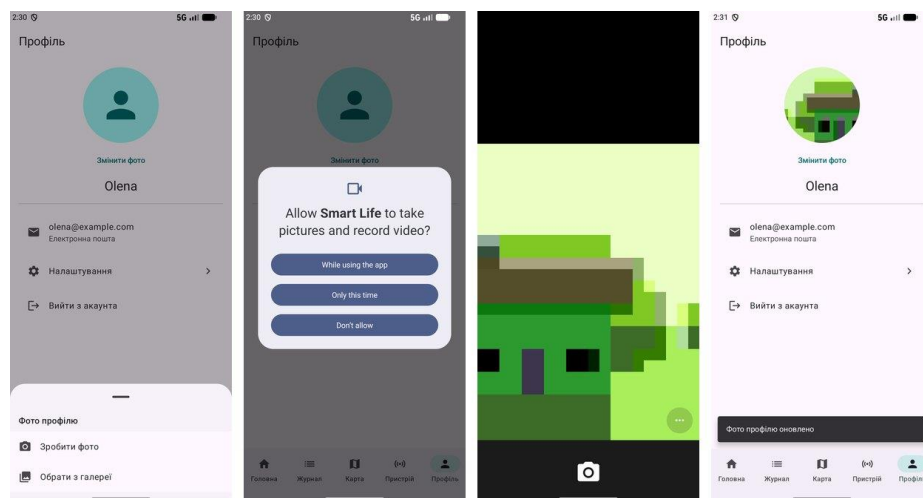


Рисунок 4.6 – Фото профілю: вибір джерела, дозвіл камери, камера емулятора, профіль з фото

На рисунку 4.6 першим показано екран профілю з відкритою нижньою панеллю «Фото профілю» і двома діями – «Зробити фото» та «Обрати з галереї». Другий екран – системний запит дозволу «Allow Smart Life to take pictures and record video?». Третій – застосунок камери емулятора, що показує віртуальну сцену, і кнопка затвора внизу. Четвертий – профіль після збереження: у круглому аватарі відображається зроблений знімок, а знизу – повідомлення «Фото профілю оновлено».

4.6 Сенсори

Android підтримує три групи сенсорів: **руху** (акселерометр – прискорення по осях x , y , z у м/с^2 , разом із силою тяжіння; гіроскоп – кутова швидкість у рад/с), **положення** (магнітометр, датчик наближення) і **середовища** (освітленість у люксах, тиск, температура). Вісь x спрямована вправо, y – вгору вздовж екрана, z – з екрана до користувача; телефон, що лежить на столі, показує $z \approx 9,81 \text{ м/с}^2$.

Доступ до сенсорів дає `SensorManager`: `getDefaultSensor(type)` повертає сенсор або `null`, якщо його немає на пристрої. Слухач `SensorEventListener` отримує події `onSensorChanged` з частотою, заданою параметром (`SENSOR_DELAY_UI` – близько 15 разів за секунду). Сенсори витрачають батарею, тому від них **обов'язково відписуються**, коли екран не видно. Дозволи для базових сенсорів не потрібні (крім датчиків тіла – `BODY_SENSORS`).

За значеннями акселерометра обчислюють кути нахилу: $\text{pitch} = \text{atan2}(y, z)$, $\text{roll} = \text{atan2}(-x, \sqrt{y^2 + z^2})$; різкий стрибок модуля прискорення відносно g означає струшування.

Сенсори, з якими працює Smart Life, їхні групи та одиниці вимірювання наведено в таблиці 4.7.

Таблиця 4.7 – Сенсори, що використовуються в Smart Life

Сенсор (Sensor.TYPE...)	Група	Значення	Одиниці
ACCELEROMETER	руху	прискорення по осях x , y , z , включно з g	м/с^2
GYROSCOPE	руху	швидкість обертання навколо осей	рад/с
LIGHT	середовища	освітленість	лк
PROXIMITY	положення	відстань до об'єкта або «близько/далеко»	см
STEP_COUNTER (завдання)	руху	кроки з моменту перезавантаження	кроки

Практичний приклад. Smart Life: екран «Сенсори» (вкладка «Пристрій») показує значення акселерометра, кути нахилу, лічильник струшувань, гіроскоп, освітленість з текстовою оцінкою і стан датчика наближення, а також інформацію про кожен сенсор.

Спочатку створіть клас `sensors/SensorReader.kt` – обгортку над `SensorManager`, яка повертає дані будь-якого сенсора як `Flow` і вміє повідомляти інформацію про сенсор (назву, виробника, діапазон). Потім створіть `SensorsViewModel` з окремим `StateFlow` для кожного сенсора; обчислення нахилу і підрахунок струшувань винесіть у чисту функцію `computeMotion`, щоб її можна було протестувати. Наостанок створіть екран `SensorsScreen` з картками сенсорів і додайте пункт «Сенсори» на екран «Пристрій».

Ключові фрагменти реалізації наведено нижче.

```
// sensors/SensorReader.kt - сенсор як Flow
fun readings(type: Int, delay: Int = SensorManager.SENSOR_DELAY_UI): Flow<FloatArray> {
    val sensor = sensorManager.getDefaultSensor(type) ?: return emptyFlow()
    return callbackFlow {
        val listener = object : SensorEventListener {
            override fun onSensorChanged(e: SensorEvent) { trySend(e.values.clone()) }
            override fun onAccuracyChanged(s: Sensor, accuracy: Int) = Unit
        }
        sensorManager.registerListener(listener, sensor, delay)
        awaitClose { sensorManager.unregisterListener(listener) } // економія батареї
    }
}

// ui/sensors/SensorsViewModel.kt
val motion: StateFlow<MotionState> = sensorReader.readings(Sensor.TYPE_ACCELEROMETER)
    .runningFold(MotionState()) { prev, v -> computeMotion(prev, v[0], v[1], v[2]) }
    .stateIn(viewModelScope, SharingStarted.WhileSubscribed(1_000), MotionState())
```

```

fun computeMotion(previous: MotionState, x: Float, y: Float, z: Float): MotionState {
    val magnitude = sqrt(x * x + y * y + z * z)
    val pitch = Math.toDegrees(atan2(y.toDouble(), z.toDouble())).toFloat()
    val isShake = abs(magnitude - GRAVITY) > SHAKE_THRESHOLD &&
        abs(previous.magnitude - GRAVITY) <= SHAKE_THRESHOLD
    // ... roll; повертає MotionState(x, y, z, magnitude, pitch, roll, shakes + 0/1)
}
// ... light: map { it[0] }, isNear: map { it[0] < maxRange }

```

Функція `readings` спочатку просить у системи сенсор потрібного типу. Якщо на пристрої його немає, `getDefaultSensor` повертає `null`, і функція повертає порожній потік – екран просто покаже, що сенсор недоступний. Інакше в `callbackFlow` створюється слухач `SensorEventListener`: метод `onSensorChanged` отримує кожне нове вимірювання і надсилає в потік копію масиву значень. Копія потрібна тому, що система перевикористовує той самий масив для наступних подій. Реєстрація слухача відбувається, коли потік починають збирати, а відписка – в `awaitClose`.

У `ViewModel` потік акселерометра проходить через оператор `runningFold`: він передає в `computeMotion` не лише нові значення `x`, `y`, `z`, а й попередній стан, що дає змогу рахувати струшування. Результат перетворюється на `StateFlow` з параметром `WhileSubscribed(1_000)` – через секунду після того, як екран зник, сенсор відключається.

Функція `computeMotion` обчислює модуль вектора прискорення як корінь із суми квадратів трьох складових; для нерухомого телефона він приблизно дорівнює $9,8 \text{ м/с}^2$. Кут нахилу вперед-назад (`pitch`) дає арктангенс відношення `y` до `z`. Струшування зараховується, коли модуль відхиляється від `g` більше ніж на 6 м/с^2 , але лише в момент переходу через поріг – тож один різкий рух дає одне струшування, а не десяток.

Запуск на емуляторі. Відкрийте *Extended Controls* ▶ *Virtual sensors*: на вкладці *Device Pose* повертайте модель телефона – змінюються акселерометр, гіроскоп і кути нахилу; на вкладці *Additional sensors* змініть *Light* і *Proximity*. Для перевірки струшування використайте телефон.

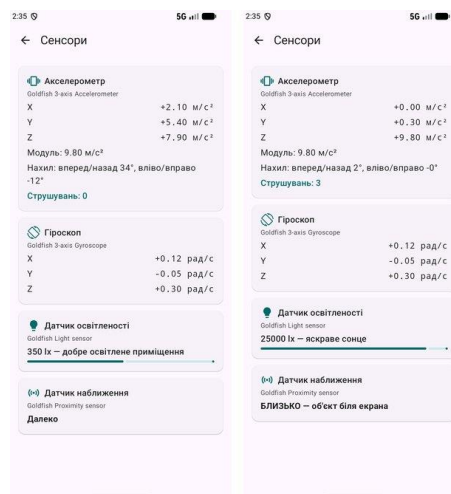


Рисунок 4.7 – Екран «Сенсори»: пристрій лежить рівно (ліворуч) і після зміни положення та освітленості у віртуальних сенсорах (праворуч)

На рисунку 4.7 кожен сенсор показано окремою карткою з назвою сенсора емулятора (Goldfish 3-axis Accelerometer тощо). Ліворуч пристрій нахилено: акселерометр показує складові $+2.10$, $+5.40$ і $+7.90 \text{ м/с}^2$, модуль 9.80 м/с^2 і нахил 34° вперед-назад та -12° вліво-вправо; освітленість 350 лк оцінено як «добре освітлене приміщення», датчик наближення показує «Далеко». Праворуч пристрій лежить рівно ($z = 9.80 \text{ м/с}^2$, нахил близький до нуля), лічильник показує три струшування, освітленість 25000 лк відповідає яскравому сонцю, а датчик наближення – «БЛИЗЬКО – об'єкт біля екрана». Значення задано у віртуальних сенсорах емулятора.

4.7 Типові помилки

Bluetooth LE. Виняток *SecurityException: Need BLUETOOTH_SCAN permission* означає, що дозвіл оголошено в маніфесті, але не запитано в користувача під час роботи. Якщо на Android 11 і старіших сканування нічого не знаходить, імовірно, на телефоні вимкнено геолокацію – на цих версіях вона потрібна для BLE.

Функція зворотного виклику `onScanFailed` з кодом 6 (*SCANNING_TOO_FREQUENTLY*) повідомляє, що сканування запускали понад 5 разів за 30 секунд. Якщо підключення вдалося, але пульс не надходить, перевірте, що записано дескриптор CCCD і що підписка виконується в `onServicesDiscovered`, а не раніше. І не забувайте після роботи викликати `gatt.disconnect()` та `gatt.close()`, інакше з'єднання залишаться відкритими.

Геолокація. На щойно створеному емуляторі `lastLocation` часто повертає `null`, бо жодна програма ще не запитувала координати: використайте `getCurrentLocation()` або задайте точку в *Extended Controls* ► *Location*. Якщо координати не змінюються в режимі відстеження, на емуляторі не встановлено нову точку або не викликано `requestLocationUpdates`. Швидкий розряд батареї свідчить, що застосунок не відписується від оновлень через `removeLocationUpdates`. Пам'ятайте також, що користувач може обрати лише приблизну точність, і застосунок повинен коректно працювати з дозволом *COARSE*.

Карти. Порожня сіра карта *osmdroid* найчастіше означає, що немає інтернету або не задано `userAgentValue` – тоді сервер відхиляє запити плиток. Якщо карта перестає реагувати після повернення на екран, не викликано `onResume()`. Маркери, що дублюються при кожному оновленні, свідчать, що перед додаванням нових шарів не очищено `overlays`.

Для Google Maps головний симптом проблем – сірий екран і повідомлення *Authorization failure* у Logcat. Причини описано в підрозділі 4.4: відсутній або неправильно обмежений ключ, не ввімкнено *Maps SDK for Android*, не збігаються назва пакета чи SHA-1 відбиток, або емулятор використовує образ без Google APIs.

Камера. Виняток *Failed to find configured root* означає, що папку файлу не описано в `file_paths.xml` або `authority` в коді не збігається з маніфестом. Якщо камера повертає `false`, їй не передано `Uri` або не створено папку для файлу. *SecurityException* при повторному читанні зображення з галереї виникає через те, що доступ до `Uri` тимчасовий, – тому файл потрібно копіювати одразу. А якщо після зміни фото на екрані залишається старе, *Coil* показує його з кешу: використовуйте нове ім'я файлу при кожному збереженні.

Сенсори. На пристроях без певного сенсора (наприклад, гіроскопа) застосунок падає з *NullPointerException*, якщо не перевірити результат `getDefaultSensor()`. Швидкий розряд батареї означає, що слухача не відписано. Надто висока частота опитування (*SENSOR_DELAY_FASTEST*) перевантажує інтерфейс без жодної користі для відображення. А якщо значення «стрибають», застосуйте згладжування – фільтр нижніх частот.

4.8 Практичні завдання

Практичні завдання розділу наведено в таблиці 4.8. Основне завдання виконується на основі версії Smart Life, отриманої в підрозділі, а завдання для самостійної роботи поглиблює тему і може оцінюватися додатково.

Таблиця 4.8 – Практичні завдання розділу

Тема	Практичне завдання	Завдання для самостійної роботи
Bluetooth LE	Додайте читання рівня заряду (сервіс Battery 0x180F, характеристика 0x2A19) і покажіть його на картці підключення.	Сортуйте знайдені пристрої за RSSI і покажіть орієнтовну відстань (близько / середньо / далеко); додайте фільтр сканування лише за сервісом Heart Rate (ScanFilter).
Геолокація	Покажіть на екрані геолокації відстань від точки, де було зроблено перше вимірювання, до	Визначайте назву населеного пункту за координатами (<code>Geocoder.getFromLocation()</code>) і

Тема	Практичне завдання	Завдання для самостійної роботи
	поточної позиції (<code>Location.distanceBetween</code>).	покажіть її у картці погоди замість «Ваше місце».
Карти	З'єднайте маркери записів лінією в хронологічному порядку (<code>Polyline</code>) і покажіть загальну довжину маршруту.	Реалізуйте той самий екран на Google Maps (<code>maps-compose</code>) з власним API-ключем і порівняйте обидва підходи.
Камера	Додайте до запису журналу можливість прикріпити фото (наприклад, тарілки з їжею) і покажіть мініатюру на екрані деталей.	Створіть власний екран камери на <code>CameraX</code> з попереднім переглядом і кнопкою зйомки.
Сенсори	Додайте крокомір на основі сенсора <code>TYPE_STEP_COUNTER</code> (потрібен дозвіл <code>ACTIVITY_RECOGNITION</code>) і збереження кількості кроків у журнал.	Реалізуйте «рівень» (бульбашковий рівень): кружок, що рухається по екрану відповідно до нахилу пристрою.

4.9 Контрольні питання

1. Чим BLE відрізняється від Bluetooth Classic?
2. Назвіть етапи взаємодії з BLE-пристроєм.
3. Що таке сервіс, характеристика і дескриптор GATT?
4. Навіщо записувати дескриптор CCCD?
5. Які дозволи потрібні для BLE на Android 12+?
6. Чим відрізняються дозволи FINE і COARSE?
7. Як Fused Location Provider поєднує джерела координат?
8. Чим `lastLocation` відрізняється від `getCurrentLocation()`?
9. Чому потрібно відписуватися від оновлень?
10. Як задати координати на емуляторі?
11. Що таке плитки карти?
12. Як вбудувати класичний View у Compose?
13. Навіщо синхронізувати карту з життєвим циклом?
14. Як отримати й захистити API-ключ Google Maps?
15. Назвіть причини сірого екрана замість Google-карти.
16. Що таке Activity Result API і контракти?
17. Навіщо потрібен FileProvider?
18. Чому для Photo Picker не потрібен дозвіл на галерею?
19. Чому фото копіюють у внутрішню пам'ять?
20. Чим CameraX відрізняється від виклику системної камери?
21. Які групи сенсорів підтримує Android?
22. Що показує акселерометр телефона, який лежить на столі?
23. Чому потрібно відписуватися від сенсорів?
24. Як обчислити нахил пристрою?
25. Як перевірити сенсори на емуляторі?

РОЗДІЛ 5. СПОВІЩЕННЯ, ФОНОВІ ЗАДАЧІ, АВТОРИЗАЦІЯ ТА БЕЗПЕКА

5.1 Мета розділу та результати навчання

Мета розділу – навчити взаємодіяти з користувачем поза інтерфейсом застосунку за допомогою сповіщень і фонових задач, реалізувати автентифікацію користувача через REST API та захистити дані й збірку застосунку. Після розділу Smart Life нагадує пити воду навіть закритим, працює лише для авторизованого користувача, зберігає токени зашифрованими і збирається як підписаний реліз. Результати навчання наведено в таблиці 5.1, основні поняття – в таблиці 5.2.

Таблиця 5.1 – Результати навчання за темами розділу

Тема	Що студент знає і вміє після опрацювання
Сповіщення	Будову сповіщення (іконка, заголовок, текст, дії); канали і рівні важливості (Android 8+); дозвіл на сповіщення (Android 13+); NotificationCompat.Builder; PendingIntent і прапорець FLAG_IMMUTABLE; обробку дії через BroadcastReceiver.
Фонові задачі	Способи фонової роботи (корутини, WorkManager, Foreground Service, AlarmManager) і вибір між ними; обмеження Doze і App Standby; класи CoroutineWorker, PeriodicWorkRequest, OneTimeWorkRequest, Constraints; унікальні задачі; стани задачі; результати success, retry, failure; перевірку задач через Logcat і App Inspection.
Авторизація	Різницю між автентифікацією і авторизацією; токених схему (access і refresh token, JWT); заголовок Authorization: Bearer; валідацію форм; зберігання сесії; вибір стартового екрана за станом сесії; вихід з акаунта; обробку помилки 401.
Безпека	Основні ризики OWASP Mobile Top 10; пісочницю застосунку; Android Keystore і шифрування AES-GCM; network_security_config; правила зберігання секретів (local.properties, BuildConfig); мінімізацію дозволів і експорту компонентів; R8; підпис APK і захист keystore; що не можна писати в логи.

Таблиця 5.2 – Основні поняття розділу

Поняття	Визначення
Канал сповіщень	категорія сповіщень з власною важливістю та налаштуваннями
heads-up	спливаюче сповіщення поверх інших застосунків (для каналів високої важливості, якщо це дозволяють налаштування)
PendingIntent	намір, який система виконає пізніше від імені застосунку
BroadcastReceiver	компонент, що обробляє широкомовні повідомлення
goAsync()	продовження роботи ресивера у фоні до ~10 с
Doze	енергозберігаючий режим, що відкладає фонову роботу
Worker / CoroutineWorker	клас з методом doWork(), що виконує задачу
WorkRequest	запит на виконання задачі (одноразовий або періодичний)
Constraints	умови запуску задачі (мережа, батарея, зарядка)
унікальна задача	задача з ім'ям, що існує лише в одному екземплярі
Автентифікація	підтвердження особи користувача
access token	короткоживучий ключ доступу до API
refresh token	довгоживучий ключ для отримання нового access token
JWT	JSON Web Token – підписаний токен з даними користувача
сесія	збережений стан входу користувача
Пісочниця	ізоляція застосунку та його даних від інших застосунків
Android Keystore	системне сховище криптографічних ключів
AES-GCM	симетричне шифрування з перевіркою цілісності
R8	інструмент зменшення, оптимізації та обфускації коду
keystore (.jks)	файл із ключем підпису релізних збірок

5.2 Сповіщення

Сповіщення – повідомлення, яке застосунок показує поза своїм інтерфейсом: у рядку стану, шторці і на екрані блокування. З Android 8 кожне сповіщення належить **каналу** – категорії, яку користувач може окремо вимкнути або налаштувати. Важливість

каналу визначає поведінку: `IMPORTANCE_DEFAULT` – звук і значок, `IMPORTANCE_HIGH` – ще й може показувати спливаюче сповіщення (heads-up), якщо це дозволяють системні налаштування і налаштування каналу. З Android 13 для показу сповіщень потрібен небезпечний дозвіл `POST_NOTIFICATIONS`.

Дія при натисканні на сповіщення або його кнопку описується **PendingIntent** – «відкладеним наміром», який система виконає від імені застосунку: відкрити Activity (`getActivity`) або надіслати широкомовне повідомлення (`getBroadcast`) до `BroadcastReceiver`, що виконає дію без відкриття застосунку (рис. 5.1). З Android 12 для `PendingIntent` обов’язково вказують `FLAG_IMMUTABLE` або `FLAG_MUTABLE`.

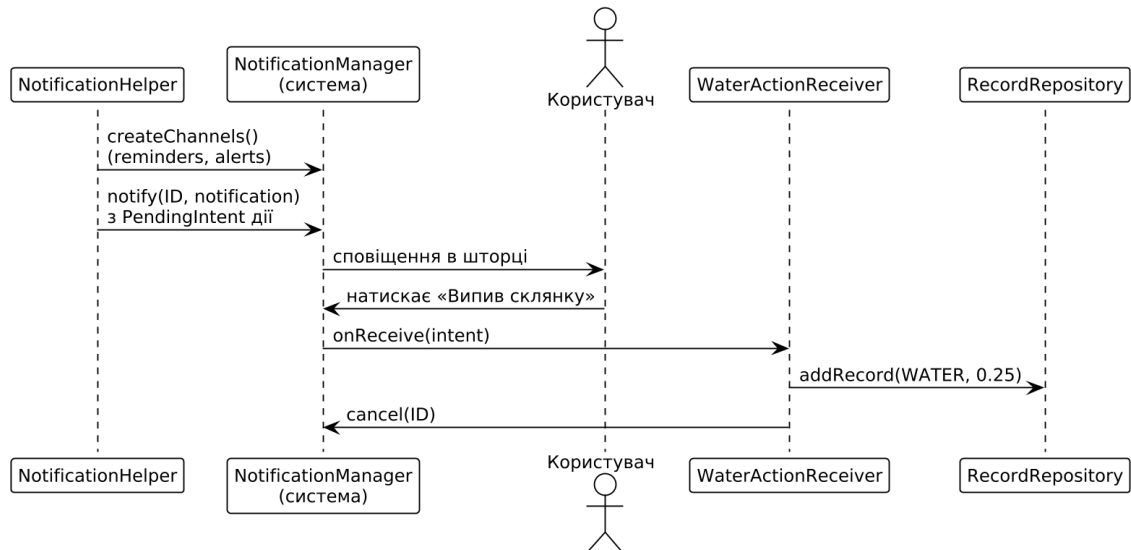


Рисунок 5.1 – Сповіщення з дією «Випив склянку»

На рисунку 5.1 показано шлях натискання кнопки у сповіщенні. `NotificationHelper` створює канали і передає сповіщення з `PendingIntent` системному `NotificationManager`, який показує його в шторці. Коли користувач натискає «Випив склянку», система доставляє намір до `WaterActionReceiver`; ресивер додає запис води в репозиторій і прибирає сповіщення. Застосунок при цьому не відкривається.

Важливість каналу визначає, наскільки помітним буде сповіщення (табл. 5.3). Обирати високу важливість слід лише для справді термінової інформації: якщо застосунок зловживає спливаючими сповіщеннями, користувач вимкне канал або всі сповіщення застосунку.

Таблиця 5.3 – Рівні важливості каналів сповіщень

Рівень	Поведінка	Приклад
<code>IMPORTANCE_HIGH</code>	звук; може спричиняти heads-up-сповіщення, якщо це дозволено системними налаштуваннями і налаштуваннями каналу	попередження про високий пульс
<code>IMPORTANCE_DEFAULT</code>	звук, значок у рядку стану, шторка	нагадування пити воду
<code>IMPORTANCE_LOW</code>	без звуку, лише в шторці	хід синхронізації
<code>IMPORTANCE_MIN</code>	без звуку, згорнуте в шторці	службова інформація

Крім тексту і кнопок дій, сповіщення можуть мати розгорнутий вигляд: `BigTextStyle` показує довгий текст, `BigPictureStyle` – зображення, `InboxStyle` – кілька рядків, а `setProgress` – смугу прогресу. Сповіщення одного застосунку можна групувати, а натискання на сповіщення зазвичай відкриває потрібний екран застосунку через `PendingIntent.getActivity`.

Практичний приклад. Smart Life: два канали – «Нагадування» і «Попередження про здоров’я»; тестове нагадування з кнопкою «Випив склянку», що додає 0,25 л води без

відкриття застосунку; спливаюче попередження, якщо записано пульс понад 100 уд/хв; дозвіл на сповіщення запитується в налаштуваннях.

Спершу внесіть зміни в маніфест: оголосіть дозвіл `POST_NOTIFICATIONS` і ресивер `<receiver android:name=".notifications.WaterActionReceiver" android:exported="false"/>`. Потім створіть векторну іконку `ic_notification` – одноколірну на прозорому фоні, бо система сама фарбує малу іконку сповіщення.

Далі створіть клас `notifications/NotificationHelper.kt` з методами створення каналів і показу сповіщень та викличте `createChannels()` у методі `onCreate()` класу `SmartLifeApplication`, щоб канали існували ще до першого сповіщення. Після цього створіть `WaterActionReceiver` для кнопки дії, а на екран налаштувань додайте картку запиту дозволу і кнопку «Надіслати тестове нагадування». Наостанок у `RecordsViewModel.addRecord()` викликайте попередження, якщо записаний пульс перевищує 100 уд/хв.

Ключові фрагменти реалізації наведено нижче.

```
// notifications/NotificationHelper.kt
fun createChannels() {
    val system = context.getSystemService(NotificationManager::class.java)
    system.createNotificationChannel(NotificationChannel(CHANNEL_REMINDERS,
        "Нагадування", NotificationManager.IMPORTANCE_DEFAULT))
    system.createNotificationChannel(NotificationChannel(CHANNEL_ALERTS,
        "Попередження про здоров'я", NotificationManager.IMPORTANCE_HIGH))
}

fun showWaterReminder(text: String) {
    val drinkIntent = Intent(context, WaterActionReceiver::class.java)
        .setAction(WaterActionReceiver.ACTION_DRINK_WATER)
    val drink = PendingIntent.getBroadcast(context, REQUEST_DRINK, drinkIntent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
    val notification = NotificationCompat.Builder(context, CHANNEL_REMINDERS)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Smart Life").setContentText(text)
        .setContentIntent(openAppIntent()) // натискання відкриває застосунок
        .setAutoCancel(true)
        .addAction(R.drawable.ic_notification, "Випив склянку", drink)
        .build()
    if (canPostNotifications()) manager.notify(ID_WATER_REMINDER, notification)
}
// ... showHighPulseAlert(bpm): канал ALERTS, BigTextStyle, PRIORITY_HIGH

// notifications/WaterActionReceiver.kt
override fun onReceive(context: Context, intent: Intent) {
    val pending = goAsync()
    CoroutineScope(SupervisorJob() + Dispatchers.IO).launch {
        try {
            container.recordRepository.addRecord(
                HealthRecord(type = RecordType.WATER, value = 0.25, note = "Зі сповіщення"))
            container.notificationHelper.cancelWaterReminder()
        } finally { pending.finish() }
    }
}
```

Метод `createChannels()` створює два канали з різною важливістю. Канал «Нагадування» має важливість `IMPORTANCE_DEFAULT` – сповіщення з'являється в шторці зі звуком. Канал «Попередження про здоров'я» має `IMPORTANCE_HIGH`, тому його сповіщення спливає поверх екрана. Повторне створення каналу безпечно: система просто ігнорує дублікат.

Для кнопки «Випив склянку» спочатку створюється `Intent`, адресований нашому `WaterActionReceiver`, з власною дією `ACTION_DRINK_WATER`. `PendingIntent.getBroadcast` загортає його у «відкладений намір», який система зможе виконати від імені застосунку, навіть коли той закритий; прапорець `FLAG_IMMUTABLE` забороняє іншим застосункам змінювати намір. Далі `NotificationCompat.Builder` збирає сповіщення: малу іконку, заголовок, текст, дію при натисканні на саме сповіщення (відкрити застосунок), автоматичне закриття і кнопку дії. Перед показом `canPostNotifications()` перевіряє, що є дозвіл і користувач не вимкнув сповіщення.

Коли користувач натискає кнопку, система викликає `onReceive` ресивера. Цей метод має завершитися швидко, тому `goAsync()` дозволяє продовжити роботу у фоні (до 10 секунд): у корутині з `Dispatchers.IO` в базу додається запис 0.25 л води, сповіщення прибирається, а `pending.finish()` у блоці `finally` повідомляє системі, що робота завершена.

Запуск на емуляторі. У «Налаштуваннях» Smart Life натисніть «Надати дозвіл» і дозвольте сповіщення, потім «Надіслати тестове нагадування». Відкрийте шторку (протягніть від верхнього краю) і натисніть «Випив склянку» – у журналі з’явиться запис «Зі сповіщення». Додайте запис пульсу 118 – з’явиться спливаюче попередження.

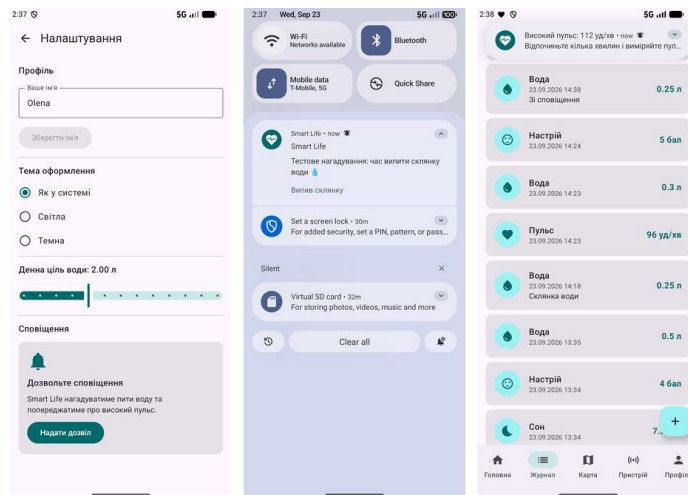


Рисунок 5.2 – Сповіщення: запит дозволу в налаштуваннях, нагадування з дією в шторці, спливаюче попередження про пульс

На рисунку 5.2 ліворуч – екран налаштувань із карткою «Дозвольте сповіщення», яка пояснює, що застосунок нагадуватиме пити воду і попереджатиме про високий пульс, і має кнопку «Надати дозвіл». Посередині – відкрита шторка сповіщень Android: сповіщення Smart Life з текстом «Тестове нагадування: час випити склянку води» і кнопкою дії «Випив склянку». Праворуч – спливаюче попередження «Високий пульс: 112 уд/хв» над журналом; у журналі також видно запис «Вода 0.25 л – Зі сповіщення», доданий натисканням кнопки в шторці.

5.3 Фонові задачі

Корутини у `viewModelScope` зупиняються разом з екраном, тому підходять лише для роботи, пов’язаної з інтерфейсом. Для зберігання батареї Android обмежує фонову роботу: у режимі **Doze** (екран вимкнено, пристрій нерухомий) мережа і таймери відкладаються, а застосунки в групах **App Standby** запускаються рідше.

WorkManager – рекомендований інструмент для **відкладеної надійної** роботи: WorkManager зберігає заплановану задачу і намагається виконати її після закриття застосунку або перезавантаження пристрою з урахуванням системних обмежень і умов (Constraints: мережа, заряд батареї, зарядка). Періодичні задачі мають мінімальний інтервал 15 хвилин, а точний час запуску визначає система. Для негайної тривалої роботи, яку бачить користувач (музика, навігація), використовують **Foreground Service**, для точного будильника – **AlarmManager**.

Задача проходить стани `ENQUEUED` → `RUNNING` → `SUCCEEDED/FAILED` (для періодичної – повернення в `ENQUEUED`), її можна скасувати (`CANCELLED`) (рис. 5.3). Результат `Result.retry()` просить повторити спробу з експоненційною затримкою.

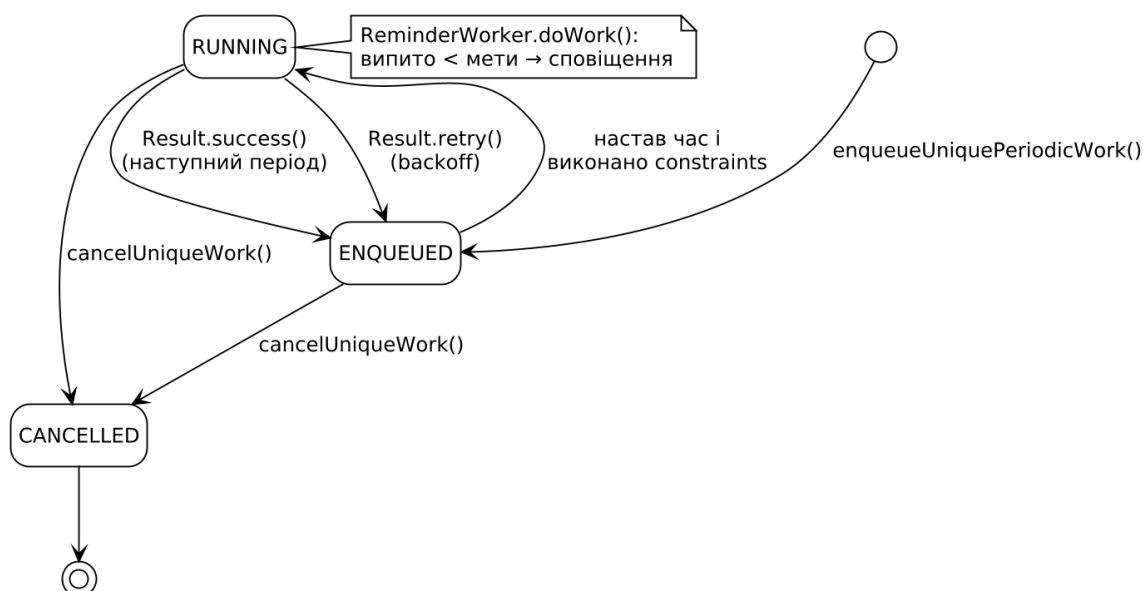


Рисунок 5.3 – Стани періодичної задачі WorkManager

Діаграма станів на рисунку 5.3 показує життя періодичної задачі. Після реєстрації задача переходить у стан **ENQUEUED** і чекає, поки настане час і виконаються обмеження. Тоді вона переходить у **RUNNING** і виконує `doWork()`. І результат `success()`, і `retry()` повертають періодичну задачу в **ENQUEUED** – до наступного періоду або до повторної спроби. З будь-якого стану задачу можна скасувати, після чого вона опиняється в стані **CANCELLED**. Примітка праворуч нагадує, що саме робить наш воркер.

Вибір механізму для фонової роботи залежить від того, коли і як довго вона має виконуватися. Порядок прийняття рішення показано на рисунку 5.4, а порівняння механізмів – у таблиці 5.4.

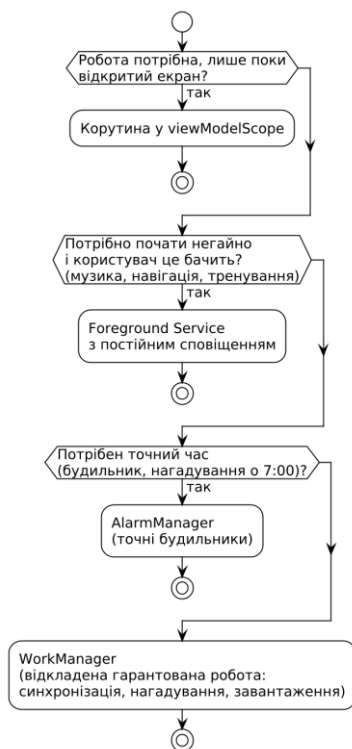


Рисунок 5.4 – Вибір механізму фонової роботи

Алгоритм на рисунку 5.4 ставить три запитання по черзі. Якщо робота потрібна лише поки відкритий екран, достатньо корутини у `viewModelScope`. Якщо робота має

початися негайно і триває довго на очах у користувача, використовують `Foreground Service` з постійним сповіщенням. Якщо дія має відбутися в точно заданий час, застосовують `AlarmManager`. В усіх інших випадках – для відкладеної роботи, яка має надійно виконатися навіть після закриття застосунку – правильним вибором є `WorkManager`.

Таблиця 5.4 – Механізми фонової роботи в Android

Механізм	Коли виконується	Переживає закриття застосунку	Приклад
Корутина у <code>viewModelScope</code>	одразу, поки існує екран	ні	завантаження погоди
<code>WorkManager</code>	відкладено, з урахуванням умов	так, переживає перезавантаження (з урахуванням обмежень системи)	нагадування про воду, синхронізація
<code>Foreground Service</code>	одразу і безперервно	так, поки видно сповіщення	запис тренування, плеєр
<code>AlarmManager</code>	у точний час	так	будильник, прийом ліків о 8:00

Практичний приклад. `Smart Life`: у налаштуваннях можна увімкнути нагадування з інтервалом 15, 30, 60 або 120 хвилин. Фонова задача перевіряє, скільки води випито сьогодні, і, якщо менше за норму, показує нагадування з кнопкою «Випив склянку». Налаштування показує стан задачі і час наступного запуску, кнопка «Перевірити зараз» запускає задачу одразу.

Спочатку підключіть бібліотеку `work-runtime-ktx`. Потім створіть клас `work/ReminderWorker.kt`, що успадковує `CoroutineWorker` і в методі `doWork()` перевіряє випиту за день воду. Далі створіть `work/ReminderScheduler.kt`: він планує періодичну задачу, скасовує її, запускає разову перевірку і повідомляє поточний стан задачі. Наостанок збережіть параметри нагадувань (увімкнено чи ні, інтервал) у `DataStore` і додайте на екран налаштувань блок «Нагадування про воду» з перемикачем, вибором інтервалу, станом задачі та кнопкою «Перевірити зараз».

Ключові фрагменти реалізації наведено нижче.

```
// work/ReminderWorker.kt
class ReminderWorker(ctx: Context, params: WorkerParameters) : CoroutineWorker(ctx, params) {
    override suspend fun doWork(): Result {
        val container = (applicationContext as SmartLifeApplication).container
        return try {
            val drunk = container.recordRepository.observeRecords().first()
                .toDailySummary().waterLiters
            val goal = container.settingsRepository.settings.first().waterGoalLiters
            if (drunk < goal) container.notificationHelper.showWaterReminder(
                "Сьогодні випито %.2f з %.2f л. Час випити склянку води".format(drunk, goal))
            Result.success()
        } catch (e: Exception) {
            if (runAttemptCount < 3) Result.retry() else Result.failure()
        }
    }
}

// work/ReminderScheduler.kt
fun schedule(intervalMinutes: Long) {
    val request = PeriodicWorkRequestBuilder<ReminderWorker>(intervalMinutes, TimeUnit.MINUTES)
        .setConstraints(Constraints.Builder().setRequiresBatteryNotLow(true).build())
        .build()
    workManager.enqueueUniquePeriodicWork("water_reminder",
        ExistingPeriodicWorkPolicy.UPDATE, request)
}

fun observeStatus(): Flow<ReminderStatus> =
    workManager.getWorkInfosForUniqueWorkFlow("water_reminder").map { infos ->
        ReminderStatus(infos.firstOrNull()?.state, infos.firstOrNull()?.nextScheduleTimeMillis)
    }

// ... cancel(): cancelUniqueWork; runOnceNow(): enqueue(OneTimeWorkRequest)
```

`ReminderWorker` успадковує `CoroutineWorker`, тому метод `doWork()` є `suspend-` функцією і виконується у фоновому потоці. Воркер отримує контейнер залежностей через контекст застосунку і читає два значення: список записів з бази, з якого функція

`toDailySummary()` обчислює випиту сьогодні воду, і денну норму з налаштувань. Оператор `first()` бере з кожного `Flow` лише поточне значення, не чекаючи наступних. Якщо випито менше за норму, показується нагадування з конкретними цифрами. Метод повертає `Result.success()`, а при помилці – `Result.retry()`, і `WorkManager` повторить спробу пізніше зі зростаючою затримкою; після трьох невдалих спроб повертається `Result.failure()`.

`ReminderScheduler.schedule()` створює періодичний запит `PeriodicWorkRequestBuilder` з інтервалом, обраним у налаштуваннях, і обмеженням `setRequiresBatteryNotLow(true)` – при низькому заряді нагадування не турбуватимуть. Метод `enqueueUniquePeriodicWork` реєструє задачу під ім'ям `water_reminder`: якщо задача з таким ім'ям уже існує, політика `UPDATE` лише оновлює її параметри, тож повторне ввімкнення не створює дублікатів. Стан задачі екран отримує з `getWorkInfosForUniqueWorkFlow` – потоку, який видає новий `WorkInfo` щоразу, коли `WorkManager` змінює стан або планує наступний запуск.

Запуск на емуляторі. Увімкніть нагадування в налаштуваннях – стан зміниться на «заплановано» з часом наступного запуску. Натисніть «Перевірити зараз» і закрийте застосунок: через кілька секунд з'явиться сповіщення. Журнал роботи видно в *Logcat* з фільтром `tag:ReminderWorker`, а список задач – в *App Inspection* ▶ *Background Task Inspector*.

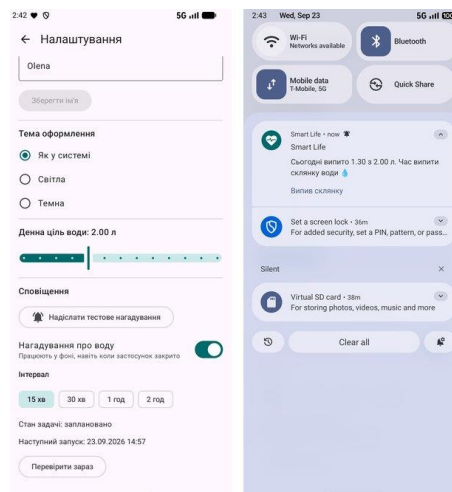


Рисунок 5.5 – Фонові нагадування: налаштування зі станом задачі `WorkManager` і сповіщення, створене фоновію задачею

На рисунку 5.5 ліворуч – екран налаштувань з блоком «Нагадування про воду»: перемикач увімкнено, обрано інтервал «15 хв», під ним виведено стан задачі «заплановано» і час наступного запуску, а також кнопку «Перевірити зараз». Праворуч – сповіщення, створене фоновію задачею після закриття застосунку: «Сьогодні випито 1.30 з 2.00 л. Час випити склянку води» з кнопкою «Випив склянку».

5.4 Авторизація та робота з користувачами

Автентифікація – перевірка, хто користувач (логін і пароль), **авторизація** – перевірка, що йому дозволено. Мобільні застосунки зазвичай використовують **токени**: після успішного входу сервер повертає **access token** (короткоживучий, передається в кожному запиті заголовком `Authorization: Bearer <token>`) і **refresh token** (довгоживучий, лише для отримання нового access token). Токени часто мають формат **JWT** – три частини в Base64 (заголовок, дані, підпис). Пароль у застосунку не зберігається ніколи.

Коли сервер відповідає **401 Unauthorized**, застосунок один раз оновлює токен (`POST /auth/refresh`) і повторює запит; якщо і refresh token недійсний – сесія завершується і користувач повертається на екран входу (рис. 5.6). Перед надсиланням

запиту форму перевіряють на клієнті (порожні поля, довжина, формат email) – це зменшує кількість зайвих запитів, але не замінює перевірку на сервері.

У курсі використано навчальний сервер **DummyJSON** (<https://dummyjson.com>): вхід `POST /auth/login` (тестовий користувач `emilys / emilyspass`), профіль `GET /auth/me`, оновлення токена `POST /auth/refresh`, реєстрація `POST /users/add` (імітація, користувач не зберігається).

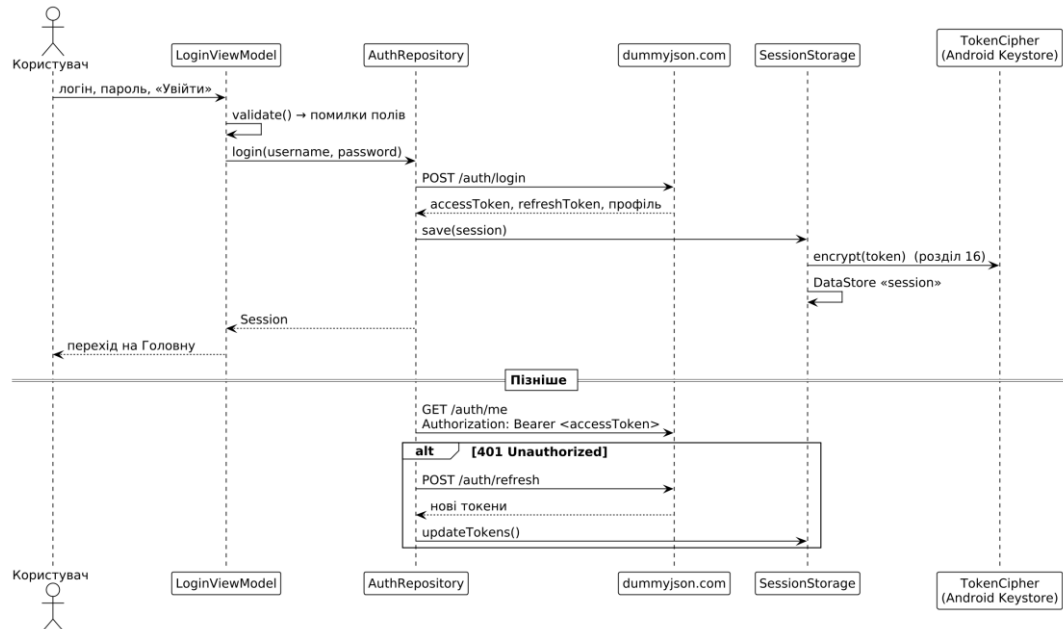


Рисунок 5.6 – Вхід користувача і збереження сесії

На рисунку 5.6 угорі показано вхід. Користувач вводить дані, ViewModel перевіряє поля і передає їх репозиторію, репозиторій надсилає `POST /auth/login` і отримує токени та профіль. Сесія зберігається в `SessionStorage`, яка шифрує токени через `TokenCipher` (це буде додано в підрозділі 5.5), і користувач потрапляє на головний екран. Нижня частина, відокремлена написом «Пізніше», показує запит профілю з токеном у заголовку і гілку alt для відповіді 401: репозиторій оновлює токени запитом `POST /auth/refresh` і зберігає нову пару.

Токен JWT складається з трьох частин, розділених крапками (табл. 5.5). Перші дві частини лише закодовані в Base64, а не зашифровані, тому будь-хто може прочитати їх вміст – у токені не можна зберігати секрети. Захист забезпечує третя частина, підпис: змінити дані, не знаючи секретного ключа сервера, неможливо.

Таблиця 5.5 – Структура токена JWT

Частина	Вміст	Приклад поля
Заголовок (header)	алгоритм підпису і тип токена	"alg": "HS256"
Дані (payload)	ідентифікатор користувача, ім'я, час видачі і закінчення дії	"id": 1, "exp": 1795000000
Підпис (signature)	підпис перших двох частин секретним ключем сервера	–

Практичний приклад. Smart Life: екран входу з валідацією полів, індикатором завантаження і повідомленням про неправильний пароль; екран реєстрації; сесія в `DataStore`; після перезапуску застосунків відкривається одразу на головному екрані; у профілі – дані з `GET /auth/me` і кнопка «Вийти з акаунта».

Спочатку додайте в `BuildConfig` адресу сервера авторизації `AUTH_BASE_URL` і створіть у пакеті `data/remote` інтерфейс `AuthApi` разом з DTO запитів і відповідей. Потім створіть сховище сесії `data/auth/SessionStorage.kt` на окремому `DataStore` з ім'ям

session і репозиторій AuthRepository, який виконує вхід, реєстрацію, запит профілю і вихід.

Далі створіть пари ViewModel і екранів для входу (LoginViewModel, LoginScreen) та реєстрації (RegisterViewModel, RegisterScreen). Після цього створіть SessionViewModel, який повідомляє, чи є збережена сесія, і в SmartLifeApp обирайте стартовий маршрут залежно від неї. Наостанок додайте в профіль кнопку виходу, яка очищає сесію і стек навігації.

Ключові фрагменти реалізації наведено нижче.

```
// data/remote/AuthApi.kt
interface AuthApi {
    @POST("auth/login") suspend fun login(@Body request: LoginRequest): LoginResponse
    @GET("auth/me") suspend fun me(@Header("Authorization") authorization: String): UserDto
    @POST("auth/refresh") suspend fun refresh(@Body request: RefreshRequest): RefreshResponse
    @POST("users/add") suspend fun register(@Body request: RegisterRequest): UserDto
}

// data/repository/AuthRepository.kt - повтор запиту після оновлення токена
suspend fun fetchProfile(): UserDto {
    val current = sessionStorage.session.first() ?: throw AuthException("Сесію не знайдено")
    return try {
        api.me("Bearer ${current.accessToken}")
    } catch (e: HttpException) {
        if (e.code() != 401) throw e
        val tokens = try { api.refresh(RefreshRequest(current.refreshToken)) }
            catch (refreshError: HttpException) {
                logout(); throw AuthException("Сесія завершилася. Увійдіть знову")
            }
        sessionStorage.updateTokens(tokens.accessToken, tokens.refreshToken)
        api.me("Bearer ${tokens.accessToken}")
    }
}

// ... login(): 400/401 -> AuthException("Неправильний логін або пароль"), save(session)

// ui/auth/LoginViewModel.kt - перевірка полів до запиту
fun validate(username: String, password: String): Pair<String?, String?> {
    val usernameError = when {
        username.isBlank() -> "Введіть логін"
        username.trim().length < 3 -> "Логін має містити щонайменше 3 символи"
        else -> null
    }
    val passwordError = if (password.length < 6)
        "Пароль має містити щонайменше 6 символів" else null
    return usernameError to passwordError
}
```

Інтерфейс AuthApi описує чотири запити. Вхід і оновлення токена – це POST з тілом у форматі JSON (@Body), а запит профілю GET /auth/me передає токен у заголовок, який задає параметр з анотацією @Header("Authorization").

Функція fetchProfile() показує повний цикл роботи з токенами. Спочатку з SessionStorage береться поточна сесія; якщо її немає, генерується помилка з повідомленням для користувача. Далі виконується запит профілю із заголовком Bearer <accessToken>. Якщо сервер відповів кодом 401, access token прострочений: функція надсилає refresh token на POST /auth/refresh, отримує нову пару токенів, зберігає її через updateTokens і повторює запит профілю. Якщо недійсним виявився і refresh token, сесія очищається і користувач має увійти знову. Будь-які інші коди помилок передаються далі без змін.

Перевірка полів форми винесена у функцію validate, яка повертає пару повідомлень – для логіна і для пароля, де null означає, що поле заповнено правильно. Логін не може бути порожнім і має містити щонайменше 3 символи, пароль – щонайменше 6 символів. ViewModel викликає validate до запиту: якщо є помилки, вони виводяться під полями, і запит на сервер не надсилається. Оскільки функція чиста і не залежить від Android, її перевіряють unit-тести в підрозділі 6.2.

Запуск на емуляторі. Спробуйте увійти з коротким паролем – з'являться помилки під полями; з неправильним паролем – повідомлення сервера. Увійдіть як emilys /

emilyspass, закрийте й знову відкрийте застосунок – відкриється головний екран. У профілі натисніть «Дані з сервера (GET /auth/me)».

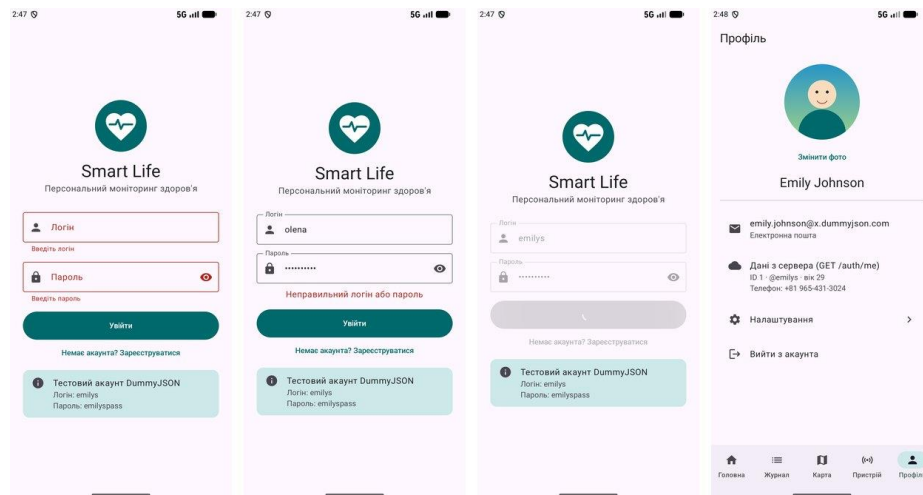


Рисунок 5.7 – Авторизація: помилки валідації, неправильний пароль, завантаження, профіль з даними сервера

На рисунку 5.7 перший екран показує помилки перевірки: поля підсвічено червоним, під ними виведено «Введіть логін» і «Введіть пароль»; внизу екрана підказка з тестовим обліковим записом DummyJSON. На другому екрані введено неіснуючого користувача, і під полями з'явилася відповідь сервера «Неправильний логін або пароль». Третій екран – стан завантаження: поля й кнопки неактивні, а на кнопці входу крутиться індикатор. Четвертий – профіль після входу як emilys: ім'я Emily Johnson, електронна пошта і рядок «Дані з сервера (GET /auth/me)» з ідентифікатором, віком і телефоном, отриманими із сервера.

5.5 Безпека мобільних застосунків

Кожен застосунок працює у **пісочниці**: власний Linux-користувач і приватна папка даних. Проте на пристрої з root-доступом, у резервних копіях або через debug-збірку дані можна прочитати, тому чутливі дані (токени, персональні дані) **шифрують**. **Android Keystore** генерує і зберігає криптографічні ключі так, що застосунок не отримує їх у вигляді звичайного експортованого ключового матеріалу: він лише просить систему виконати шифрування чи розшифрування. Ступінь апаратного захисту залежить від реалізації пристрою – наявності довіреного середовища виконання (TEE) або окремого захищеного чипа (StrongBox). Для шифрування використовують **AES-GCM** – алгоритм, що одночасно забезпечує конфіденційність і контроль цілісності; для кожного шифрування генерується новий вектор ініціалізації (IV).

Інші заходи: весь трафік – лише **HTTPS** (заборона cleartext у `network_security_config.xml`); **секрети не зберігають у git** – ключі API задаються в `local.properties` і передаються через `BuildConfig` (але будь-який ключ у APK можна витягти, тому справжні секрети мають бути на сервері); **мінімум дозволів** і `exported="false"` для внутрішніх компонентів; **без чутливих даних у логах** (`redactHeader("Authorization")`); **R8** зменшує і обфускує код релізу; релізний APK **підписується** ключем, який зберігають окремо від коду: втрата ключа означає неможливість оновити застосунок.

Систематизований перелік ризиків мобільних застосунків веде спільнота OWASP у списку Mobile Top 10 і стандарті перевірки MASVS. У таблиці 5.6 наведено основні ризики і те, як кожен з них закрито в Smart Life.

Таблиця 5.6 – Основні ризики безпеки мобільних застосунків і заходи захисту

Ризик (за OWASP Mobile Top 10)	Суть	Захід у Smart Life
Неналежне використання	секрети і ключі в коді або	ключі в <code>local.properties</code> і <code>BuildConfig</code> ,

Ризик (за OWASP Mobile Top 10)	Суть	Захід у Smart Life
облікових даних	репозиторії	keystore поза git
Небезпечне зберігання даних	відкриті токени і персональні дані на пристрої	шифрування токенів ключем Android Keystore, allowBackup="false"
Небезпечний зв'язок	передавання даних через HTTP	лише HTTPS, заборона cleartext у network_security_config
Недостатня автентифікація	помилки в роботі з сесією	токени з обмеженим терміном дії, оновлення через refresh token
Надмірні дозволи	доступ до даних, які не потрібні	мінімальний набір дозволів, запит у момент використання
Недостатній захист бінарного файлу	легкий аналіз коду APK	обфускація R8 у релізі
Витік через журнали	токени і персональні дані в Logcat	redactHeader("Authorization"), логуювання лише в debug

Практичний приклад. Smart Life: токени в DataStore шифруються ключем з Android Keystore; заборонено незашифрований трафік; ключ демо-API і адреси серверів передаються через BuildConfig; заголовок Authorization приховано в логах; реліз збирається з R8 і підписується ключем з файлу keystore.properties, що не потрапляє в git.

Спершу створіть клас data/security/TokenCipher.kt, який шифрує і розшифровує рядки ключем з Android Keystore, і використайте його в SessionStorage: токени тепер зберігаються під ключами access_token_encrypted і refresh_token_encrypted. Потім створіть файл res/xml/network_security_config.xml, який забороняє незашифрований трафік, підключіть його в маніфесті атрибутом android:networkSecurityConfig і вимкніть резервне копіювання даних (allowBackup="false"). У налаштуваннях логуювання OkHttp додайте redactHeader("Authorization"), щоб токени не потрапляли в Logcat.

Далі підготуйте релізну збірку. Створіть ключ підпису майстром Build ▸ Generate Signed App Bundle or APK ▸ Create new (рис. 5.8) або командою keytool -genkeypair, збережіть шлях і паролі у файлі keystore.properties і додайте цей файл та *.jks у .gitignore. Наостанок у app/build.gradle.kts для типу збірки release увімкніть isMinifyEnabled, isShrinkResources і підпис signingConfig.

Скріншот для виконання студентом – Рисунок 5.8. Створення ключа підпису в Android Studio

вікно New Key Store з полями Key store path, Password, Alias, Validity

Цей рисунок студент створює самостійно у своєму середовищі (Android Studio). Штучні зображення замість реального скриншота не використовуються.

Ключові фрагменти реалізації наведено нижче.

```
// data/security/TokenCipher.kt
private fun getOrCreateKey(): SecretKey {
    (keyStore.getEntry(keyAlias, null) as? KeyStore.SecretKeyEntry)
        ?.let { return it.secretKey }
    val spec = KeyGenParameterSpec.Builder(keyAlias,
        KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
        .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
        .setKeySize(256).build()
    return KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore")
        .apply { init(spec) }.generateKey()
}

fun encrypt(plainText: String): String {
    val cipher = Cipher.getInstance("AES/GCM/NoPadding")
    cipher.init(Cipher.ENCRYPT_MODE, getOrCreateKey()) // новий випадковий IV
    val encrypted = cipher.doFinal(plainText.toByteArray(Charsets.UTF_8))
    return Base64.getEncoder().encodeToString(cipher.iv + encrypted)
}

// ... decrypt(): відокремлює 12 байт IV, GCMParameterSpec(128, iv); null при помилці
// app/build.gradle.kts - реліз з R8 і підписом
```

```

buildTypes {
    release {
        isMinifyEnabled = true
        isShrinkResources = true
        proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro")
        if (keystoreProperties.isNotEmpty())
            signingConfig = signingConfigs.getByName("release")
    }
}
// res/xml/network_security_config.xml: <base-config cleartextTrafficPermitted="false">

```

Функція `getOrCreateKey()` спочатку шукає в Keystore ключ з нашим псевдонімом і, якщо він є, повертає його. Якщо ключа немає (перший запуск), `KeyGenParameterSpec` описує, яким має бути новий ключ: призначення – шифрування і розшифрування, режим GCM без доповнення, довжина 256 біт. `KeyGenerator` створює ключ безпосередньо всередині Android Keystore – застосунок отримує лише посилання на нього, а сам ключ не можна прочитати чи скопіювати з пристрою.

Функція `encrypt()` отримує екземпляр шифру AES/GCM/NoPadding і ініціалізує його ключем. Під час ініціалізації Keystore сам генерує випадковий вектор ініціалізації (IV) – застосунку заборонено задавати його вручну, щоб IV ніколи не повторювався. Після `doFinal` результатом є IV, до якого дописано шифротекст, закодовані в Base64 для зберігання в DataStore. Функція `decrypt()` робить зворотнє: відокремлює перші 12 байт як IV, ініціалізує шифр з параметром `GCMParameterSpec(128, iv)` і розшифровує решту. Режим GCM також перевіряє цілісність даних: якщо шифротекст змінено або ключ інший, розшифрування не вдасться, і функція поверне `null` – користувач просто увійде знову.

У фрагменті збірки `isMinifyEnabled` вмикає R8, який видаляє невикористаний код і замінює імена класів короткими, `isShrinkResources` видаляє невикористані ресурси. Параметри підпису читаються з `keystore.properties`; якщо файлу немає, наприклад у студента, який щойно клонував репозиторій, реліз збирається без підпису, і проект не ламається.

Запуск на емуляторі. Увійдіть у застосунок і перегляньте файл сесії командою `adb shell run-as ua.edu.mobile.smartlife cat files/datastore/session.preferences_pb` – токени зберігаються в зашифрованому вигляді. Зберіть реліз (`./gradlew assembleRelease`) і встановіть його: команда `run-as` для релізу не працює (*package not debuggable*), а розмір APK зменшився з 22,9 МБ (debug) до 2,5 МБ.

Найкраще результат шифрування видно у вмісті файлу сесії. Нижче наведено реальний фрагмент цього файлу до шифрування (підрозділ 5.4) і після (підрозділ 5.5): у першому рядку токен JWT читається як є, у другому – лише шифротекст у Base64, з якого без ключа з Keystore відновити токен неможливо.

```

access_token      eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUi...
access_token_encrypted R++h/+f+kPc7T9+Ki022/eLHU+dRk3QgzZvz3jctMGtvFkCeIIQrr+psQ/wCx...

```

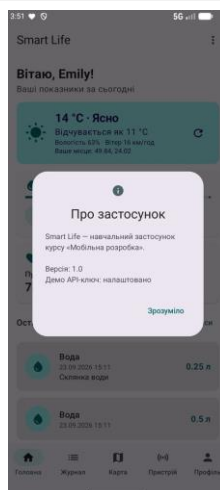


Рисунок 5.9 – Діалог «Про застосунок» з версією і параметрами з BuildConfig

На рисунку 5.9 поверх головного екрана відкрито діалог «Про застосунок». Він показує версію 1.0 і рядок «Демо API-ключ: налаштовано»: значення взято з класу `BuildConfig`, згенерованого під час збірки з файлу `local.properties`, а сам ключ у кодї й репозиторії відсутній.

5.6 Типові помилки

Сповіщення. Якщо сповіщення не з'являється на Android 13 і новіших, користувач не надав дозвіл `POST_NOTIFICATIONS`. Виняток *`IllegalArgumentOutOfRangeException: Targeting S+ requires FLAG_IMMUTABLE or FLAG_MUTABLE`* означає, що для `PendingIntent` не вказано прапорець змінюваності. Білий квадрат замість іконки з'являється, коли для малої іконки використано кольорове зображення. Нарешті, змінити важливість каналу після його створення може лише користувач у налаштуваннях системи – під час розробки для цього простіше перевстановити застосунок.

Фонові задачі. Найчастіше студенти дивуються, що задача не запускається частіше ніж раз на 15 хвилин: це мінімальний інтервал періодичних задач, тому для перевірки використовуйте разову задачу («Перевірити зараз»). Якщо задачі дублюються, замість `enqueueUniquePeriodicWork` використано звичайний `enqueue`. Після перейменування класу воркера старі заплановані задачі падають з *`ClassNotFoundException`* – скасуйте їх або перевстановіть застосунок. На телефонах деяких виробників задачі не виконуються через агресивне енергозбереження; у такому разі застосунок потрібно додати у виключення оптимізації батареї.

Авторизація. Якщо кожен запит повертає 401, у заголовку забуто префікс `Bearer` або використовується прострочений токен. Якщо після входу кнопка «Назад» повертає на екран входу, маршрут `LoginRoute` не видалено зі стеку. «Блимання» екрана входу під час запуску виникає, коли стартовий маршрут обирається раніше, ніж прочитано сесію, – поки стан невідомий (`null`), слід показувати індикатор завантаження. І головне правило безпеки: пароль на пристрої не зберігається, зберігаються лише токени.

Безпека. Виняток *`InvalidAlgorithmParameterException: Caller-provided IV not permitted`* виникає при спробі задати власний IV для ключа з `Keystore` – використовуйте `cipher.iv`, згенерований системою. Повідомлення *`CLEARTEXT communication not permitted`* означає, що запит іде через `http://`, а політика мережевої безпеки дозволяє лише HTTPS. Якщо після ввімкнення R8 реліз падає під час серіалізації, R8 видалив або перейменував потрібні класи – додайте правила в `proguard-rules.pro`. І якщо `keystore` чи паролі потрапили в `git`, негайно замініть їх: файли ключів потрібно додавати в `.gitignore` ще до першого коміту.

5.7 Практичні завдання

Практичні завдання розділу наведено в таблиці 5.7. Основне завдання виконується на основі версії Smart Life, отриманої в підрозділі, а завдання для самостійної роботи поглиблює тему і може оцінюватися додатково.

Таблиця 5.7 – Практичні завдання розділу

Тема	Практичне завдання	Завдання для самостійної роботи
Сповіщення	Додайте до нагадування другу дію «Нагадати пізніше», яка приховує сповіщення і показує його знову через 10 хвилин (підрозділ 5.3).	Створіть сповіщення з прогрес-баром (<code>setProgress</code>), що показує, яку частину денної норми води випито.
Фонові задачі	Додайте щоденну задачу, яка о 21:00 показує підсумок дня (вода, пульс, сон), розрахувавши початкову затримку <code>setInitialDelay</code> .	Створіть задачу синхронізації записів із сервером з обмеженням <code>NetworkType.CONNECTED</code> і ланцюжком <code>beginWith(...).then(...)</code> .
Авторизація	Додайте на екран входу перемикач «Запам'ятати мене»: якщо його вимкнено, сесія очищається при закритті застосунку.	Реалізуйте автоматичне оновлення токена для всіх запитів через <code>OkHttp Authenticator</code> .
Безпека	Зашифруйте нотатки записів у базі <code>Room</code> за допомогою <code>TokenCipher</code> і переконайтеся в <code>Database Inspector</code> , що вони нечитабельні.	Додайте біометричне підтвердження входу (<code>BiometricPrompt</code>) перед відкриттям застосунку з активною сесією.

5.8 Контрольні питання

1. Навіщо потрібні канали сповіщень?
2. Чим відрізняються рівні важливості DEFAULT і HIGH?
3. Що таке PendingIntent?
4. Як обробити натискання кнопки дії без відкриття застосунку?
5. Для чого `goAsync()`?
6. Чому не можна використовувати звичайні потоки для тривалої фонові роботи?
7. Що таке режим Doze?
8. Коли обирати WorkManager, а коли Foreground Service?
9. Навіщо потрібні унікальні задачі?
10. Що означають результати `retry()` і `failure()`?
11. Чим автентифікація відрізняється від авторизації?
12. Навіщо два токени – access і refresh?
13. Як передається токен у запиті?
14. Що робить застосунок при відповіді 401?
15. Чому валідації на клієнті недостатньо?
16. Що таке пісочниця застосунку?
17. Чому ключ шифрування зберігають в Android Keystore?
18. Навіщо для кожного шифрування новий IV?
19. Як заборонити незашифрований HTTP?
20. Що станеться, якщо втратити ключ підпису?

РОЗДІЛ 6. ТЕСТУВАННЯ ТА ФІНАЛЬНИЙ ПРОЄКТ «SMART LIFE»

6.1 Мета розділу та результати навчання

Мета розділу – навчити автоматично перевіряти логіку та інтерфейс застосунку, узагальнити матеріал курсу на прикладі фінальної версії Smart Life 1.0 і підготувати застосунок до випуску. Результати навчання наведено в таблиці 6.1, основні поняття – в таблиці 6.2.

Таблиця 6.1 – Результати навчання за темами розділу

Тема	Що студент знає і вміє після опрацювання
Тестування	Піраміду тестування; різницю між локальними (test) та інструментальними (androidTest) тестами; JUnit 4 і структуру тесту Arrange – Act – Assert; тестові двійники (fake); тестування корутин (runTest, тестовий диспетчер, MainDispatcherRule); Compose UI-тести (createComposeRule, onNodeWithText, performClick, assertIsDisplayed); запуск тестів з Android Studio і Gradle.
Фінальний проєкт	Архітектуру застосунку в цілому і зв'язок компонентів; як додати нову функцію, не порушуючи архітектуру; як підготувати реліз (версія, R8, підпис, перевірка); як представити і захистити проєкт.

Таблиця 6.2 – Основні поняття розділу

Поняття	Визначення
Unit-тест	тест окремої функції або класу, ізолюваний від Android
інструментальний тест	тест, що виконується на пристрої або емуляторі
fake	спрощена робоча реалізація залежності для тестів
runTest	запуск тесту з корутинами у віртуальному часі
семантичне дерево	опис елементів Compose, за яким їх знаходять тести
Реліз	версія застосунку для користувачів: оптимізована і підписана
versionCode / versionName	внутрішній номер збірки / номер версії для людей
APK / AAB	файл встановлення / формат публікації в Google Play
контрольний список	перелік перевірок перед випуском

6.2 Тестування мобільного застосунку

Піраміда тестування: в основі – багато швидких **unit-тестів** (перевіряють окремі функції і класи на JVM комп'ютера, мілісекунди), вище – менше **інтеграційних** і **UI-тестів** (запускаються на емуляторі, секунди), на вершині – ручне тестування. Локальні тести лежать у app/src/test, інструментальні – в app/src/androidTest.

Тест складається з трьох частин: **Arrange** (підготувати дані), **Act** (виконати дію), **Assert** (перевірити результат). Щоб тестувати ViewModel без бази, мережі і GPS, залежності замінюють **фейками** – простими реалізаціями тих самих інтерфейсів (InMemoryRecordRepository, FakeLocationSource, FakeHealthAlerts). Саме тому в попередніх розділах ViewModel залежали від інтерфейсів.

viewModelScope використовує головний диспетчер Dispatchers.Main, якого немає на JVM, тому в тестах його підміняють правилом MainDispatcherRule; runTest виконує корутини у віртуальному часі, а advanceUntilIdle() чекає їх завершення. UI-тести Compose знаходять елементи за текстом або описом (contentDescription) і виконують дії, як користувач; тестують функції ...Content, які не залежать від ViewModel.

Види тестів, що застосовуються в Android-розробці, порівняно в таблиці 6.3. Чим вище тест у піраміді, тим ближче він до реального використання, але тим повільніший, дорожчий у підтримці і більш схильний до випадкових збоїв. Тому основну логіку перевіряють unit-тестами, а UI-тести пишуть для ключових сценаріїв.

Таблиця 6.3 – Види тестів Android-застосунку

Вид тесту	Де виконується	Швидкість	Інструменти	Що перевіряє в Smart Life
Unit-тест	JVM комп'ютера (src/test)	мілісекунди	JUnit, kotlinx-coroutines-test	розбір пульсу, підсумки дня,

Вид тесту	Де виконується	Швидкість	Інструменти	Що перевіряє в Smart Life
				валідація, ViewModel
Інтеграційний	емулятор або JVM	секунди	Room in-memory, MockWebServer	робота DAO з базою (завдання)
UI-тест	емулятор або телефон (src/androidTest)	секунди	Compose UI Test, Espresso	екрани входу і журналу
Ручне тестування	телефон	хвилини	контрольний список	BLE, GPS, камера, фонові задачі

Якість тестів оцінюють не лише кількістю. Добрий тест перевіряє одну поведінку, має зрозумілу назву, не залежить від інших тестів і від порядку запуску, а також дає однаковий результат при кожному запуску. Для оцінки повноти використовують **покриття коду** – частку рядків, виконаних під час тестів, яку можна отримати в Android Studio командою *Run with Coverage*.

Практичний приклад. Smart Life має 27 unit-тестів (розбір пакета пульсу, підсумки дня, повідомлення про мережеві помилки, валідація входу і реєстрації, обчислення руху, RecordsViewModel, RecordDetailsViewModel) і 8 UI-тестів екранів входу та журналу.

Спершу підключіть бібліотеки для тестів. Для локальних тестів (testImplementation) потрібні junit і kotlinx-coroutines-test, для інструментальних (androidTestImplementation) – androidx.test.ext:junit, espresso-core і compose-ui-test-junit4. Потім у папці app/src/test/... створіть правило MainDispatcherRule, яке підміняє головний диспетчер корутин на час кожного тесту.

Далі пишіть тести від простого до складного: спочатку чисті функції (розбір пакета пульсу, підсумки дня, перевірка форм), потім ViewModel з фейковими залежностями. UI-тести для функцій LoginContent і RecordsContent розмістіть у папці app/src/androidTest/.... Запускати тести можна зеленою стрілкою біля класу або методу тесту в Android Studio, а також командами ./gradlew testDebugUnitTest і ./gradlew connectedDebugAndroidTest.

Ключові фрагменти реалізації наведено нижче.

```
// test/.../ui/records/RecordsViewModelTest.kt - unit-тест ViewModel з фейками
@get:Rule val mainDispatcherRule = MainDispatcherRule()

@Test
fun `high pulse shows alert, normal pulse does not`() =
    runTest(mainDispatcherRule.testDispatcher) {
        val alerts = FakeHealthAlerts() // Arrange
        val viewModel = RecordsViewModel(InMemoryRecordRepository(),
            FakeLocationSource(null), alerts)

        viewModel.addRecord(RecordType.PULSE, 72.0, "") // Act
        viewModel.addRecord(RecordType.PULSE, 118.0, "")
        advanceUntilIdle()

        assertEquals(listOf(118), alerts.shownPulses) // Assert
    }
// ... added record is saved with current location:
// FakeLocationSource(lviv) -> latitude 49.8397

// androidTest/.../ui/records/RecordsContentTest.kt - UI-тест Compose
@get:Rule val composeTestRule = createComposeRule()

@Test
fun recordsAreDisplayed() {
    val records = listOf(HealthRecord(1, RecordType.PULSE, 72.0, "спокій"),
        HealthRecord(2, RecordType.WATER, 0.5, ""))
    composeTestRule.setContent {
        SmartLifeTheme {
            RecordsContent(records, onAddRecord = { , , -> }, onRecordClick = {})
        }
    }
    composeTestRule.onNodeWithText("72 уд/хв").assertIsDisplayed()
    composeTestRule.onNodeWithText("Журнал показників (2)").assertIsDisplayed()
}
```

Правило `MainDispatcherRule`, підключене анотацією `@get:Rule`, перед кожним тестом замінює `Dispatchers.Main` тестовим диспетчером, а після тесту повертає справжній. Без нього `ViewModel` не змогла б запустити жодну корутину на JVM комп'ютера, де головного потоку `Android` немає.

Тест `high pulse shows alert, normal pulse does not` має класичну структуру з трьох частин. У частині `Arrange` створюються фейкові залежності: репозиторій у пам'яті, джерело координат, що повертає `null`, і `FakeHealthAlerts`, який лише запам'ятовує, для яких значень пульсу було показано попередження. У частині `Act` тест додає два записи пульсу – 72 і 118 уд/хв – і викликає `advanceUntilIdle()`, щоб дочекатися завершення корутин `viewModelScope`. У частині `Assert` перевіряється, що попередження показано рівно один раз і саме для 118. Функція `runTest` виконує все це у віртуальному часі, тому тест триває мілісекунди.

UI-тест `recordsAreDisplayed` через `composeTestRule.setContent` показує лише функцію `RecordsContent` з одним тестовим записом – без бази, `ViewModel` і навігації. Потім `onNodeWithText("72 уд/хв")` знаходить у семантичному дереві елемент з таким текстом, а `assertIsDisplayed()` перевіряє, що він видимий на екрані. Так само інші тести знаходять кнопки за описом (`contentDescription`) і натискають їх методом `performClick()`.

Запуск на емуляторі. Unit-тести запускаються без емулятора. Для UI-тестів запустіть емулятор і виконайте `./gradlew connectedDebugAndroidTest`; звіт з'явиться в `app/build/reports/androidTests/connected/index.html`.

Реальний результат запуску тестів (фрагмент звіту):

```
RecordsViewModelTest          tests=4 failures=0
  PASSED: added record is saved with current location
  PASSED: high pulse shows alert, normal pulse does not
MotionTest                    tests=3 failures=0
TOTAL 27 FAILED 0
RecordsContentTest            4 tests, 0 failures 11.443 s
  PASSED recordsAreDisplayed          2.107 s
  PASSED invalidValueDisablesSaveButton 4.555 s
BUILD SUCCESSFUL in 2m 54s
```

У звіті видно, що всі 27 локальних тестів пройшли успішно (TOTAL 27 FAILED 0), серед них перевірки збереження координат і попередження про пульс. Інструментальні тести `RecordsContentTest` виконувалися на емуляторі кілька секунд кожен – це значно довше за unit-тести і пояснює, чому в піраміді тестування їх має бути менше.

6.3 Фінальний проєкт Smart Life 1.0

`Smart Life 1.0` – застосунок персонального моніторингу здоров'я, у якому поєднано всі теми курсу. Архітектуру показано на рисунку 6.1: екрани `Compose` відображають стан своїх `ViewModel`; `ViewModel` звертаються до репозиторіїв і менеджерів пристрою, які створює `AppContainer`; дані зберігаються в `Room`, `DataStore` та `Android Keystore`, мережеві дані надходять з `Open-Meteo` і `DummyJSON`. Кожен шар можна змінити незалежно, а бізнес-логіку – перевірити тестами.

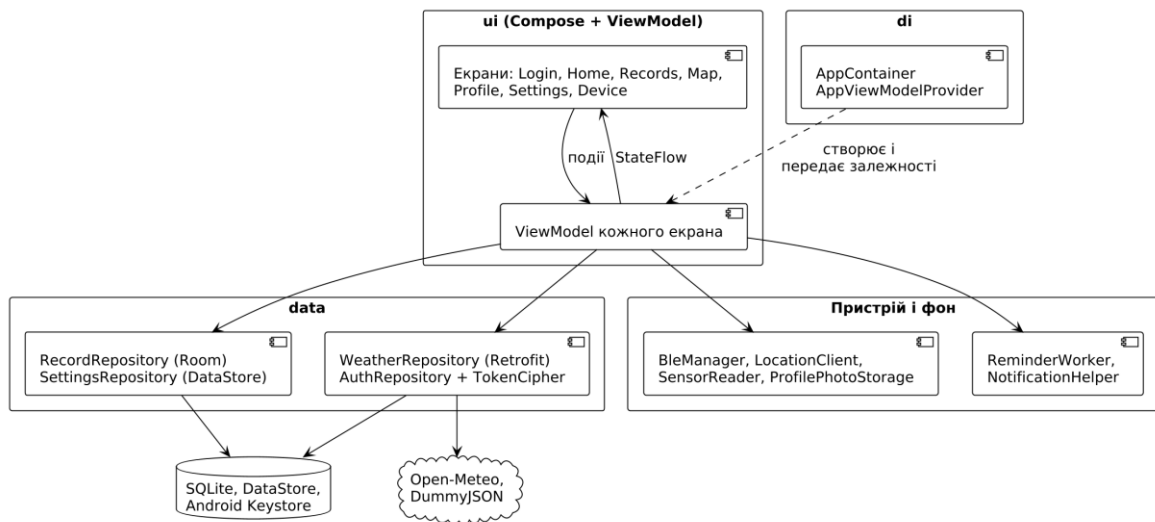


Рисунок 6.1 – Архітектура застосунку Smart Life 1.0

Рисунок 6.1 показує застосунок як набір пов'язаних блоків. У верхньому блоці ui екрани передають події своїм ViewModel і отримують від них стан через StateFlow. Блок di створює ViewModel і передає їм залежності. ViewModel звертаються до трьох груп компонентів: репозиторіїв локальних даних (Room і DataStore), мережевих репозиторіїв (Retrofit, авторизація з шифруванням) і компонентів пристрою та фону (Bluetooth, геолокація, сенсори, фото, воркер нагадувань і сповіщення). Унизу показано, де фізично зберігаються дані: база SQLite, DataStore і Android Keystore на пристрої та сервіси Open-Meteo і DummyJSON в інтернеті.

Можливості фінальної версії, використані технології та підрозділи посібника, у яких їх реалізовано, наведено в таблиці 6.4.

Таблиця 6.4 – Можливості Smart Life 1.0

Можливість	Технології	Підрозділ
вхід, реєстрація, сесія	Retrofit, DataStore, Keystore	5.4, 5.5
журнал показників (CRUD)	Room, Flow, MVVM	2.3, 2.4, 3.3
головна: підсумки дня, вода, погода	StateFlow, Retrofit, геолокація	3.4, 4.4
карта записів	osmdroid, геолокація	4.4
пристрій: BLE, геолокація, сенсори	Bluetooth LE, Fused Location, SensorManager	4.3, 4.4, 4.6
профіль з фото	ActivityResult API, FileProvider, Coil	4.5
налаштування, тема, нагадування	DataStore, сповіщення, WorkManager	3.3, 5.2, 5.3

Практичний приклад. Фінальна версія (versionName "1.0") містить п'ять вкладок: «Головна», «Журнал», «Карта», «Пристрій», «Профіль». Структуру пакетів наведено в додатку А.

Підготовку релізу почніть з версії: в app/build.gradle.kts задайте versionCode = 1 і versionName = "1.0". Потім запустіть усі тести з підрозділу 6.2 і перевірте код інструментом Code ▶ Inspect Code, виправивши попередження Lint.

Коли код готовий, зберіть підписаний реліз командою ./gradlew assembleRelease або через майстер Build ▶ Generate Signed App Bundle or APK. Встановіть отриманий APK на емулятор або телефон командою adb install app-release.apk і пройдіть контрольний список, наведений нижче, – саме на релізній збірці, бо R8 може змінити її поведінку. Наостанок позначте версію в репозиторії міткою git tag v1.0.

Ключові фрагменти реалізації наведено нижче.

```
// ui/SmartLifeApp.kt - точка збирання застосунку
@Composable
fun SmartLifeApp(
    sessionViewModel: SessionViewModel = viewModel(factory = AppViewModelProvider.Factory)
) {
    val isLoggedIn by sessionViewModel.isLoggedIn.collectAsStateWithLifecycle()
    if (isLoggedIn == null) { /* ... індикатор, поки читається сесія */ return }
    val startDestination: Any = remember { if (isLoggedIn == true) HomeRoute else LoginRoute }
```

```
// ... Scaffold з NavigationBar для 5 вкладок і NavHost з 12 маршрутами (рис. 2.1)
}

// di/AppContainer.kt - усі залежності застосунку
class AppContainer(private val context: Context) {
    val recordRepository: RecordRepository by lazy {
        RoomRecordRepository(database.recordDao())
    }
    val settingsRepository by lazy { SettingsRepository(context) }
    val weatherRepository: WeatherRepository by lazy { /* Retrofit + Open-Meteo */ }
    val authRepository by lazy { /* Retrofit + DummyJSON + SessionStorage(TokenCipher) */ }
    // ... bleManager, locationClient, sensorReader, profilePhotoStorage,
    //      notificationHelper, reminderScheduler
}
```

Функція SmartLifeApp – єдина composable-функція, яку викликає MainActivity. Вона отримує SessionViewModel через спільну фабрику і підписується на isLoggedIn. Поки сесію ще не прочитано з DataStore, значення дорівнює null, і функція показує лише індикатор завантаження, завершуючись оператором return. Коли стан відомий, у remember один раз обирається стартовий маршрут – головний екран для авторизованого користувача або екран входу. Далі йдуть Scaffold з нижньою панеллю на п'ять вкладок і NavHost з дванадцятьма маршрутами, граф яких показано на рисунку 2.1.

AppContainer у фінальній версії – це повний перелік залежностей застосунку. Репозиторій записів працює з Room, налаштування – з DataStore, погода і авторизація – з Retrofit, причому авторизація використовує сховище сесії з шифруванням. Решту властивостей – менеджер Bluetooth, клієнт геолокації, читач сенсорів, сховище фото, помічник сповіщень і планувальник нагадувань – позначено коментарем. Усі вони створюються через by lazy лише тоді, коли знадобляться.

Будь-яку нову функцію до застосунку додають за однією схемою: створюють джерело даних або менеджер пристрою, реєструють його в контейнері, пишуть ViewModel з StateFlow, екран у вигляді пари ...Screen і ...Content, додають маршрут у NavHost і покривають логіку тестами.

Запуск на емуляторі. Контрольний список: вхід (emilys / emilyspass) і вихід; додавання, редагування, видалення запису; збереження даних після перезапуску; погода з мережею і без неї; дозволи геолокації, камери, Bluetooth, сповіщень (надання і відмова); карта з маркерами; фото профілю; сенсори; тестове сповіщення і фонове нагадування; темна тема; поворот екрана; реліз встановлюється і працює. Перевірку на телефоні описано в додатку Г.

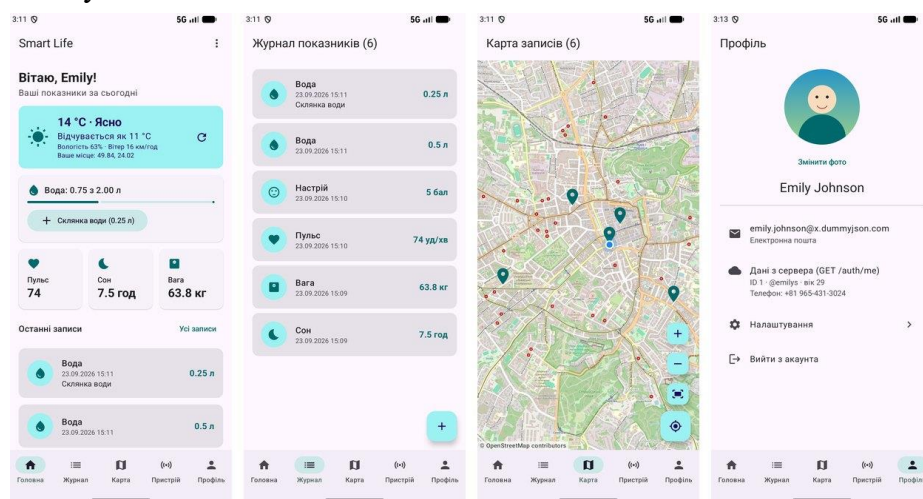


Рисунок 6.2 – Smart Life 1.0: головний екран, журнал, карта записів, профіль

На рисунку 6.2 показано чотири основні вкладки фінальної версії. Головний екран вітає користувача Emily, показує погоду для його місця («14 °C · Ясно»), картку води з кнопкою «Склянка води», підсумки пульсу, сну і ваги та останні записи. Журнал містить шість записів різних типів і кнопку додавання. Карта показує маркери записів на карті

Львова і поточну позицію. Профіль відображає фото, ім'я та email з сервера, дані запиту GET /auth/me, перехід до налаштувань і кнопку виходу.

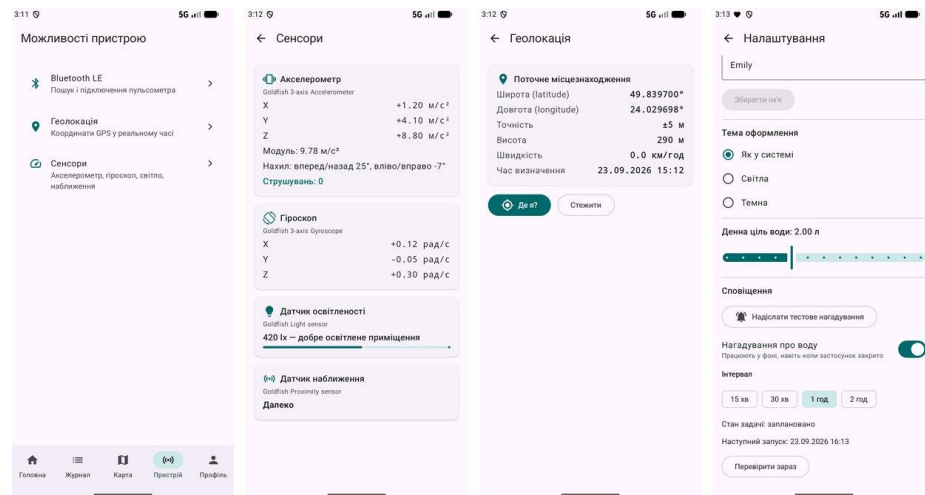


Рисунок 6.3 – Smart Life 1.0: пристрій, сенсори, геолокація, налаштування

На рисунку 6.3 показано вкладку «Пристрій» зі списком можливостей (Bluetooth LE, геолокація, сенсори), екран сенсорів з показаннями акселерометра, гіроскопа, освітленості і наближення, екран геолокації з поточними координатами і точністю ± 5 м та екран налаштувань, де обрано системну тему, денну ціль 2 л і ввімкнено нагадування про воду з інтервалом 1 год.

6.4 Типові помилки

Тестування. Помилка *Module with the Main dispatcher is missing* з'являється, коли в unit-тесті ViewModel не підмінено Dispatchers.Main, – підключіть MainDispatcherRule. Якщо тест StateFlow, створеного через stateIn(WhileSubscribed), отримує лише початкове значення, у потоку немає підписника: запустіть збір у backgroundScope з UnconfinedTestDispatcher. Повідомлення *No compose hierarchies found* означає, що не викликано setContent або тест помилково розміщено в test замість androidTest. А якщо інструментальні тести падають на новій версії Android, оновіть espresso-core і бібліотеки androidx.test.

Реліз. Якщо реліз падає, а debug-версія працює, R8 видалив потрібні класи: перевірте правила ProGuard і завжди тестуйте саме релізну збірку. Якщо мережа працює лише в debug-версії, поля BuildConfig задано тільки для типу збірки debug. Новий екран, який звертається до бази прямо з composable-функції, порушує архітектуру – дотримуйтеся описаної вище схеми додавання функцій. А якщо нова версія не встановлюється поверх попередньої, не збільшено versionCode або реліз підписано іншим ключем.

6.5 Практичні завдання

Практичні завдання розділу наведено в таблиці 6.5. Основне завдання виконується на основі версії Smart Life, отриманої в підрозділі, а завдання для самостійної роботи поглиблює тему і може оцінюватися додатково.

Таблиця 6.5 – Практичні завдання розділу

Тема	Практичне завдання	Завдання для самостійної роботи
Тестування	Напишіть unit-тести для RegisterViewModel.validate() (некоректний email, короткий пароль, порожні поля) і UI-тест екрана налаштувань.	Напишіть інструментальний тест RecordDao з базою в пам'яті (Room.inMemoryDatabaseBuilder).
Фінальний проєкт	Виконайте індивідуальне завдання курсового проєкту (додаток Ж) на основі Smart Life або власного застосунку з тими самими технологіями.	Підготуйте застосунок до публікації в Google Play: зберіть AAB (./gradlew bundleRelease), створіть іконку.

6.6 Контрольні питання

1. Що таке піраміда тестування?
2. Чим unit-тест відрізняється від інструментального?
3. Навіщо фейки і чому ViewModel мають залежати від інтерфейсів?
4. Як тестувати код з корутинами?
5. Як знайти елемент у Compose UI-тесті?
6. З яких шарів складається архітектура Smart Life?
7. Яка роль AppContainer?
8. Які кроки потрібні для додавання нової функції?
9. Що перевіряють перед випуском релізу?
10. Чим APK відрізняється від AAB?

ВИСНОВКИ

У посібнику послідовно розглянуто повний цикл створення нативного Android-застосунку мовою Kotlin з інтерфейсом на Jetpack Compose – від першого проєкту HelloMobile до застосунку Smart Life 1.0, що поєднує основні можливості сучасного смартфона.

Студент, що пройшов курс, набуває практичних навичок роботи в Android Studio та з емулятором; володіє мовою Kotlin (null-безпека, data class, колекції, лямбди, sealed-ієрархії, корутини і Flow); уміє будувати декларативний інтерфейс на Compose з навігацією і правильною роботою зі станом; проєктує застосунок за архітектурою MVVM з репозиторіями і впровадженням залежностей; працює з локальними даними (Room, DataStore, файли) і REST API з обробкою помилок; використовує Bluetooth LE, геолокацію, карти, камеру і сенсори з коректним запитом дозволів; створює сповіщення і фонові задачі WorkManager; застосовує базові заходи безпеки (токени, Android Keystore, HTTPS, R8, підпис релізу) і пише unit- та UI-тести.

Головний результат курсу – власноруч створений, протестований і задокументований застосунок, а також уміння самостійно освоювати нові API за офіційною документацією. Подальше навчання доцільно спрямувати на Hilt або Koin, Kotlin Multiplatform, push-сповіщення (Firebase Cloud Messaging), CameraX і ML Kit, а також на публікацію застосунку в Google Play.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. ДСТУ 3017:2015. Інформація та документація. Видання. Основні види. Терміни та визначення понять. [Чинний від 2016-07-01]. Вид. офіц. Київ : ДП «УкрНДНЦ», 2016.
2. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. [Чинний від 2016-07-01]. Вид. офіц. Київ : ДП «УкрНДНЦ», 2016.
3. Android Developers. Documentation. URL: <https://developer.android.com/docs> (дата звернення: 23.09.2026).
4. Android Developers. Guide to app architecture. URL: <https://developer.android.com/topic/architecture> (дата звернення: 23.09.2026).
5. Android Developers. Jetpack Compose. URL: <https://developer.android.com/develop/ui/compose> (дата звернення: 23.09.2026).
6. Android Developers. Navigation with Compose. URL: <https://developer.android.com/develop/ui/compose/navigation> (дата звернення: 23.09.2026).
7. Android Developers. ViewModel overview. URL: <https://developer.android.com/topic/libraries/architecture/viewmodel> (дата звернення: 23.09.2026).
8. Android Developers. Save data in a local database using Room. URL: <https://developer.android.com/training/data-storage/room> (дата звернення: 23.09.2026).
9. Android Developers. DataStore. URL: <https://developer.android.com/topic/libraries/architecture/datastore> (дата звернення: 23.09.2026).
10. Android Developers. Bluetooth Low Energy. URL: <https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview> (дата звернення: 23.09.2026).
11. Android Developers. Request location permissions. URL: <https://developer.android.com/develop/sensors-and-location/location/permissions> (дата звернення: 23.09.2026).
12. Android Developers. Sensors overview. URL: https://developer.android.com/develop/sensors-and-location/sensors/sensors_overview (дата звернення: 23.09.2026).
13. Android Developers. Create a notification. URL: <https://developer.android.com/develop/ui/views/notifications/build-notification> (дата звернення: 23.09.2026).
14. Android Developers. WorkManager. URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата звернення: 23.09.2026).
15. Android Developers. Android Keystore system. URL: <https://developer.android.com/privacy-and-security/keystore> (дата звернення: 23.09.2026).
16. Android Developers. Network security configuration. URL: <https://developer.android.com/privacy-and-security/security-config> (дата звернення: 23.09.2026).
17. Android Developers. Test apps on Android. URL: <https://developer.android.com/training/testing> (дата звернення: 23.09.2026).
18. Android Developers. Photo picker. URL: <https://developer.android.com/training/data-storage/shared/photo-picker> (дата звернення: 23.09.2026).
19. Kotlin Documentation. URL: <https://kotlinlang.org/docs/home.html> (дата звернення: 23.09.2026).

20. Kotlin Documentation. Coroutines guide. URL: <https://kotlinlang.org/docs/coroutines-guide.html> (дата звернення: 23.09.2026).
21. Kotlin Serialization Guide. URL: <https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/serialization-guide.md> (дата звернення: 23.09.2026).
22. Retrofit. A type-safe HTTP client for Android and Java. URL: <https://square.github.io/retrofit/> (дата звернення: 23.09.2026).
23. OkHttp. URL: <https://square.github.io/okhttp/> (дата звернення: 23.09.2026).
24. Coil. Image loading for Android and Compose Multiplatform. URL: <https://coil-kt.github.io/coil/> (дата звернення: 23.09.2026).
25. osmdroid. OpenStreetMap-Tools for Android. URL: <https://github.com/osmdroid/osmdroid> (дата звернення: 23.09.2026).
26. OpenStreetMap. Copyright and License. URL: <https://www.openstreetmap.org/copyright> (дата звернення: 23.09.2026).
27. Maps Compose Library. Google Maps Platform. URL: <https://developers.google.com/maps/documentation/android-sdk/maps-compose> (дата звернення: 23.09.2026).
28. Open-Meteo. Weather Forecast API. URL: <https://open-meteo.com/en/docs> (дата звернення: 23.09.2026).
29. DummyJSON. Auth. URL: <https://dummyjson.com/docs/auth> (дата звернення: 23.09.2026).
30. Bluetooth SIG. Assigned Numbers. URL: <https://www.bluetooth.com/specifications/assigned-numbers/> (дата звернення: 23.09.2026).
31. Bluetooth SIG. Heart Rate Service 1.0. URL: <https://www.bluetooth.com/specifications/specs/heart-rate-service-1-0/> (дата звернення: 23.09.2026).
32. OWASP. Mobile Application Security Verification Standard (MASVS). URL: <https://mas.owasp.org/MASVS/> (дата звернення: 23.09.2026).
33. Material Design 3. URL: <https://m3.material.io> (дата звернення: 23.09.2026).
34. PlantUML. Language Reference Guide. URL: <https://plantuml.com/guide> (дата звернення: 23.09.2026).
35. PlantText : онлайн-редактор діаграм PlantUML. URL: <https://www.planttext.com> (дата звернення: 23.09.2026).

ДОДАТОК А. СТРУКТУРА ФІНАЛЬНОГО ПРОЄКТУ

Kotlin-код застосунку знаходиться в `app/src/main/java/ua/edu/mobile/smartlife/`, ресурси – в `app/src/main/res/`, unit-тести – в `app/src/test/`, UI-тести – в `app/src/androidTest/`. Повний код кожного файлу – у репозиторії SmartLife.

Таблиця А.1 – Пакети застосунку Smart Life 1.0

Пакет	Призначення	Основні класи
(корінь), di	точка входу, залежності	MainActivity, SmartLifeApplication, AppContainer
data.model	моделі предметної області	HealthRecord, RecordType, DailySummary, Weather
data.local	база даних Room	AppDatabase, RecordEntity, RecordDao
data.repository	репозиторії	RecordRepository, RoomRecordRepository, WeatherRepository, AuthRepository
data.remote	мережа	NetworkModule, WeatherApi, AuthApi, DTO, NetworkErrors
data.settings, data.auth, data.security, data.profile	налаштування, сесія, шифрування, фото	SettingsRepository, SessionStorage, TokenCipher, ProfilePhotoStorage
ble, location, sensors	можливості пристрою	BleManager, LocationClient, SensorReader
notifications, work	сповіщення, фонові задачі	NotificationHelper, WaterActionReceiver, ReminderWorker, ReminderScheduler
ui, ui.navigation, ui.theme, ui.components	корінь UI, маршрути, тема, спільні компоненти	SmartLifeApp, AppViewModelProvider, Routes.kt, Permissions.kt
ui.auth, ui.home, ui.records, ui.map, ui.device, ui.ble, ui.location, ui.sensors, ui.profile, ui.settings	екрани та їх ViewModel	*Screen, *ViewModel

ДОДАТОК Б. ІНСТРУКЦІЯ З ЗАПУСКУ ПРОЄКТУ

Вимоги: Android Studio 2025.3.3 Patch 1 (Panda 3) або новіша, 16 ГБ оперативної пам'яті (рекомендовано), 20 ГБ вільного місця, інтернет під час першої збірки.

Порядок запуску:

1. Відкрийте папку `SmartLife` через *File* ▶ *Open* і дочекайтеся Gradle Sync.
2. У *Device Manager* створіть емулятор з Android 15–17.
3. Натисніть *Run*.
4. Увійдіть як `emilys / emilyspass`.

Збірка з командного рядка: `./gradlew assembleDebug` (Windows: `gradlew.bat assembleDebug`), тести – `./gradlew testDebugUnitTest`. Стан проєкту після будь-якого розділу: `git checkout stage-NN`, повернення до фінальної версії – `git checkout v1.0`. Якщо збірка не вдається: перевірте JDK 17+ (*Settings* ▶ *Build Tools* ▶ *Gradle* ▶ *Gradle JDK*), виконайте *File* ▶ *Invalidate Caches* і повторіть Sync; якщо на емуляторі немає інтернету – перезапустіть його з параметром `-dns-server 8.8.8.8`.

ДОДАТОК В. СТВОРЕННЯ APK

Debug-APK: *Build* ▶ *Build App Bundle(s) / APK(s)* ▶ *Build APK(s)* або `./gradlew assembleDebug`; файл `app/build/outputs/apk/debug/app-debug.apk` підписано налагоджувальним ключем і придатний лише для тестування. **Release-APK:** *Build* ▶ *Generate Signed App Bundle or APK* ▶ *APK*, оберіть або створіть keystore (підрозділ 5.5), варіант `release`; або створіть `keystore.properties` (`storeFile`, `storePassword`, `keyAlias`, `keyPassword`) і виконайте `./gradlew assembleRelease` — файл `app/build/outputs/apk/release/app-release.apk`. Перевірка підпису: `apksigner verify --print-certs app-release.apk`; встановлення: `adb install -r app-release.apk`. Для Google Play збирають **ААВ**: `./gradlew bundleRelease`. Файл keystore і паролі зберігайте поза репозиторієм і робіть резервну копію.

ДОДАТОК Г. ТЕСТУВАННЯ НА ФІЗИЧНОМУ ANDROID-ПРИСТРОЇ

1. На телефоні відкрийте *Налаштування* ► *Про телефон* і сім разів натисніть «Номер збірки» – з'явиться меню «Для розробників».
2. Увімкніть у ньому «Налагодження USB» (або «Бездротове налагодження» для Android 11+).
3. Підключіть телефон кабелем і підтвердьте RSA-ключ комп'ютера.
4. Оберіть телефон у списку пристроїв Android Studio і натисніть *Run*.

На телефоні обов'язково перевірте: BLE-підключення до пульсометра або nRF Connect, реальний GPS на вулиці, фото з камери, струшування і датчик наближення, сповіщення на екрані блокування, фонові нагадування після кількох годин роботи з вимкненим екраном. У телефонах деяких виробників (Xiaomi, Huawei, Samsung) агресивне енергозбереження може зупиняти фонові задачі – додайте застосунок у виключення (див. dontkillmyapp.com).

ДОДАТОК Д. КОНТРОЛЬНІ ПИТАННЯ ДО КУРСУ

1. Порівняйте нативну і кросплатформну розробку.
2. Опишіть шлях від Kotlin-коду до APK на пристрої.
3. Поясніть життєвий цикл Activity і наслідки повороту екрана.
4. Які засоби Kotlin забезпечують null-безпеку?
5. Що таке рекомпозиція і як Compose визначає, що перемалювати?
6. Поясніть односпрямований потік даних і підйом стану.
7. Чому ViewModel переживає поворот екрана?
8. Опишіть шари архітектури і роль репозиторію.
9. Як працює ручне впровадження залежностей через контейнер?
10. Порівняйте DataStore, Room і файли.
11. Що таке міграція бази даних?
12. Опишіть HTTP-запит і коди статусу.
13. Як обробляти помилки мережі в застосунку?
14. Опишіть етапи взаємодії з BLE-пристроєм і структуру GATT.
15. Як правильно запитувати небезпечні дозволи?
16. Як Fused Location Provider визначає координати?
17. Як вбудувати карту в Compose і отримати ключ Google Maps?
18. Навіщо FileProvider і Photo Picker?
19. Як отримати дані акселерометра і чому треба відписуватися від сенсорів?
20. Що таке канали сповіщень і PendingIntent?
21. Коли використовувати WorkManager?
22. Як працює токена автентифікація з оновленням токена?
23. Як Android Keystore захищає ключі шифрування?
24. Які заходи захисту застосовують перед випуском релізу?
25. Опишіть піраміду тестування і тестування корутин.

ДОДАТОК Е. ПРАКТИЧНІ (ЛАБОРАТОРНІ) РОБОТИ

Таблиця Е.1 – Перелік лабораторних робіт

№	Назва роботи	Підрозділи	Результат, що здається
1	Середовище розробки і перший застосунок	1.2, 1.3	HelloMobile з кнопкою скидання
2	Основи Kotlin	1.4	консольна програма з тижневим звітом
3	Інтерфейс на Jetpack Compose	2.2	прототип Smart Life 0.3 + екран «Про застосунок»
4	Стан і архітектура	2.3, 2.4	версії 0.4–0.5 + фільтр журналу за типом
5	Корутини і локальні дані	3.2, 3.3	версія 0.6 + експорт журналу в CSV
6	Мережа і REST API	3.4	версія 0.7 + прогноз на 3 дні
7	Дозволи і Bluetooth LE	4.2, 4.3	версія 0.8; скриншоти підключення на телефоні
8	Геолокація і карта	4.4	версії 0.9–0.10 + «маршрут дня»
9	Камера і сенсори	4.5, 4.6	версії 0.11–0.12 + фото до запису, «рівень»
10	Сповіщення і фонові задачі	5.2, 5.3	версії 0.13–0.14 + «Нагадати пізніше»
11	Авторизація і безпека	5.4, 5.5	версії 0.15–0.16 + OkHttp Authenticator
12	Тестування	6.2	версія 0.17 + 5 власних тестів
13	Фінальний проєкт	6.3	Smart Life 1.0 або індивідуальний проєкт; звіт і захист

Звіт з лабораторної роботи містить: мету; короткі теоретичні відомості; хід виконання з фрагментами коду і власними скриншотами з підписами «Рисунок N – ...»; відповіді на контрольні питання; висновки; посилання на коміт у репозиторії.

ДОДАТОК Ж. ВАРІАНТИ ІНДИВІДУАЛЬНИХ ПРОЄКТІВ

Кожен варіант містить обов'язкове ядро: авторизацію, навігацію з нижньою панеллю, MVVM з репозиторіями, Room з CRUD, DataStore, REST API з обробкою помилок, щонайменше три можливості пристрою, сповіщення, фонову задачу WorkManager і не менше 10 unit-тестів.

Таблиця Ж.1 – Варіанти індивідуальних проєктів

№	Тема	Можливості пристрою	№	Тема	Можливості пристрою
1	Трекер пробіжок	GPS, карта, акселерометр, BLE	11	Спортивний зал	BLE-пульсометр, таймер, сповіщення
2	Щоденник харчування	камера, штрихкод, сповіщення	12	Метеостанція	барометр, освітленість, геолокація
3	Розумний будинок (демо)	BLE, освітленість, наближення	13	Бібліотека книг	камера, штрихкод ISBN
4	Туристичний путівник	карта, геолокація, камера	14	Сканер BLE-пристроїв	BLE (читання/запис), карта знахідок
5	Облік домашніх рослин	камера, освітленість, нагадування	15	Навчальний планувальник	сповіщення, WorkManager, календар
6	Прийом ліків	сповіщення з діями, камера	16	Волонтерський центр	карта, геолокація, камера
7	Велопрокат	карта, геолокація, QR-код	17	Контроль гідратації	акселерометр, сповіщення, віджет
8	Фото-щоденник подорожей	камера, галерея, карта	18	Квест-гра по місту	геозони, карта, камера
9	Трекер сну	акселерометр, освітленість, наближення	19	Моніторинг шуму і світла	мікрофон, освітленість, карта
10	Облік витрат	камера (чек), геолокація	20	Щоденник настрою	камера, геолокація, сповіщення

ДОДАТОК К. КРИТЕРІЇ ОЦІНЮВАННЯ ФІНАЛЬНОГО ПРОЄКТУ

Таблиця К.1 – Розподіл балів за фінальний проєкт

Критерій	Бали	Критерій	Бали
функціональність ядра	20	архітектура та якість коду	15
мережа та обробка помилок	10	безпека	5
можливості пристрою (≥ 3)	15	тестування	10
сповіщення і фонові задачі	10	UI/UX і доступність	5
звіт і документація	5	захист проєкту	5

Разом – 100 балів. Шкала: 90–100 – «відмінно» (A); 82–89 – «добре» (B); 74–81 – «добре» (C); 64–73 – «задовільно» (D); 60–63 – «задовільно» (E); 35–59 – FX; 0–34 – F. Умови допуску до захисту: проєкт збирається без помилок, не падає на основних сценаріях, має осмислену історію комітів, не містить секретів у репозиторії, а студент може пояснити будь-який фрагмент свого коду.

ДОДАТОК Л. ГЛОСАРІЙ ОСНОВНИХ ТЕРМІНІВ

Таблиця Л.1 – Глосарій

Термін	Визначення
Activity	компонент Android, що представляє екран і має життєвий цикл
APK / AAB	інсталяційний пакет Android / формат публікації в Google Play
BLE, GATT	енергоощадний Bluetooth / профіль даних BLE-пристрою
Composable-функція	функція з @Composable, що описує частину інтерфейсу
корутина	призупинювана асинхронна задача Kotlin
CRUD	створення, читання, оновлення, видалення даних
DI	передача залежностей об'єкту ззовні
DTO	клас, що повторює структуру JSON-відповіді
Flow / StateFlow	асинхронний потік значень / потік з поточним значенням
FileProvider	компонент, що видає content:// Uri на файли застосунку
JWT	підписаний токен з даними користувача
MVVM	архітектурний шаблон Model – View – ViewModel
PendingIntent	намір, що виконується системою від імені застосунку
R8	інструмент зменшення та обфускації коду релізу
рекомпозиція	повторний виклик composable-функцій після зміни стану
репозиторій	клас-посередник між ViewModel і джерелами даних
ViewModel	клас, що зберігає стан екрана і переживає поворот
WorkManager	бібліотека надійного планування відкладених фонових задач

ДОДАТОК М. ВИКОРИСТАНІ БІБЛІОТЕКИ

Таблиця М.1 – Бібліотеки фінального проєкту

Бібліотека	Версія	Призначення
Compose BOM, activity-compose, material-icons-extended	2026.09.00, 1.13.0, 1.7.8	інтерфейс, Activity Result API, іконки
navigation-compose, kotlinx-serialization-json	2.10.1, 1.11.0	навігація, JSON
lifecycle-viewmodel-compose, lifecycle-runtime-compose	2.11.0	ViewModel, collectAsStateWithLifecycle
Room + KSP, datastore-preferences	2.8.5 + 2.3.12, 1.2.1	база даних, налаштування
Retrofit, OkHttp logging-interceptor	3.0.0, 4.12.0	REST API, логування
play-services-location, kotlinx-coroutines-play-services	21.4.0, 1.11.0	геолокація, Task.await()
osmdroid-android, coil-compose, work-runtime-ktx	6.1.20, 3.6.3, 2.11.2	карта, зображення, фонові задачі
JUnit, kotlinx-coroutines-test, espresso-core, compose ui-test	4.13.2, 1.11.0, 3.7.0, за BOM	тестування

Зовнішні сервіси: Open-Meteo (погода, без ключа), DummyJSON (навчальна авторизація), OpenStreetMap (дані карти, ліцензія ODbL). Вихідні коди схем посібника (PlantUML) знаходяться в папці `diagrams/`; кожен файл `*.puml` можна вставити в редактор planttext.com для перегляду і редагування.